

The Backend Engineer's Library

Signing, Provenance, and Attestation

Volume C05

Contents

Cryptographic Foundations for Supply Chain Security
Classic Code Signing and Its Failure Modes
Sigstore Architecture: Cosign, Fulcio, Rekor
Keyless Signing and Workload Identity
Transparency Logs: Merkle Trees, Rekor, and CT Lessons
in-toto: Attestations, Layouts, and Policies
TUF: The Update Framework
Provenance Verification in Practice
Key Management and PKI for the Enterprise
Designing Attestation-Based Deployment Gates

Chapter 1 — Cryptographic Foundations for Supply Chain Security

What this chapter covers. This book is about proving things about software: that an artifact came from a build you trust, that it has not been altered since, that a specific identity vouched for it, and that these facts are recorded somewhere no one can quietly rewrite. Every one of those guarantees rests on a small set of cryptographic primitives — hash functions, digital signatures, certificates, Merkle trees, and the envelope and key-management machinery that wires them together. The rest of Book 5 (Sigstore in Chapters 3–4, transparency logs in Chapter 5, in-toto in Chapter 6, TUF in Chapter 7, key management in Chapter 9) assumes you hold precise mental models of these primitives. This chapter builds them. It is deliberately *not* a general cryptography course — there is no elliptic-curve arithmetic, no number theory beyond what changes an engineering decision. It is a rigorous, practical refresher aimed at exactly the operations supply chain security performs: hash an artifact, sign a digest, verify a signature against an identity, prove membership in a log, and manage the keys underneath all of it.

Learning goals — after this chapter you should be able to:

- State the three security properties of a cryptographic hash (preimage, second-preimage, collision resistance), explain which one **SHAttered** broke for SHA-1, and explain why "the hash *is* the name" (content addressing) is the load-bearing integrity pattern of the entire supply chain.
- Explain what a digital signature actually proves — authenticity, integrity, non-repudiation — and, just as importantly, what it does **not** prove: that the signed bytes are *safe* (the SolarWinds lesson).
- Choose correctly among **RSA (PKCS#1 v1.5 vs PSS)**, **ECDSA (P-256)**, and **Ed25519**, and say why Ed25519 is the modern default; distinguish a signature from an **HMAC**.
- Read an **X.509** certificate chain, describe how a verifier validates it, and explain why **revocation** is the hard part — setting up the short-lived-certificate design of Chapters 3–4.

- Explain **Merkle inclusion and consistency proofs** at the level Chapter 5 needs, describe the **DSSE** envelope and its **PAE** encoding, and give an accurate account of **threshold signing** and the **post-quantum** transition (ML-DSA / SLH-DSA, standardized 2024).

Hash functions: turning bytes into names

A cryptographic hash function H maps an input of any length to a fixed-length output — 256 bits for SHA-256, 512 for SHA-512 — such that the output behaves, for all practical purposes, like a random fingerprint of the input. Change one bit of a 4 GB container image and the digest changes completely and unpredictably (the *avalanche* property). This is the workhorse of supply chain integrity, and the reason is that a good hash gives you three distinct security properties, which are worth keeping separate because attacks and use cases target them individually.

- **Preimage resistance.** Given a digest h , it is computationally infeasible to find *any* input m with $H(m) = h$. This is what lets a hash stand in for a secret you never reveal: a password-hash comparison, or a commitment to a value you will disclose later.
- **Second-preimage resistance.** Given a *specific* input m_1 , it is infeasible to find a *different* $m_2 \neq m_1$ with $H(m_2) = H(m_1)$. This is the property that matters when a hash is used as a name for a known artifact: an attacker who wants to swap `nginx:1.27` for a backdoored image must produce a *different* image with the *same* digest.
- **Collision resistance.** It is infeasible to find *any* pair $m_1 \neq m_2$ with $H(m_1) = H(m_2)$. This is strictly stronger than second-preimage resistance, because the attacker gets to choose *both* messages. It matters when the attacker controls the content being hashed — for example, submitting a benign document for signing and later claiming the signature covers a malicious one that hashes identically.

The generic cost of these attacks is set by the output length n . A brute-force preimage or second-preimage search costs about 2^n operations; finding a collision costs only about $2^{(n/2)}$ because of the birthday paradox. So SHA-256 offers ~ 256 -bit preimage resistance but only ~ 128 -bit collision resistance — which is why 256 bits is the modern floor: it puts the *weaker* of the two properties at the 128-bit security level we treat as safe.

Why MD5 and SHA-1 are dead

A hash is "broken" when an attack beats these generic bounds, and the supply chain has watched two workhorses fall.

MD5 (128-bit) lost collision resistance to Wang et al. in 2004; by 2008 researchers used MD5 collisions to forge a rogue CA certificate, and in 2012 the **Flame** espionage malware used an MD5 chosen-prefix collision to forge a Microsoft code-signing certificate and pass itself off as a legitimate Windows Update. That last one is the canonical demonstration that a broken hash *inside a signing scheme* is a signing-scheme break: Flame was validly signed malware whose validity flowed entirely from a collision.

SHA-1 (160-bit) was theoretically wounded for years, then killed in practice. On **February 23, 2017**, researchers at Google and CWI Amsterdam published **SHAttered** — the first *practical* SHA-1 collision: two different PDF files with the same SHA-1 digest, produced with roughly 2^{63} SHA-1 computations (far below the 2^{80} a generic birthday attack would need). In 2020, Leurent and Peyrin extended this to a **chosen-prefix** collision ("SHA-1 is a Shambles"), which is the dangerous kind: the attacker can collide two meaningfully different, attacker-chosen messages, cheaply enough to forge PGP web-of-trust certificates.

This matters directly for **git**, which names every object — blobs, trees, commits — by its SHA-1 hash. A chosen-prefix collision is, in principle, a way to make two different commits share an object ID, which is a foundational integrity failure for a content-addressed store. In practice git mitigated the immediate risk by adopting **SHA-1DC** (hardened SHA-1 with collision *detection*, which rejects inputs bearing the SHATTERED attack's fingerprint) and is migrating toward SHA-256 object IDs — a transition explored in Book 7 (Source and Insider Threats), because it touches every tool that assumes a git hash is 40 hex characters. The lesson for this book: a content-addressed system inherits the collision resistance of its hash, and no more.

The modern hash toolbox

Hash	Output	Structure	Status for supply chain
MD5	128-bit	Merkle-Damgård	Broken (collisions trivial). Never for security.

Hash	Output	Structure	Status for supply chain
SHA-1	160-bit	Merkle-Damgård	Broken (SHattered 2017). Legacy only; git migrating off.
SHA-256	256-bit	Merkle-Damgård	Recommended default. OCI digests, cosign, most tooling.
SHA-512	512-bit	Merkle-Damgård	Fine; faster than SHA-256 on 64-bit CPUs.
SHA-512/256	256-bit	Merkle-Damgård (truncated)	SHA-512 internally, 256-bit output; length-extension safe.
SHA3-256 / SHA3-512	256 / 512-bit	Keccak sponge	Sound, different design; useful for algorithm diversity.
BLAKE3	256-bit (XOF)	Merkle-tree, keyed	Very fast, parallel; used by some newer tooling.

Two design-family notes matter operationally. **SHA-2** (SHA-256/512) uses the Merkle-Damgård construction, which has a **length-extension** weakness: given $H(\text{secret} \parallel m)$ and the length of `secret`, an attacker can compute $H(\text{secret} \parallel m \parallel \text{padding} \parallel m')$ without knowing the secret. This never affects plain artifact hashing or signatures, but it is why you must never build a MAC as $H(\text{secret} \parallel \text{message})$ — use HMAC or SHA-512/256, which truncates the internal state and closes the extension. **SHA-3** is a completely different construction (the Keccak sponge), which is valuable precisely because it shares no structural DNA with SHA-2: if a break is ever found in the Merkle-Damgård family, SHA-3 is unlikely to fall to the same technique. That diversity is the seed of *crypto-agility*, a theme we return to at the end of the chapter.

Content addressing: the hash *is* the name

Here is the pattern that makes hashes the backbone of supply chain security, not merely a tool in it. In a **content-addressed** system, an artifact's identifier *is* its hash. The name is derived from the bytes, so the name is a self-verifying claim about the bytes: anyone who has the

artifact can recompute the hash and confirm they have exactly the object the name refers to — no trusted third party, no lookup, no metadata to forge.

Supply chain infrastructure is content-addressed nearly everywhere:

- **OCI image digests.** `nginx@sha256:a8f2...` names an image by the SHA-256 of its manifest. The manifest in turn lists each layer and config blob *by digest*, so the single top-level digest transitively commits to every byte of the image. Pulling by digest — rather than by the mutable tag `nginx:1.27` — is a security primitive: the registry, a MITM, or a compromised mirror cannot serve you different bytes than the digest names.
- **git commits.** A commit hashes its tree, parent commit(s), author, and message; the tree hashes its entries; each blob hashes its content. A single commit SHA is therefore a Merkle root over the entire repository state at that point (more on Merkle structure below).
- **Lockfile integrity hashes.** `package-lock.json` integrity fields (Subresource Integrity, `sha512-...`), Go's `go.sum`, Cargo's `Cargo.lock` checksums, and Python's `--require-hashes` all pin a dependency's *content* by hash. Note the contrast with a **purl** (`pkg:npm/lodash@4.17.21`), which is a *coordinate* — a mutable name a registry resolves — not a content address; the lockfile is exactly the layer that binds the coordinate to an immutable hash so that "resolve `lodash@4.17.21` " cannot silently return different bytes tomorrow.

The security move in all of these is the same: **pinning by digest**. A coordinate (`:1.27` , `@4.17.21` , `latest`) is a promise someone else can break; a digest is a fact you can verify. Throughout this book, "pin by digest" is the baseline, and signatures and provenance are what you layer *on top* of a digest to answer questions the digest alone cannot — namely *who* built these exact bytes and *whether you should trust them*.

Digital signatures: the core primitive

A hash tells you whether bytes changed. It does not tell you *who* produced them, because anyone can compute a hash. Digital signatures add that missing axis: origin.

Asymmetric cryptography in one page

A signature scheme uses an **asymmetric keypair**: a **private key** (kept secret) and a mathematically related **public key** (published freely). The defining property is that the two keys play different roles that cannot be swapped:

- **Signing** takes the private key and a message and produces a signature.
- **Verification** takes the public key, the message, and the signature and returns valid/invalid.

Only the holder of the private key can produce a signature that verifies under the corresponding public key, and this holds even though the public key is known to everyone. That asymmetry — sign with private, verify with public — is what makes signatures *broadcast authentication*: you can hand your public key to the entire world, and any of them can verify your signatures, but none of them can forge one.

```
flowchart LR
    subgraph Signer["Signer (holds private key)"]
        A["Artifact bytes"] -->|H()| D["Digest (sha256)"]
        D -->|"Sign with PRIVATE key"| S["Signature"]
    end
    A -->|"distributed"| A2["Artifact bytes"]
    S -->|"distributed"| S2["Signature"]
    PUB["Public key\n(published)"] --> V
    subgraph Verifier["Verifier (holds public key)"]
        A2 -->|H()| D2["Digest'"]
        D2 --> V{"Verify(pub, digest', sig)"}
        S2 --> V
        V -->|"valid"| OK["Origin + integrity confirmed"]
        V -->|"invalid"| NO["Reject"]
    end
```

What a signature proves — and what it does not

A valid signature under a public key you trust establishes three things:

- **Integrity.** The bytes were not altered after signing; any change breaks the signature.
- **Authenticity.** The signature was produced by the holder of the corresponding private key.

- **Non-repudiation.** Because only the private-key holder could have produced it, the signer cannot later credibly deny having signed — a property MACs (below) do *not* provide.

Now the single most important sentence in this chapter: **a signature proves origin, not honesty.** It says "the holder of this key vouches for these exact bytes." It says *nothing* about whether those bytes are safe, correct, or benign. A signature over malware is a perfectly valid signature over malware.

This is not a hypothetical. In the **SolarWinds** compromise (2020; dissected in Book 1, Chapter 3), the attackers implanted the SUNBURST backdoor into the Orion build *before* SolarWinds' own signing step. The malicious DLL was then signed with SolarWinds' legitimate code-signing key, as part of the normal, authorized release process. Every downstream customer who verified the signature got a valid result — because the signature was, in the narrow cryptographic sense, entirely valid. The signing key was not stolen; it did not need to be. The attack subverted the *input* to signing, and the signature dutifully certified the origin of code that was already backdoored.

A valid signature proves	A valid signature does not prove
The bytes are unchanged since signing (integrity)	The bytes are free of vulnerabilities
A specific private key produced it (authenticity)	The bytes are non-malicious ("safe")
The signer cannot deny signing (non-repudiation)	The signer <i>should</i> be trusted for this use
<i>If</i> bound to identity via PKI: which identity signed	That the signing process was not subverted upstream
That you got the same bytes the signer signed	That the <i>right</i> thing was signed (see SolarWinds)

The takeaway shapes the entire book. Signing is necessary but not sufficient. It answers "who vouches for these bytes?" — a question worth answering — but it must be composed with *provenance* (what process produced them, Book 4, Chapter 3), *transparency* (is this signature publicly recorded, Chapter 5), and *policy* (do I trust this signer for this action, Chapter 10). A signature is a building block, not a verdict.

Signature algorithms, accurately

You will almost never implement a signature algorithm, but you must choose one and configure it correctly, so precision matters.

RSA. The oldest of the three, based on the difficulty of factoring large integers. RSA signing has *two* padding schemes, and the distinction is not cosmetic:

- **PKCS#1 v1.5** is the older, deterministic padding. It is still ubiquitous (TLS certificates, older tooling) and there is no known practical forgery against RSA *signatures* with correct v1.5 padding — but the scheme lacks a security proof and has a long history of implementation pitfalls (the "Bleichenbacher" family, and signature-forgery bugs when verifiers parse padding sloppily, e.g. BERserk).
- **RSA-PSS** (Probabilistic Signature Scheme) is the modern padding: randomized, with a tight security proof reducing forgery to the RSA problem. New designs should prefer PSS.

RSA's cost is key and signature *size*: 2048-bit RSA gives roughly 112-bit security, and you need **3072-bit** for the 128-bit level, producing 384-byte signatures. RSA verification is fast (small public exponent); signing is comparatively slow.

ECDSA. Elliptic Curve Digital Signature Algorithm, based on the elliptic-curve discrete-log problem. **P-256** (a.k.a. `secp256r1` / `prime256v1`) gives 128-bit security with 32-byte keys and 64-byte signatures — an order of magnitude smaller than equivalent RSA. It is fast and widely supported in hardware and standards. Its one sharp edge: ECDSA requires a fresh, uniformly random secret **nonce** `k` for every signature, and *any* reuse or bias in `k` leaks the private key. This is not theoretical — it is how the **PlayStation 3** code-signing key was recovered in 2010 (Sony reused a constant `k`), and it recurs whenever an implementation's RNG is weak. We return to this under "correct usage."

EdDSA / Ed25519. The modern default. Ed25519 is EdDSA instantiated over the twisted Edwards curve `edwards25519` (RFC 8032), and it was engineered specifically to remove the footguns that sink ECDSA deployments:

- **Deterministic.** The per-signature nonce is derived by hashing the private key together with the message, so there is no RNG to reuse or bias. The PS3-class failure is impossible by construction.

- **Fast.** Signing and verification are very fast, and batch verification is supported.
- **Misuse-resistant.** No parameter choices to get wrong, no point-validation subtleties left to the caller, 32-byte keys and 64-byte signatures.
- **128-bit security** at those small sizes.

For new signing systems, Ed25519 is the right default, and much of the modern supply chain tooling reflects that. (Sigstore's Fulcio issues ECDSA-P-256 leaf certificates for ecosystem and hardware-support reasons, but cosign, TUF, SSH, and many others use or prefer Ed25519 for raw-key signing.)

Algorithm	Security level	Key / sig size	Nonce risk	Recommendation
RSA-2048 PKCS#1 v1.5	~112-bit	256 / 256 B	none (det.)	Legacy compat only
RSA-3072 PSS	128-bit	384 / 384 B	none (det.)	OK if RSA required
ECDSA P-256	128-bit	32 / 64 B	high (needs good RNG)	Fine with vetted lib
ECDSA P-384	192-bit	48 / 96 B	high	When 192-bit needed
Ed25519	128-bit	32 / 64 B	none (deterministic)	Default choice

Sign the hash, not the artifact

A subtlety that trips people up: signature algorithms do not sign the artifact. They sign its **digest**. A 4 GB image is hashed to 32 bytes, and the signature operation runs on those 32 bytes. This is why "sign" and "hash" are inseparable in practice, and it has two consequences. First, the collision resistance of the hash is part of the signature's security: if you can find two artifacts with the same digest, a signature over one is a valid signature over the other — which is *exactly* how the Flame malware forged its Microsoft certificate. Signing a digest is only as strong as the hash under it, which is why signing schemes deprecate SHA-1 in lockstep

with everything else. Second, it means signing is cheap and streaming-friendly: you hash the artifact once (linear, no secret needed) and perform one small asymmetric operation on the result.

MACs versus signatures

A **MAC** (Message Authentication Code) — in practice **HMAC** (RFC 2104), built from a hash like `HMAC-SHA256` — also authenticates a message and detects tampering, and it is faster and smaller than any signature. But it is **symmetric**: the *same* secret key both produces and verifies the tag. Everyone who can verify can also forge. This has one decisive consequence for supply chain use: an HMAC provides **no non-repudiation**. If Alice and Bob share an HMAC key and Bob receives a validly-tagged message, Bob cannot prove to a *third party* that Alice sent it — Bob could have produced the tag himself. A signature, being asymmetric, gives exactly the property HMAC cannot: a verifiable link to a *single* party who alone could have produced it.

The rule of thumb: use an **HMAC** when the two parties are the same trust domain and you just need fast integrity/authenticity on a channel (webhook signatures like GitHub's `X-Hub-Signature-256`, session cookies, API request signing, deriving sub-keys). Use a **digital signature** whenever a third party must independently verify origin, or whenever the signer must be held to their signature — which is *every* artifact-signing scenario in this book. Signing an artifact with an HMAC would be meaningless: the whole point is that the world, not just the holder of a shared secret, can verify it.

PKI and certificates: binding keys to identities

Signatures have a bootstrapping problem. Verification tells you "the holder of *this public key* signed these bytes." But how do you know *whose* key it is? An attacker can generate a keypair, sign malware with it, and hand you the public key claiming it belongs to "nginx maintainers." The signature will verify perfectly against the key they gave you. The math is sound; the *identity binding* is missing. This is the **trust problem**, and Public Key Infrastructure (PKI) is one of the two answers to it (the other, keyless signing bound to workload identity, is Chapters 3–4).

Certificates and the X.509 chain

A **certificate** is a signed statement that binds a **public key** to an **identity**. In the web PKI it is an **X.509 v3** certificate, whose essential fields are the subject (the identity — a domain, an email, a service), the subject's public key, a validity window (`notBefore` / `notAfter`), the issuer, and a set of extensions (Subject Alternative Names, Key Usage, Extended Key Usage, Basic Constraints marking whether the cert may itself act as a CA). Crucially, the certificate is **signed by a Certificate Authority (CA)** — an issuer whose own key you already trust. The certificate is thus a signature that says: *"I, this CA, attest that this public key belongs to this identity, until this date."*

Trust is not flat; it is a **chain**. You do not trust every CA directly. You trust a small set of **root** CAs — self-signed certificates baked into a **trust store** (your OS/browser root store, a container base image's `ca-certificates`, a private corporate root). Roots delegate to **intermediate** CAs, which issue the **leaf** (end-entity) certificates that actually bind service identities. Each certificate is signed by the one above it, forming a chain from leaf to root.

flowchart TD

```
ROOT["Root CA\n(self-signed)\nin trust store"] -->|signs|  
INT["Intermediate CA\nBasicConstraints CA:TRUE"]  
INT -->|signs| LEAF["Leaf certificate\nSubject:  
build.example.com\nEKU: codeSigning"]  
LEAF --> KEY["Public key+\n identity binding"]  
TS["Trust store\n(roofs the verifier trusts)"] -.anchors.-> ROOT  
style ROOT fill:#1f6feb,color:#fff  
style TS fill:#2ea043,color:#fff
```

Chain validation, which every TLS client and signature verifier performs, walks from the leaf up to a trusted root and checks, at each link: (1) the signature on this certificate verifies under the issuer's public key; (2) the current time is within the certificate's validity window; (3) the issuer is actually permitted to issue — `BasicConstraints` marks it a CA, `pathLenConstraint` is not exceeded, and Key/Extended-Key-Usage and name constraints permit this issuance; (4) the certificate has not been **revoked**; and (5) the top of the chain is a root in the trust store. If any check fails, the whole chain is untrusted. The identity you may then rely on is the leaf's subject/SAN — *and only because* an unbroken chain of signatures connects it to a root you decided, out of band, to trust.

Revocation is the hard part

Certificates carry an expiry, but expiry is coarse. What happens when a private key is *stolen* before its certificate expires, or a certificate was *misissued*? You need **revocation** — a way to say "stop trusting this certificate now." PKI offers two mechanisms, and both are operationally painful:

- **CRL** (Certificate Revocation List): the CA publishes a signed list of revoked serial numbers. Lists grow large, are fetched infrequently, and a verifier with a stale CRL happily accepts a revoked cert.
- **OCSP** (Online Certificate Status Protocol): the verifier asks the CA's responder, in real time, "is serial X still good?" This leaks the verifier's browsing/verification activity to the CA (a privacy problem), adds a network round-trip on the hot path, and — the fatal flaw — most clients **soft-fail**: if the OCSP responder is unreachable, they proceed *as if the cert were valid*, because hard-failing would make every CA outage a global outage. An attacker who can block the OCSP request therefore neutralizes revocation entirely. **OCSP stapling** (the server presents a recent signed status itself) mitigates the privacy and latency issues but not the soft-fail one, and the industry has been actively *deprecating* OCSP.

The deep lesson — the one that motivates half of Chapters 3-5 — is that **revocation of long-lived credentials does not work well at scale**. Every revocation mechanism is either stale, privacy-leaking, or fail-open. The modern answer is to sidestep revocation by making certificates **short-lived**: if a certificate lives for 90 days (Let's Encrypt) or **~10 minutes** (Sigstore's Fulcio, Chapter 4), the revocation window is small enough that expiry *is* the revocation mechanism. A stolen key that expires in ten minutes is a much smaller problem than one that expires in two years and whose revocation the ecosystem may never notice. Short-lived certificates fit ephemeral CI perfectly (Book 4, Chapter 8): the build runs for minutes, gets a certificate for minutes, and the credential is worthless the moment the job ends.

The CA trust model's weaknesses

The web PKI has a structural fragility: **any** trusted CA can issue a certificate for **any** identity. Your browser trusts scores of root CAs; a compromise or misbehavior at *any one* of them produces a certificate that

validates cleanly for `*.yourcompany.com`. This is not hypothetical: in 2011 the Dutch CA **DigiNotar** was breached and issued fraudulent certificates for Google and other domains, which were used to intercept traffic — the incident ended with DigiNotar removed from trust stores and bankrupt. The same year, **Comodo** suffered fraudulent issuance. Misissuance and CA compromise are the web PKI's persistent failure mode.

The response is **Certificate Transparency** (CT, RFC 6962): CAs must log every certificate they issue to public, append-only **Merkle-tree logs**, so that a domain owner can *detect* a certificate issued for their domain that they did not request. CT does not prevent misissuance; it makes misissuance **discoverable**. That same architecture — append-only transparency over a signing authority — is exactly what Sigstore's **Rekor** applies to software signatures (Chapter 5). To understand either, you need Merkle trees, which we turn to now.

Key management: protecting the crown jewels

Everything above assumes the private key is secret. That assumption is the hardest thing to uphold in the whole system, and it is where real signing programs live or die. Chapter 9 develops key management fully; here is the foundation.

The asymmetry that makes signatures powerful also makes them fragile: a stolen signing key lets the attacker **sign as you**, indefinitely, with output indistinguishable from your legitimate signatures. There is no algorithm that can tell a signature made with your stolen key from one you made yourself — that is the entire point of the key. A signing key is therefore a **crown-jewels** asset (the framing of Book 4, Chapter 6): its compromise does not merely expose data, it lets the attacker *manufacture trust* under your name until you detect it and revoke — and we just saw how poorly revocation scales.

The discipline that follows:

- **Generation.** Keys must be generated with a strong CSPRNG, ideally *inside* the hardware that will hold them, so the private key never exists in exportable form.
- **Storage.** The gold standard is that **the private key never leaves hardware**. A **HSM** (Hardware Security Module) or a cloud **KMS** (AWS KMS, GCP Cloud KMS, Azure Key Vault) holds the key in a tamper-resistant boundary and exposes only a *sign* operation: you

send a digest, it returns a signature, the key material never crosses the boundary. Even a full compromise of the application host does not exfiltrate the key — though note it *can* let the attacker request signatures while access lasts, which is why access to the sign operation is itself a guarded, audited privilege.

- **Rotation.** Keys should be rotated on a schedule so that the blast radius of an undetected compromise is bounded in time, and so the operational muscle for rotation exists *before* an incident forces an emergency one.
- **Revocation.** When compromise is suspected, you must be able to revoke — and, as above, revocation of long-lived keys is unreliable, which is the whole motivation for the next move.

This is the operational pain that **keyless signing** (Chapters 3–4) is designed to eliminate. The insight: if the durable, high-value signing key is the problem, get rid of it. Fulcio issues a *short-lived* certificate bound to a *workload identity* (an OIDC token from your CI provider) and the private key exists only for the ~10 minutes of a single build, then is discarded. There is no long-lived signing key to steal, rotate, or revoke — the identity provider and the transparency log carry the trust instead. Understanding *why* that is attractive requires having felt the weight of key management, which at fleet scale — thousands of services each signing on every deploy — is considerable.

Advanced primitives the domain depends on

Merkle trees, inclusion proofs, and consistency proofs

A **Merkle tree** is a binary tree of hashes that lets a single small root value commit to an arbitrarily large set of items, while allowing anyone to prove membership of one item cheaply. Each **leaf** is the hash of a data item; each **internal node** is the hash of the concatenation of its two children; the **root** hash therefore depends on every leaf. Change any leaf and the root changes. (In RFC 6962's construction, leaves are hashed with a `0x00` prefix and internal nodes with a `0x01` prefix — *domain separation* that prevents an attacker from passing off an internal node as a leaf, a second-preimage defense worth knowing exists.)


```

flowchart TD
    ROOT["Root = H(H12 || H34)"]
    ROOT --> H12["H12 = H(H1 || H2)"]
    ROOT --> H34["H34 = H(H3 || H4) ← proof"]
    H12 --> L1["H1 = H(d1)"]
    H12 --> L2["H2 = H(d2) ← proof"]
    H34 --> L3["H3 = H(d3) TARGET"]
    H34 --> L4["H4 = H(d4)"]
    style L3 fill:#2ea043,color:#fff
    style L2 fill:#1f6feb,color:#fff
    style H34 fill:#1f6feb,color:#fff

```

An **inclusion proof** (audit proof) demonstrates that a specific item is a leaf under a known root, *without* revealing the other items, using only the sibling hashes along the path from the leaf to the root — about $\log_2(n)$ hashes for n leaves. In the diagram, to prove `d3` is in the tree given the trusted root, the log supplies just two hashes: `H4` (its sibling) and `H12` (its uncle). The verifier computes `H3 = H(d3)`, then `H34 = H(H3 || H4)`, then `H(H12 || H34)`, and checks the result equals the trusted root. Two hashes prove membership in a tree of four; for a log of a *billion* entries, only ~ 30 hashes are needed. This logarithmic cost is precisely what makes transparency logs practical: a verifier can confirm "my signature is recorded in Rekor" by fetching ~ 30 hashes, not the whole log.

A **consistency proof** demonstrates that a *later* version of the log is a superset of an *earlier* one — that entries were only **appended**, never modified or deleted. Given the root at size m and the root at size $n > m$, the log provides another $\sim \log(n)$ hashes that let a verifier confirm the size- n tree contains the size- m tree unchanged as a prefix. This is what makes a log **append-only** in a way clients can *verify* rather than merely trust: even the log operator cannot rewrite history without producing an inconsistency that monitors will catch. Merkle inclusion + consistency proofs are the mechanical heart of both **Certificate Transparency** and **Rekor**, and Chapter 5 is essentially a deep dive on this diagram. They also underlie git (a commit is a Merkle root) and every content-addressed store.

DSSE and PAE: signing arbitrary payloads unambiguously

Once you leave the world of "sign a raw blob" and start signing structured statements — "this artifact has this SBOM," "this build produced this provenance" — you need an envelope that carries the payload, says *what kind* of payload it is, and holds the signature(s). That

envelope is **DSSE**, the **Dead Simple Signing Envelope**, and it is the format in-toto attestations (Chapter 6) and SLSA provenance (Book 4, Chapter 3) travel in.

A DSSE envelope is a small JSON object with three parts:

- `payload` — the actual statement, base64-encoded, opaque to DSSE.
- `payloadType` — a URI naming the payload's type/format (e.g. `application/vnd.in-toto+json`). This tells the verifier how to interpret the bytes.
- `signatures` — a list, each with a `sig` value and an optional `keyid`. A list because a payload may carry multiple signatures (multi-sig, below).

```
flowchart TD
    subgraph ENV["DSSE Envelope (JSON)"]
        P["payload (base64 statement)"]
        T["payloadType (e.g. application/vnd.in-toto+json)"]
        SIGS["signatures: [ { sig, keyid }, ... ]"]
    end
    P --> PAE
    T --> PAE
    PAE["PAE(payloadType, payload)\n= 'DSSEv1' SP len(type) SP type SP len(payload) SP payload"]
    PAE --> SIG["Sign(PAE)"]
    SIG --> SIGS
```

The subtle and important part is **PAE — Pre-Authentication Encoding**. The signature is *not* computed over the raw payload bytes. It is computed over:

```
PAE(payloadType, payload) = "DSSEv1" SP LEN(payloadType) SP
payloadType SP LEN(payload) SP payload
```

where `SP` is an ASCII space and `LEN(x)` is the byte length of `x` in ASCII decimal. Why bother? Because signing raw payload bytes invites **ambiguity attacks**. If the signature covered only the payload, an attacker might re-present the *same signed bytes* under a *different payloadType*, tricking a verifier into parsing an in-toto statement as, say, a different format whose semantics flip the meaning — the same signature "means" two things. PAE closes this by **binding the type into the signed data** and length-prefixing every field, so there is exactly one unambiguous byte string the signature can correspond to; you cannot

slide the boundary between fields or reinterpret the type without changing what was signed. It also sidesteps JSON-canonicalization headaches: DSSE signs the exact payload bytes (via PAE), not a re-serialized version, so two verifiers can never disagree about which bytes the signature covers. "Dead simple" is the point — the format is small enough to implement correctly, which is itself a security property.

Threshold and multi-signature schemes

Sometimes one key — one person, one machine — should not be sufficient to make a trusted statement. **m-of-n** schemes require some threshold `m` of `n` authorized keys to agree.

There are two flavors, and they are often conflated:

- **Multi-signature** collects `m` *separate, independently-verifiable* signatures over the same payload. A verifier checks that at least `m` of the `n` known keys signed. This is what **TUF** (The Update Framework, Chapter 7) uses: each metadata role (root, targets, snapshot, timestamp) specifies a set of authorized keys and a signature **threshold**, so that compromising a single key does not let an attacker forge that role's metadata. Sigstore's own root of trust is governed this way — a quorum of geographically distributed keyholders must sign to change it. The DSSE `signatures` list is the on-the-wire form of this: multiple `sig` entries over one payload.
- **Threshold signatures** (true cryptographic threshold, e.g. **FROST** for Schnorr) split a *single* private key across `n` parties such that any `m` can jointly produce **one** ordinary signature, and no coalition smaller than `m` learns the key. The verifier sees a single normal signature and cannot even tell it was produced collaboratively.

For supply chain purposes, the relevant idea is the same and TUF is the canonical user: **resilience through distribution of trust**. No single stolen key is catastrophic, because no single key is sufficient. This is the counterweight to the crown-jewels problem — you cannot protect any one key perfectly, so you arrange for no one key to be enough.

A calibrated word on post-quantum

A sufficiently large, fault-tolerant quantum computer running **Shor's algorithm** would break the hard problems underneath RSA (factoring) and ECDSA/Ed25519 (elliptic-curve discrete log) — not weaken them,

break them, making private keys recoverable from public keys. Such a machine does not exist today and its arrival date is genuinely uncertain, so calibrate accordingly.

Two points keep the risk in proportion for *signing*:

- The urgent quantum worry — **"harvest now, decrypt later,"** where an adversary records encrypted traffic today to decrypt once quantum arrives — is a **confidentiality** problem. It does **not** apply to signatures: you cannot retroactively forge a 2026 signature with a 2035 quantum computer, because the signature was already verified at the time it mattered. The real signing concerns are narrower: (a) **long-lived roots of trust and long-lived artifacts** whose signatures must still be *trustworthy* years from now, and (b) the lead time to migrate a whole ecosystem's tooling.
- **Hashes are largely fine.** Grover's algorithm gives only a quadratic speedup against hashes and symmetric ciphers, so SHA-256 retains ~128-bit preimage resistance against a quantum attacker — no migration panic there.

NIST finalized the first post-quantum standards in **August 2024**: **FIPS 203 ML-KEM** (Kyber, key encapsulation), **FIPS 204 ML-DSA** (Dilithium, lattice-based signatures), and **FIPS 205 SLH-DSA** (SPHINCS+, a **stateless hash-based** signature). SLH-DSA is especially interesting for supply chain because its security rests *only* on hash functions — the most conservative assumption available — at the cost of large signatures (kilobytes). Stateful hash-based schemes (LMS/XMSS, RFC 8554 / 8391) are also standardized and suit firmware signing where the number of signatures is bounded and controllable.

The practical mandate is not "migrate now" but **crypto-agility**: build systems that can swap signature algorithms without re-architecting, so that when migration is warranted it is a configuration and rotation exercise, not a rewrite. Knowing *which* algorithms are deployed across a fleet is a prerequisite for agility, which is why a **CBOM** (Cryptography Bill of Materials, expressible in CycloneDX and covered in Book 3, Chapter 3) is the inventory that makes a coordinated migration feasible at all.

Crypto-agility and correct usage

Two operational disciplines separate systems that use cryptography from systems that use it *safely*.

Do not roll your own. The failure modes above — nonce reuse, padding-oracle parsing, length-extension MACs, canonicalization ambiguity — are exactly the mistakes that vetted, peer-reviewed libraries have already eliminated and that hand-rolled schemes reliably reintroduce. Use a maintained library (libsodium/NaCl, Go's `crypto/ed25519`, `ring`, BoringSSL, Tink) and a maintained *format* (DSSE, not a bespoke JSON-signing convention). "Don't design your own signing scheme" is not conservatism; it is an acknowledgment that the design space is littered with subtle, catastrophic mistakes that took the field decades to catalog. The place to be creative in supply chain security is *policy and architecture*, never the primitive.

Design for agility. Algorithms weaken on human timescales — MD5, SHA-1, RSA-1024 all went from "fine" to "forbidden" within the working life of systems still running. A signing system should therefore (1) record *which* algorithm and key produced each signature (as DSSE and X.509 do), so verifiers can reason about it and migrations can be staged; (2) support verifying multiple algorithms during a transition window; and (3) treat algorithm choice as configuration, not a hardcoded assumption. The alternative — an algorithm baked so deeply that changing it means re-signing the world under emergency conditions — is precisely the position git found itself in with SHA-1.

Prefer misuse-resistant primitives. This is why Ed25519's determinism is more than a nicety: it removes the single most common way ECDSA deployments leak their private keys. When a primitive makes a class of catastrophic bug *impossible by construction* rather than *avoidable with care*, choose it. Care does not scale across thousands of signing operations; construction does.

Distributed-systems lens

Pull these threads together at fleet scale — many services, many teams, many builds per day — and the primitives stop being isolated tools and become the *coordination substrate* for trust across a system where no component trusts every other.

Content addressing is how immutable references work without a trusted intermediary. In a distributed build-and-deploy system, artifacts move through registries, mirrors, caches, and CDNs you do not all control. A digest (`sha256:...`) is a name that any node can independently verify against the bytes it received, so a compromised mirror cannot substitute different content — the name would no longer match. Pinning by digest turns "trust every hop" into "trust the math," which is the only thing that scales when the number of hops is large and their operators are heterogeneous. OCI digests, git SHAs, and lockfile hashes are the same primitive applied at three layers of the stack.

Signatures plus transparency logs enable trust without trusting every intermediary. A signature lets a deploy-time admission controller (Chapter 10) verify origin without contacting the builder; a transparency log lets it confirm the signature is *publicly recorded* without the builder and verifier ever having communicated directly. Trust becomes a checkable property of the artifact and its attestations, decoupled in time and space from their production — which is exactly the property a distributed system needs, since the signer and verifier are usually different teams, different clusters, and different points in time.

Key management is the operational hard part, and it is where scale bites. One signing key is manageable. Thousands of services, each signing on every deploy, each needing a key that must be generated, stored in an HSM/KMS, rotated, and revocable, is an operational burden that dominates the design. This is the pressure that pushes the whole field toward **keyless signing and KMS-backed identity** (Chapters 3–4 and 9): replace a durable secret you must protect forever with a **short-lived credential** minted per-build from a **workload identity**. Ephemeral CI — runners that exist for minutes (Book 4, Chapter 8) — is a natural fit for credentials that also live for minutes; a ten-minute certificate matches a ten-minute build far better than a two-year key matches a fleet of ephemeral runners. The arc of the rest of this book is, in large part, the story of trading long-lived keys for short-lived, verifiable, logged identity — and every primitive in this chapter is a piece of how that trade is made sound.

Key takeaways

- **Hashes give you three separable properties** — preimage, second-preimage, and collision resistance — and the weakest (collision, at

$\sim 2^{(n/2)}$ sets the floor. SHA-256 is the default; MD5 and SHA-1 are broken (SHAttered, 2017), which is a *signing*-scheme problem wherever those hashes are signed.

- **The hash is the name.** Content addressing (OCI digests, git SHAs, lockfile hashes) makes identifiers self-verifying, and pinning by digest is a security primitive that removes trust from every intermediary.
- **A signature proves origin, integrity, and non-repudiation — not safety.** SolarWinds was validly signed malware. Signing is necessary but must be composed with provenance, transparency, and policy.
- **Ed25519 is the modern default** — deterministic (no nonce-reuse footgun), fast, small. ECDSA is fine with a vetted library; RSA needs PSS and 3072-bit keys. Use HMAC only when a shared secret suffices and non-repudiation is not required.
- **PKI binds keys to identities via CA-signed certificate chains**, but **revocation does not scale** (stale CRLs, soft-fail OCSP), and any trusted CA can misissue (DigiNotar). This drives short-lived certificates and Certificate Transparency.
- **Protecting the private key is the crown-jewels problem.** HSM/KMS keep keys in hardware; rotation and revocation bound the damage — and the difficulty of doing this at fleet scale motivates keyless signing.
- **Merkle inclusion proofs (log n hashes) and consistency proofs** are the mechanical basis of transparency logs; **DSSE + PAE** sign typed payloads unambiguously; **m-of-n thresholds** (TUF) distribute trust so no single key is sufficient.
- **Post-quantum is real but calibrated:** Shor's breaks RSA/ECC eventually; NIST standardized ML-DSA and SLH-DSA in 2024; the mandate for signing is **crypto-agility**, not panic. Never roll your own; prefer misuse-resistant primitives and vetted libraries.

Further reading

- **NIST FIPS 180-4** (Secure Hash Standard, SHA-2) and **FIPS 202** (SHA-3 / Keccak) — the authoritative hash specifications.
- **SHAttered** — Stevens, Bursztein, et al., "The first collision for full SHA-1" (2017), and Leurent & Peyrin, "SHA-1 is a Shambles" (2020), for the chosen-prefix extension.

- **RFC 8032** — Edwards-Curve Digital Signature Algorithm (EdDSA / Ed25519).
- **RFC 8017** — PKCS #1 v2.2 (RSA, including PKCS#1 v1.5 and PSS).
RFC 2104 — HMAC.
- **RFC 5280** — X.509 PKI certificate and CRL profile. **RFC 6960** — OCSP.
- **RFC 6962** — Certificate Transparency, including the Merkle tree, inclusion, and consistency proof definitions.
- **DSSE specification** — the `secure-systems-lab/dsse` repository, including the PAE definition.
- **The Update Framework (TUF) specification** — role-based threshold signing.
- **NIST FIPS 203 / 204 / 205** (ML-KEM, ML-DSA, SLH-DSA, 2024) and **NIST SP 1800-38** (Migration to Post-Quantum Cryptography).
- **Bernstein & Lange**, "SafeCurves," and the libsodium/NaCl documentation — for the "don't roll your own, prefer misuse-resistant primitives" discipline in practice.
- Book 5, Chapter 5 — Transparency Logs (Merkle trees applied to Rekor and CT); Chapters 3-4 — Sigstore and keyless signing (short-lived certs, Fulcio); Book 4, Chapter 3 — SLSA provenance (DSSE in production).

Chapter 2 — Classic Code Signing and Its Failure Modes

What this chapter covers. Before Sigstore, before transparency logs, before anyone said the words "keyless signing," the industry spent roughly three decades building code-signing systems on the same template: a publisher holds a private key and a certificate from a certificate authority, signs its artifacts, and consumers verify those signatures against a set of trusted roots. Chapter 1 gave you the cryptographic primitives — signatures, certificates, chains of trust. This chapter is about what the industry *built* with them, and, more importantly, about why those systems failed in the field with grim regularity.

The failures are the point. Every design choice in the rest of this book — Fulcio's short-lived certificates (Chapter 3), Rekor's append-only transparency log (Chapter 5), in-toto's attestations (Chapter 6), TUF's key hierarchy (Chapter 7) — is a direct, traceable response to a specific way classic signing broke. If you understand the failure modes concretely, the modern machinery stops looking like ceremony and starts looking like scar tissue. So we will be specific: real incidents, real mechanisms, real root causes.

Learning goals — after this chapter you should be able to:

- Describe the **classic code-signing trust model** precisely — publisher key plus CA certificate, sign, verify against trust anchors — and state exactly what property it does and does not provide.
- Explain the mechanics of the **major ecosystems**: Windows Authenticode, Apple code signing and notarization, Android APK Signature Schemes, Linux distro repository signing (apt and RPM), and the classic OSS **GPG detached-signature** pattern.
- Enumerate the **failure modes** — key theft, signed-but-malicious, CA/issuance failure, broken revocation, operational leakage, no transparency — and attach each to a real incident and a root cause.
- Articulate the **central lesson**: classic signing authenticates the *signer*, not the software's integrity relative to its source — which is why it did nothing to stop SolarWinds, CCleaner, or 3CX.
- Be **fair** about what classic signing gets right (distro repo signing is a genuine success) and where it is merely insufficient rather than worthless.
- Map each failure mode to the **modern mechanism** that addresses it, so the rest of Book 5 reads as a set of answers rather than a catalogue of tools.

A note on framing. This is not a chapter about cryptography being broken. RSA and ECDSA signatures did their job in essentially all of these incidents — the math held. The systems failed at the *edges* of the cryptography: key custody, issuance, revocation, and the gap between "who signed this" and "is this software what it should be." That distinction is the spine of the whole book.

The classic trust model

Strip away the ecosystem-specific details and every classic code-signing system is the same three-party arrangement, inherited more or less directly from the X.509 web PKI (Chapter 1).

- A **certificate authority (CA)** vouches for identity. It performs some vetting of the publisher, then issues a **code-signing certificate**: an X.509 cert binding the publisher's public key to a name ("Contoso Ltd", "NVIDIA Corporation"), marked with an extended key usage of `codeSigning` (OID `1.3.6.1.5.5.7.3.3`).
- A **publisher** holds the corresponding private key. It hashes an artifact, signs the hash, and attaches the signature plus its certificate chain to (or alongside) the artifact.
- A **consumer** — an OS loader, a package manager, a browser download handler — verifies the signature and walks the certificate chain up to a **trust anchor** (a root certificate) it already trusts, typically shipped in the OS or a program-managed root store.

```
flowchart LR
    CA["Certificate Authority  
vets identity, issues cert"]
    Pub["Publisher  
holds private key"]
    Art["Signed artifact  
hash + signature + cert chain"]
    Con["Consumer  
OS loader / package mgr"]
    Root["Trust anchor  
root cert in OS store"]

    CA -->|"issues code-signing cert"| Pub
    Pub -->|"sign(hash, privkey)"| Art
    Art -->|"delivered"| Con
    Con -->|"verify signature"| Art
    Con -->|"chain-build to root"| Root
    Root -->|"anchors trust"| CA
```

The system delivers exactly two properties, and it is worth stating them with pedantic precision because almost every failure in this chapter comes from someone assuming a third:

1. **Integrity** — the artifact has not been modified since it was signed. Flip one bit and the hash changes and the signature no longer verifies.
2. **Authenticity of the signer** — the artifact was signed by the holder of a private key whose certificate a trusted CA issued to a named entity.

That is *all*. In particular, the model says **nothing** about:

- Whether the software is *benign*. A signature is not a virus scan.
- Whether the signed bits correspond to any particular *source code* or build. The signer can sign anything it can hash.
- Whether the key holder is *still* the legitimate publisher, or a thief, or a compromised build server acting with the publisher's authority.

Hold onto that gap between "authentic signer" and "software you should run." Everything downstream lives in it.

The major ecosystems

The template above got instantiated a dozen times, each with local variations that matter in practice. We cover the ones a backend engineer actually meets.

Windows Authenticode

Authenticode is Microsoft's PE (Portable Executable) signing scheme, dating to the mid-1990s and still load-bearing today. It signs `.exe`, `.dll`, `.sys`, MSI installers, and more. Mechanically, the signer computes a hash over the PE file — deliberately *excluding* the checksum field, the certificate table pointer, and the attribute certificate table itself, so the signature can be embedded back into the very file it signs — wraps that hash in a `SignedData` PKCS#7 / CMS structure along with the signer's certificate chain, and stuffs it into the PE's attribute certificate table. `signtool sign` is the canonical tool. Verification (`signtool verify`, or the loader via `WinVerifyTrust`) recomputes the hash over the same excluded ranges and checks the chain to a root in the Microsoft-managed trusted root program.

Two Authenticode-specific mechanisms recur later:

- **Catalog files** (`.cat`). Rather than embedding a signature in every file, Windows lets you sign a *catalog* — a list of file hashes — and the OS treats any file whose hash appears in a trusted catalog as signed. This is how most OS components and many drivers are signed. It decouples the signature from the file, which is convenient and, as we will see with driver signing, occasionally dangerous.
- **Timestamping**. Because certificates expire, Authenticode signatures can carry an RFC 3161 countersignature from a trusted timestamp authority attesting *when* the signing happened. If a signature was timestamped while the cert was valid, it remains valid after the cert expires. This is a usability win and, as the NVIDIA case shows, a security liability.

On top of the raw signature Microsoft layered **SmartScreen reputation** and **EV code-signing certificates**. An EV (Extended Validation) code-signing cert requires stricter organizational vetting and mandates that the private key live on hardware (a smart card or HSM). In exchange, EV-signed binaries accrue SmartScreen reputation faster — a fresh standard-signed installer may still trip the "Windows protected your PC" warning, while an EV-signed one from an established publisher sails through. Note what this does: it makes the *reputation of the signer* a security signal layered on top of the signature, precisely because the signature alone says nothing about benignity.

Finally, **kernel-mode driver signing**. Since 64-bit Windows Vista, kernel drivers must be signed to load, and modern Windows (10 1607+) requires production drivers to be signed through Microsoft's attestation/portal process, chaining to a Microsoft-controlled CA. The kernel is the highest-value place to run code on a Windows box, which is exactly why stolen driver-signing capability shows up repeatedly in the incident record below.

Apple: Developer ID, Gatekeeper, notarization

Apple's model is stricter and more centralized than Windows'. Every macOS binary carries a **code signature** structured around a *code directory* — a hash of hashes, one per page of the executable, so the kernel can verify pages lazily as they are faulted in, plus hashes of the bundle's resources. This is stronger than a single whole-file hash: it

provides *continuous* integrity enforcement at runtime, not just a one-time check at launch.

For software distributed outside the App Store, Apple issues **Developer ID** certificates from Apple's own CA (there is no third-party CA market here — Apple is the only issuer). **Gatekeeper** is the enforcement point: when you download and first launch an app, macOS checks that it is signed by a valid Developer ID and — since macOS 10.15 Catalina — that it has been **notarized**.

Notarization is the interesting part and a genuine partial answer to the "signature $\neq$ benign" problem. The developer uploads the signed app to Apple's **notary service**, which runs automated malware and policy checks, and if the app passes, Apple issues a **notarization ticket** — effectively Apple's own signature attesting "we scanned this build and found nothing known-bad." The developer **staples** the ticket to the app (or Gatekeeper fetches it online). So Gatekeeper checks *two* things: the developer's signature (authenticity) and Apple's ticket (a scan verdict). The **hardened runtime** complements this by opting the app into stricter runtime protections — no unsigned executable memory, library validation, restricted entitlements — reducing the blast radius if the app is later subverted.

Apple's stack is worth studying because it independently reinvented two ideas this book returns to: a *centralized attestation of build safety* (notarization, a cousin of the attestations in Chapter 6) and *revocation with teeth* (Apple can pull a notarization ticket or revoke a Developer ID cert, and because Gatekeeper phones home this actually propagates — contrast the revocation section below).

Android: APK Signature Schemes v1-v4

Android's signing history is a compressed lesson in why signing *scope* matters.

- **v1 (JAR signing)** — the original scheme, inherited from Java's signed JAR format. Each file in the APK zip has a hash listed in `META-INF/MANIFEST.MF`, and a signature covers that manifest. The fatal flaw: the ZIP structure itself — file ordering, the central directory, unsigned metadata, and any files *not* listed in the manifest — is not protected. This enabled the **Master Key** class of bugs (2013) where an attacker could add or duplicate zip entries that verification and

installation disagreed about, so a signed APK could be modified without breaking v1 verification.

- **v2 (APK Signature Scheme v2, Android 7.0)** — signs the *entire APK file* as a blob. The signature block sits in a dedicated region just before the ZIP central directory, and the scheme hashes everything else in the file. Whole-file integrity: no more entry-substitution games.
- **v3 (Android 9)** — adds **key rotation**. A v3 signing block can contain a *proof of rotation*: a chain where the old key signs the new key, so an app can migrate to a fresh signing key while proving continuity of identity. This directly addresses a real operational pain — before v3, an Android app's signing key was effectively immortal, because the platform identity of an app is its signing key and there was no way to change it without shipping a "new app."
- **v4 (Android 11)** — produces a small separate signature file designed around a Merkle hash tree over the APK, enabling incremental/streaming installation (`fs-verity` style) where the app can start running before every byte is verified.

Layered on top is **Play App Signing**, where Google — not the developer — holds the final *app signing key* in Google's infrastructure, and the developer holds only an *upload key*. The developer signs uploads with the upload key; Google re-signs with the app signing key before distribution. This is a pragmatic response to the fact that developers kept losing their signing keys (an unrecoverable catastrophe under the "key is the app's identity" model) — so custody moved to a party better at holding keys. Keep this move in mind: it is the same instinct that produces enterprise KMS (Chapter 9) and, taken to its logical end, keyless signing (Chapter 4).

Linux distributions: repository signing (apt and RPM)

The Linux distro model is different in a way that turns out to be its great strength: it does **not** primarily ask individual upstream authors to sign their software. Instead the *distribution* signs, and it signs the **repository metadata**, with package integrity chained off that metadata.

For Debian/Ubuntu **apt**:

- A repository publishes a `Release` file listing every index (`Packages` , `Sources`) with its size and SHA-256 hash.

- The `Release` file is signed by a distro key. The signature is either detached (`Release.gpg`) or inlined (`InRelease` , a cleartext-signed `Release`).
- Each `Packages` index in turn lists every `.deb` with its SHA-256 hash.

So the trust chain is: `apt` trusts a keyring of distro keys → verifies the signature on `InRelease` → trusts the hashes in `Release` → verifies `Packages` matches → trusts the hashes in `Packages` → verifies each downloaded `.deb`. One signature transitively protects the entire repository, including which *versions* are current. That last point matters: because the signed `Release` is the authoritative statement of "current," an attacker cannot silently feed you a stale, vulnerable package version without breaking the signature — this defends against freeze/rollback attacks, a property TUF (Chapter 7) generalizes and formalizes.

For Red Hat / Fedora **RPM + yum/dnf**: individual RPMs can carry a GPG signature over the package header and payload (`rpm --checksig`), and repository metadata (`repomd.xml`) is signed similarly. The keyring is managed via `rpm --import` of distro-published keys, often pinned in the base install.

The distro model works because the trust relationships are few and stable: a handful of long-lived, carefully guarded distro master keys, a curated maintainer keyring, and metadata signing that covers the *whole* view of the repository rather than isolated artifacts. When we discuss what classic signing gets right, this is Exhibit A.

The OSS pattern: GPG detached signatures and the web of trust

The classic open-source release ritual: publish a release tarball `foo-1.2.3.tar.gz` , publish a **detached signature** `foo-1.2.3.tar.gz.asc` , and publish (somewhere) the maintainer's PGP public key. The consumer runs:

```
gpg --verify foo-1.2.3.tar.gz.asc foo-1.2.3.tar.gz
```

and is supposed to check that the signing key is the maintainer's genuine key. That last clause is where it falls apart. How do you know the key is genuine? PGP's answer was the **web of trust**: users sign each other's keys at key-signing parties, and you compute a trust path from your keys

to the maintainer's through mutual signatures. In theory, decentralized, no CA required.

In practice the web of trust largely failed, for reasons worth naming because they recur:

- **Usability.** Key management, trust levels, keyservers, and signature verification defeated even sophisticated users. Almost nobody actually computes trust paths; they run `gpg --verify`, see "Good signature," and ignore the accompanying `WARNING: This key is not certified with a trusted signature.`
- **No useful revocation or discovery.** Keyservers were unauthenticated, spammable (the 2019 certificate-flooding attacks broke key imports for targeted keys), and never had a coherent story for "is this the current key for this project?"
- **TOFU in disguise.** What people actually do is trust-on-first-use: grab whatever key the project's website links, and hope the website wasn't compromised. This collapses the whole edifice back to "trust the distribution channel," which the signature was supposed to remove from the trusted base.

The web of trust's failure is instructive precisely because Sigstore's keyless model (Chapter 4) is, in a sense, the anti-WoT: instead of asking humans to manage and cross-sign long-lived keys, it binds signatures to *machine-verifiable identities* (an OIDC token from an IdP, a workload identity) and records them in a public log so trust does not depend on anyone remembering to sign anyone else's key.

The rest, briefly

Java JAR signing (`jarsigner`) is essentially the v1 Android scheme's ancestor: per-entry hashes in the manifest, a signature over the manifest, the same ZIP-structure weaknesses, plus a historically weak default of SHA-1. **NuGet** package signing supports author and repository signatures (X.509, similar to Authenticode's CMS structures). **Maven Central** requires GPG-signed artifacts. None of these change the fundamental model; they are the same template with different file formats.

Here is the landscape in one table.

System	What it signs	Trust anchor	Key strength	Notable weakness
Authenticode	PE files, MSI, catalogs, drivers	Microsoft root program CAs	Broad tooling, timestamping, EV+SmartScreen reputation	Signer ≠ benign; stolen certs sign malware; slow revocation
Apple Developer ID + notarization	macOS apps/binaries (page-hash code directory)	Apple's single CA	Notarization adds a malware scan; effective revocation; hardened runtime	Centralized in one vendor; scan is best-effort
Android APK v2/v3/v4	Whole APK (v2+), key rotation (v3)	App's own signing key (Play App Signing: Google's)	Whole-file integrity, rotation, streaming verify	Key = app identity; v1 legacy weaknesses
apt / RPM repo signing	Repo metadata → package hashes	Distro keyring	Few stable keys; covers whole-repo view; anti-rollback	Trusts distro infra; upstream compromise still passes
GPG detached sigs (OSS)	Release tarballs	Web of trust / TOFU	Simple, decentralized in principle	WoT unusable; key discovery/revocation broken

The failure modes

Now the heart of the chapter. There are six recurring ways classic signing fails. They are not exotic; they are the same handful of problems, over and over, for thirty years.

Failure mode 1: key theft and compromise

The private key is the entire system. Steal it and you *are* the publisher, cryptographically indistinguishable from the real one. Book 4, Chapter 6 called signing keys the crown jewels of the build fleet; here is what happens when the jewels walk.

Stuxnet (2010) is the canonical case. The worm's kernel-mode drivers were signed with **legitimate code-signing certificates stolen from two Taiwanese hardware companies, Realtek Semiconductor and JMicron Technology**, whose offices were physically near each other. On 64-bit Windows, unsigned kernel drivers will not load — so the attackers needed a valid driver signature, and rather than break the cryptography they stole keys that produced valid signatures. The signature verified perfectly. The trust model performed *exactly as designed* and delivered a rootkit into the kernel, because the model's job is to confirm the signer, and the (stolen) key was, cryptographically, the signer.

NVIDIA (2022) shows a nastier variant. In the LAPSUS\$ breach, attackers exfiltrated and leaked internal data including **two code-signing certificates**. Even though the certs were *expired*, malware signed with them still loaded as kernel drivers on Windows, because Windows will honor a driver signature if it was (or appears to have been) timestamped during the certificate's validity window — expiry does not automatically invalidate an already-timestamped driver signature. Security researchers observed a variety of malware (Cobalt Strike beacons, Mimikatz, backdoors) freshly signed with the leaked NVIDIA certs in the weeks after the leak. The community's stopgap was to build Windows Defender Application Control (WDAC) policies explicitly *blocklisting* those specific certificates by hash — a manual, reactive countermeasure that only helps hosts that receive and enforce the policy.

Around these headline cases sits a steady, unglamorous **market in stolen and fraudulently obtained code-signing certificates**. Underground vendors have sold valid (sometimes EV) code-signing certs for years, because a signature dramatically improves malware's odds of running and evading detection. The certificate is not a nuisance to the attacker; it is *loot*.

The common root cause: **a long-lived, high-value secret exists and can be exfiltrated**. The certificate might be valid for one to three years, it sits somewhere (a developer laptop, a build server, occasionally an

HSM), and for its entire lifetime it is a single point of catastrophic failure. This is the observation that drives Fulcio's design (Chapter 3): issue a certificate that lives for *ten minutes*, so there is essentially no long-lived key to steal.

```
sequenceDiagram
    participant Att as Attacker
    participant Vic as Victim publisher
    participant Con as Consumer / OS loader
    Att->>Vic: breach network, exfiltrate signing key + cert
    Note over Att: attacker now holds a valid signer identity
    Att->>Att: sign malware with stolen key
    Att->>Con: distribute signed malware
    Con->>Con: verify signature -> chains to trusted root -> VALID
    Con->>Con: load / execute (e.g. kernel driver)
    Note over Con: owned. signature did its job on the wrong bits.
```

Failure mode 2: signed-but-malicious (the central lesson)

Key theft at least involves a stolen key. The deeper problem needs no theft at all: **a legitimate publisher, using its own genuine key, signs malware — because the publisher's build or release process was compromised.** The signature is *authentic*. It verifies against the real cert issued to the real company. And it is worthless as a safety signal, because what got signed was poison.

SolarWinds (2020) is the defining example, dissected at length in Book 1, Chapter 3. The SUNSPOT implant lived *inside SolarWinds' own build pipeline* and injected the SUNBURST backdoor into the Orion product **during compilation**, before signing. SolarWinds then signed the trojaned Orion DLL with **its own legitimate, valid code-signing certificate**, and pushed it through its normal update channel to ~18,000 customers. Every signature was genuine. Every verification passed. Code signing contributed *exactly nothing* to stopping this, because signing happens *after* the build, and the build was the thing that was owned. The signature faithfully attested "SolarWinds made this" — which was *true*, and useless.

CCleaner (2017) is the same shape. Attackers compromised Avast/Piriform's build environment and inserted a backdoor into CCleaner, which was then **signed with the valid CCleaner/Piriform certificate** and distributed to millions of users through the official download. Validly signed the whole way. **3CX (2023)** repeats it again: a compromised build

produced a trojanized 3CXDesktopApp — itself the downstream result of an *earlier* software-supply-chain compromise (a poisoned dependency, X_TRADER) — and the malicious build was **signed with 3CX's legitimate certificate**. Three incidents, years apart, identical lesson.

```
flowchart LR
    Src["Clean source"] --> Build["Build pipeline  
COMPROMISED"]
    Build --> Mal["Injected backdoor"]
    Mal --> Build
    Build --> Art["Trojaned artifact"]
    Art --> Sign["Sign with publisher's  
OWN valid key"]
    Sign --> Dist["Official update channel"]
    Dist --> Con["Consumer verifies signature"]
    Con --> Val["VALID: really is the publisher"]
    Con --> Own["Owns Executes backdoor"]
    style Build fill:#c0392b,color:#fff
    style Own fill:#c0392b,color:#fff
```

State the lesson as bluntly as possible, because it is the thesis of the entire book:

Classic code signing verifies the signer, not the software's integrity relative to its source. It answers "did this named entity sign these bits?" It cannot answer "were these bits built from the source we reviewed, by a pipeline we trust, without tampering?" When the build is compromised, the signer is compromised, and the signature is a rubber stamp on whatever the attacker produced.

This is *the* gap that provenance and attestation (Book 4, Chapter 3; Book 5, Chapter 6) exist to close. A SLSA provenance attestation says "this artifact was built from *this* source commit by *this* builder using *this* command" — a claim that, unlike a bare signature, would have been *false* for the trojaned SolarWinds DLL had the attacker not also controlled the builder, and that hermetic/reproducible builds (Book 4, Chapter 2) let a third party independently check.

Failure mode 3: CA and issuance failures

The model assumes the CA only issues code-signing certs to the legitimate entity named in them. That assumption has broken in several ways:

- **Fraudulent issuance.** Attackers register shell companies or impersonate real ones to obtain code-signing certificates through

legitimate CA channels. Vetting for standard (non-EV) code-signing certs has historically been weak — sometimes little more than a domain and a payment — so a determined attacker can simply *buy* a valid signing identity under a plausible name.

- **CA compromise.** The web-PKI history (DigiNotar 2011, Comodo 2011) shows CAs themselves get breached and mis-issue certificates. A compromised code-signing CA can mint certs that chain cleanly to a trusted root.
- **Cross-purpose and constraint gaps.** Misconfigured intermediates, missing name constraints, or improperly scoped EKUs occasionally let certificates be used for signing contexts their issuer never intended.

The root cause is that **identity vetting is done once, by a party the consumer never interacts with, and the consumer has no way to see what that party issued.** Which leads directly to the transparency argument (below and Chapter 5): Certificate Transparency for the web PKI exists precisely because CAs mis-issue and nobody could previously *detect* it. Fulcio applies the same medicine to code signing — every issued certificate is logged.

Failure mode 4: revocation does not work

Suppose you *detect* the misuse — a stolen key, a mis-issued cert, a compromised publisher. Now revoke it. In classic signing, revocation is where hope goes to die, for reasons Chapter 1 covered in the abstract and which bite hard here:

- **CRLs and OCSP are slow, incomplete, and often soft-fail.** Certificate Revocation Lists grow large and are fetched lazily; OCSP requires an online check the verifier frequently *skips* when the responder is unreachable (soft-fail), so an attacker who can block the OCSP request defeats revocation entirely. Many code-signing verification paths don't do a live revocation check at all.
- **Already-distributed artifacts.** Revoking a certificate does nothing to the millions of copies of the malware already installed. Revocation is a statement about *future* chain-building, not a recall of executed code. SolarWinds' cert was eventually revoked — long after the backdoor had been resident in target networks for months.

- **Expiry-honoring and stapling gaps.** As NVIDIA showed, a signature timestamped during validity survives cert expiry, so "just let it expire" is not revocation. And timestamped signatures may be honored even after revocation depending on how the verifier treats the timestamp — the semantics of "was this revoked *before or after* it was signed" are subtle and inconsistently implemented.
- **Distribution of revocation state is itself a distributed-systems problem.** Every verifier in the world needs the updated revocation info, promptly, or the revocation is theoretical. Apple's Gatekeeper can do this reasonably (it phones home); a Windows offline driver-verification path or an air-gapped host cannot.

The uncomfortable truth: **classic revocation assumes you can retract trust faster than the attacker can exploit it, and you usually cannot.** This is why modern designs lean on *short-lived* credentials (nothing to revoke — it already expired) and *transparency* (detect misuse fast so containment can begin), rather than betting on revocation propagating in time.

Failure mode 5: operational failures

Even with honest CAs, safe algorithms, and working revocation, classic signing tends to fail operationally because of how organizations actually run it:

- **Keys on developer laptops and build servers.** The most common real-world custody model is the worst one: a `.pfx` or `.p12` file (or a key in the CI secret store) that any build needs to reach in order to sign. Every machine that can sign is a machine from which the key can be stolen (failure mode 1). At fleet scale this is a target-rich disaster — more on this below.
- **No transparency.** This is the quiet killer. In classic signing, **nobody can enumerate what a key has signed.** A stolen key signs malware *in secret*; the legitimate owner has no log to consult, no way to notice "my key signed 4,000 things last night and I only signed 40." Misuse is invisible until a victim reports it. Compare the web PKI before Certificate Transparency: mis-issuance was undetectable until someone stumbled on a rogue cert. CT fixed that for TLS; nothing equivalent existed for code signing until Rekor (Chapter 5).
- **Expiry management pain.** Certs expire on their own schedule, unrelated to release schedules. Teams get paged because a signing

cert lapsed and the release pipeline is wedged, which pressures them toward long validity periods and lax renewal hygiene — the opposite of what security wants.

- **Weak algorithms living too long.** Authenticode and JAR signing shipped SHA-1 defaults well past SHA-1's cryptographic sunset. SHA-1 collisions (SHattered, 2017) mean a determined adversary could in principle produce two files with the same hash, one benign and signed, one malicious. Migrating a whole ecosystem off a weak algorithm is slow because old verifiers must keep accepting old signatures.

Failure mode 6: trust-store bloat and root governance

The consumer's trust anchors are a list of root certificates, and that list is a liability of its own. Root programs (Microsoft's, Apple's, Mozilla's) each trust dozens to hundreds of CAs, any one of which can issue a certificate the whole ecosystem will honor. The trust store is an **OR** of every CA in it: your security is that of the *weakest* trusted CA, not the strongest. Root programs are the governance layer trying to keep that OR from including something rotten — auditing CAs, enforcing baseline requirements, ejecting misbehaving ones (as browsers have done). It is real, valuable work, but it means the *ground* of the whole system is a large, slowly-changing, human-governed list that any consumer implicitly trusts in full.

Here is the failure landscape mapped to incidents, root causes, and the modern fix — the table to remember from this chapter.

Failure mode	Example	Mechanism / root cause	Modern approach that addresses it
Key theft	Stuxnet (2010), NVIDIA leak (2022)	Long-lived private key exfiltrated; valid signatures on malware	Short-lived / keyless certs (Ch 3-4) — no durable key to steal
Signed-but-malicious	SolarWinds (2020), CCleaner (2017), 3CX (2023)	Build/publisher compromised; signature authentic but on poisoned bits	Provenance + attestations + reproducible builds (Bk4 Ch2-3; Ch6)

Failure mode	Example	Mechanism / root cause	Modern approach that addresses it
CA / issuance failure	Fraudulent cert purchases; DigiNotar-class CA breaches	Weak or subverted identity vetting; consumer can't see issuance	Certificate Transparency model; Fulcio logs every cert (Ch 3, Ch 5)
Broken revocation	SolarWinds cert revoked months late; OCSP soft-fail	CRL/OCSP slow, skippable; already-installed code unaffected	Short-lived creds (nothing to revoke) + transparency for fast detect
No transparency	Stolen keys signing in secret; no CT-for-code	No public record of what was signed	Rekor append-only transparency log (Ch 5)
Operational (keys on hosts, SHA-1, expiry)	SHAttered (2017); build-server key sprawl	Bad custody, weak algs, expiry friction	Keyless (Ch 4), KMS/HSM custody (Ch 9), OIDC workload identity
Trust-store bloat	Root-program-wide mis-issuance risk	Trust = OR of every root CA	Scoped trust roots, policy-controlled verification (Ch 8, Ch 10)

What classic signing gets right

It would be dishonest, and bad engineering, to conclude that classic signing is worthless. It is not. It is *insufficient alone*, which is a different and more useful claim.

When keys are genuinely safe, classic signing delivers real integrity and real signer authenticity, and that is worth having. If SolarWinds had *not* signed Orion, an attacker who merely tampered with the download in transit — without compromising the build — would have succeeded trivially; the signature closed that (lesser) attack path. Signing raises the bar from "modify the bytes anywhere in the distribution channel" to "compromise the signer or steal the key." That is a meaningful bar. Most of the incidents in this chapter are notable precisely *because* clearing

that bar required a serious operation (a kernel-level build implant, a nation-state cert theft) rather than a trivial man-in-the-middle.

And **distro repository signing is a genuine, durable success**. Billions of `apt` and `dnf / yum` transactions run every day, protected by a small number of well-guarded keys, and key-theft incidents against major distro signing infrastructure are strikingly rare relative to the volume. Why does the distro model work where the OSS-tarball-plus-GPG model failed?

- **Few keys, professionally held.** A distribution has a handful of signing keys guarded by people whose job is guarding them — not thousands of individual maintainers each with a key on a laptop.
- **Signs the *metadata view*, not just artifacts.** Signing the `Release / repomd` file covers *which versions are current*, giving anti-rollback/anti-freeze protection for free. That is a property bare artifact signatures lack, and one TUF (Chapter 7) elevates to a first-class design goal.
- **A curated, bounded trust base.** The keyring is a small, known set, shipped with the OS and updated through the same signed channel — not a sprawling web of trust or a TOFU grab from a website.

The distro model is, in effect, a working template for *internal* package signing: a small number of well-guarded organizational keys signing repository metadata that covers your whole internal artifact view. When we get to enterprise key management (Chapter 9) and deployment gates (Chapter 10), "be more like a distro, less like a pile of developer `.pfx` files" is a recurring piece of advice.

The distributed-systems lens

Everything above gets *worse* at fleet scale, and the way it gets worse is the reason large organizations abandoned classic signing's custody model.

The key-management burden is untenable at scale. A serious backend organization ships thousands of artifacts — container images, binaries, packages, Helm charts, Lambda bundles — produced by hundreds of pipelines across dozens of teams. The classic model asks: *who holds the signing key, and on what machine?* There is no good answer. Give every team its own long-lived key and you have hundreds of crown jewels scattered across the org, each a failure mode 1 waiting to

happen. Centralize on one key and you create a bottleneck and a single catastrophic target, and you lose the ability to attribute *which team* signed *what*. Neither scales.

Long-lived keys distributed across a build fleet multiply theft risk linearly with fleet size. Book 4, Chapter 6 made the point: every build server that can sign is a machine from which a signing key can be stolen. A thousand build agents with access to a signing key is a thousand-way attack surface for the single most valuable secret you own. And because CI is, by design, a place that runs semi-trusted code (pipeline poisoning, Book 4, Chapter 7), it is *exactly* the wrong place to store a durable signing key — yet under classic signing it is exactly where the key must live to be usable.

No transparency means you cannot audit signing across the fleet. With hundreds of pipelines signing continuously, "did any of our keys sign something they shouldn't have?" is a question classic signing cannot answer at all. There is no log. A compromised pipeline can sign a malicious image and you will find out when a customer does. At fleet scale, the inability to *observe* signing is itself a critical control gap — you cannot detect what you cannot see.

These three pressures — custody, distributed theft surface, and unobservability — are exactly why organizations move to **keyless / workload-identity signing** (Chapters 3–4) and **centralized KMS/HSM** custody (Chapter 9):

- Keyless signing replaces the durable per-team key with an *ephemeral* certificate minted on demand against a *workload identity* (the CI job's OIDC token). The build server never holds a long-lived key because there is no long-lived key — the certificate expires in minutes. The thousand-way theft surface collapses: there is nothing durable on any agent to steal.
- Workload identity attributes signing to *which pipeline, from which repo, at which commit* produced the artifact — restoring the attribution that a single shared org key would have destroyed, and doing it automatically.
- A transparency log (Rekor) gives you the fleet-wide audit trail classic signing never had: every signature is a public, append-only record, so "what did our identities sign?" becomes a query, and anomaly detection becomes possible.

- Where durable keys genuinely must exist (a root of trust, an offline signing ceremony), they live in an HSM/KMS under hardware custody and tight access policy (Chapter 9), not in a CI secret store — the distro model, applied internally.

```

flowchart TB
    subgraph Classic["Classic signing at fleet scale"]
        K1["team A key"]
        on build server
        K2["team B key"]
        on build server
        K3["team C key"]
        on laptop
        Kn["... hundreds more ..."]
        K1 -->|"stealable"| Risk["long-lived keys"]
        = huge theft surface
        + no audit log
        K2 --> Risk
        K3 --> Risk
        Kn --> Risk
    end
    subgraph Modern["Keyless / workload-identity signing"]
        OIDC["CI OIDC identity"]
        per job --> Fulcio["Fulcio: mint 10-min cert"]
        Fulcio --> SignM["sign artifact"]
        SignM --> Rekor["Rekor: log the signature"]
        Rekor --> Audit["fleet-wide audit trail"]
        + nothing durable to steal
    end
    end

```

The through-line: classic signing's model of "durable secret, held by the publisher, verified against a root" was designed for a world of a few publishers shipping a few products on release cycles measured in months. It does not survive contact with a continuous-delivery fleet of thousands of artifacts and hundreds of pipelines. The rest of Book 5 is what survives contact — and each piece of it is a named answer to a named failure from this chapter.

Key takeaways

- **The classic model provides exactly two things: integrity and signer authenticity.** It provably does *not* establish that software is benign, or that it was built from any particular source by any particular pipeline. Assuming a third property is the root of most code-signing disasters.

- **Key theft turns the model against you.** Stuxnet (2010, stolen Realtek/JMicron certs) and the NVIDIA leak (2022, leaked certs still loading kernel drivers post-expiry via timestamping) show that a valid signature on malware is not a cryptographic failure — it is the system doing its job on the wrong bits. Long-lived keys are the liability.
- **Signed-but-malicious is the central lesson.** SolarWinds (2020), CCleaner (2017), and 3CX (2023) were all *validly signed with the real publisher's key*, because the build/publisher was compromised. Signing did nothing, because signing authenticates the signer, not the software's integrity relative to source. This is what provenance and attestation exist to fix.
- **Revocation is not a safety net you can rely on.** CRL/OCSP are slow and soft-fail, already-installed code is unaffected, and timestamped signatures survive expiry. By the time a cert is revoked, the damage is usually done — which is why the industry moved to short-lived credentials and transparency instead of betting on revocation.
- **The quiet killer is the lack of transparency.** In classic signing nobody can enumerate what a key has signed, so misuse is invisible until a victim notices. Certificate Transparency proved the fix for the web PKI; Rekor brings it to code signing (Chapter 5).
- **Classic signing is insufficient, not worthless.** It genuinely closes the tamper-in-transit path, and **distro repository signing (apt/RPM) is a real success** — few well-guarded keys, metadata signing with anti-rollback, a bounded trust base. That model is the template for internal package signing.
- **At fleet scale the custody model collapses.** Thousands of artifacts and hundreds of pipelines make "who holds the key, on what machine?" unanswerable; long-lived keys on build agents multiply theft risk; no log means no audit. This is precisely why organizations move to keyless/workload-identity signing (Chapters 3–4) and centralized KMS (Chapter 9).

Further reading

- Microsoft, "Windows Authenticode Portable Executable Signature Format" (specification) and the Microsoft Trusted Root Program documentation.
- Apple, "Notarizing macOS software before distribution" and "Code Signing Guide" (Apple Developer documentation).
- Android Open Source Project, "APK Signature Scheme v2 / v3 / v4" and "Application Signing" documentation.
- Debian, "SecureApt" wiki; and the `apt-secure(8)` and `Release` file format documentation.
- Fedora / RPM, "RPM package signing" and `createrepo / repomd` documentation.
- Falliere, Murchu, Chien (Symantec), "W32.Stuxnet Dossier" (2011) — the stolen Realtek and JMicron certificate details.
- NVIDIA LAPSUS\$ incident (2022): security-vendor analyses of the leaked code-signing certificates and the WDAC blocklist mitigation guidance from Microsoft.
- CISA and FireEye/Mandiant reporting on SolarWinds / SUNBURST / SUNSPOT (2020–2021); and Book 1, Chapter 3 of this suite.
- Talos (Cisco), "CCleanup: A Vast Number of Machines at Risk" (2017); Mandiant reporting on the 3CX supply-chain compromise (2023).
- Stevens et al., "The first collision for full SHA-1" (SHattered, 2017).
- Certificate Transparency (RFC 9162) — the transparency model this book applies to code signing (see Book 5, Chapter 5).
- Forward references in this suite: Book 5, Chapter 3 — Sigstore Architecture; Chapter 4 — Keyless Signing and Workload Identity; Chapter 5 — Transparency Logs; Chapter 6 — in-toto; Chapter 7 — TUF; Chapter 9 — Key Management and PKI for the Enterprise; Book

Chapter 3 — Sigstore Architecture: Cosign, Fulcio, Rekor

What this chapter covers. Chapter 2 dissected classic code signing and found it broken in predictable ways: the trust anchor is a **long-lived private key**, and that key is a single point of failure that gets stolen (Stuxnet's driver certs, the NVIDIA leak), mismanaged, or misused silently — nobody outside the signer can see what it signed. This chapter is about the system that reorganizes signing to eliminate that failure class: **Sigstore** (as of early 2026; public-good instance at <https://docs.sigstore.dev/> and <https://fulcio.sigstore.dev/> / <https://rekor.sigstore.dev/>), and its three core components — **Cosign** (the client), **Fulcio** (a certificate authority that issues certificates lasting *minutes*, not years), and **Rekor** (an append-only transparency log). The central idea is **keyless signing**: instead of a durable key that you protect forever, you authenticate an **OIDC identity**, receive a certificate that binds that identity to a throwaway key, sign, and let the key evaporate. Transparency and short-lived certificates together let a verifier check that signature *long after the certificate expired* — without anyone holding a long-lived key. We take the architecture apart component by component, walk the end-to-end **sign** and **verify** flows as sequence diagrams, and make explicit which of Chapter 2's failures each design choice defeats — and which it does not. Keyless signing's identity mechanics go deeper in Chapter 4; the Merkle-tree internals of the transparency log are Chapter 5's subject; here we build the whole-system picture.

Learning goals — after this chapter you should be able to:

- State what **Sigstore** is and is not: an **OpenSSF / Linux Foundation** project (launched ~2021) providing free, easy signing whose core innovation is **keyless signing** tied to an **OIDC identity plus a transparency log** — not a new cryptographic algorithm.
- Describe the role of each component precisely: **Cosign** signs and verifies artifacts and stores signatures in the OCI registry; **Fulcio**

issues **short-lived (~10-minute) X.509 code-signing certificates** binding an OIDC identity to an ephemeral key; **Rekor** is the **tamper-evident transparency log** of signing events.

- Explain the **verify-after-expiry** mechanism — how a short-lived certificate plus a signed log timestamp lets a verifier trust a signature made by a certificate that expired minutes after it was issued, with **no long-lived key anywhere**.
- Walk the **keyless sign** and **keyless verify** flows end to end, and articulate the key mental shift: you verify an **identity** (who/what signed, against policy), not a pre-shared public key.
- Distinguish Sigstore's **two logs** — the **CT log** (for the certificates Fulcio issues) and **Rekor** (for the artifact-signing events) — which are routinely confused.
- Map each of Chapter 2's failure modes to the specific Sigstore mechanism that addresses it, and state honestly the one it does *not* fix: **signed still does not mean safe**.
- Explain how the design fits distributed CI: workload identity as signer, self-hosted Sigstore for enterprises, and the transparency log as a fleet-wide signing audit.

What Sigstore is

Sigstore is a set of free, publicly operated services and open-source clients for signing, verifying, and *logging* software signatures. It began in **2021** as a collaboration led by engineers from **Google**, **Red Hat**, **Chainguard**, and Purdue University, and is now an **Open Source Security Foundation (OpenSSF)** project under the **Linux Foundation**, with contributors across the ecosystem. Its stated goal is disarmingly simple: make signing software as easy as `cosign sign`, so that signing becomes the default rather than a specialist chore — and in doing so, make **software provenance verifiable at scale**.

The important thing to understand up front is that Sigstore did **not** invent new cryptography. It signs with the same primitives Chapter 1 covered — ECDSA over P-256, Ed25519, SHA-256, X.509 certificates, Merkle trees. What Sigstore reorganizes is the **trust model** around those primitives. Classic signing (Chapter 2) makes a **long-lived private key** the durable root of trust: the key is the identity, it must be protected for years, and its compromise is catastrophic and often silent. Sigstore replaces the

durable key with a durable **identity** — an OIDC subject like an email address or, more importantly for CI, a **workload identity** — and reduces the private key to a **momentary** object that exists only for the seconds it takes to sign and is then discarded. The thing you protect forever is no longer a secret; it is an identity provider's account and a public log.

Two properties make that substitution work, and they are the twin pillars of the whole system:

- **Keyless signing.** A signer proves an OIDC identity to a certificate authority (**Fulcio**), which issues an **X.509 certificate valid for roughly ten minutes** that binds that identity to a freshly generated key pair. The signer signs, uploads the signature and certificate to a log, and throws the private key away. There is no long-lived key to steal, leak, rotate, or escrow. (Sigstore also supports traditional keys and KMS-backed keys — `cosign sign --key` — but keyless is the design's whole point and the default in Cosign v2.)
- **Transparency.** Every signing event is recorded in **Rekor**, an append-only, cryptographically tamper-evident public log. Because signatures are logged, misuse becomes **detectable**: an identity owner (or an automated monitor) can see every artifact ever signed under an identity — something classic signing, where a stolen key signs malware in silence, structurally cannot offer.

The public-good instance versus private Sigstore

There are two ways to consume Sigstore, and the distinction matters at fleet scale.

The **public-good instance** is the community-operated deployment: `fulcio.sigstore.dev`, `rekor.sigstore.dev`, a Certificate Transparency log, a timestamp authority, and a TUF-served trust root, run by the OpenSSF/Linux Foundation and reached GA in 2022 (state described as of early 2026 — check <https://docs.sigstore.dev/> for current provider/endpoint updates). It is free, accepts a fixed set of public OIDC providers (Google, GitHub, Microsoft, GitLab, and CI token issuers), and is what `cosign sign` uses out of the box. It is superb for open source: zero setup, publicly auditable, and every signature contributes to a public transparency record.

Private / self-hosted Sigstore is the same software (Fulcio, Rekor, the CT log, a TSA, and a TUF root) deployed inside an organization —

typically via the `sigstore/scaffolding` Helm charts on Kubernetes, wired to the enterprise's own OIDC provider. Enterprises run their own for three reasons: to avoid a hard runtime dependency on a public service on the artifact-verification critical path; to keep signing events (which reveal what you build and when) **internal** rather than in a public log; and to bind Fulcio to their **internal** identity provider so that certificates carry corporate workload identities. Chapter 9 develops the operational and PKI consequences of running private Sigstore; for now, hold the fact that every architectural component below is **self-hostable**, and the public instance is one deployment of it, not the system itself.

The components

```

flowchart TB
    subgraph client["Signer / Verifier host"]
        COSIGN["Cosign CLI  
sign · verify · attest"]
    end
    IDP["OIDC Identity Provider  
(Google, GitHub Actions,  
corporate IdP, ...)"]
    FULCIO["Fulcio  
short-lived X.509 CA  
(~10 min certs)"]
    CTLOG["CT Log  
(certificate transparency  
for Fulcio-issued certs)"]
    REKOR["Rekor  
transparency log  
(Merkle tree of signing events)"]
    TSA["Timestamp Authority  
(RFC 3161, optional)"]
    REG["OCI Registry  
(image + .sig / .att  
as OCI artifacts)"]
    TUF["TUF trust root  
(Fulcio/Rekor/CT/TSA  
public keys, rotated)"]

    COSIGN -->|"1. authenticate"| IDP
    IDP -->|"2. ID token"| COSIGN
    COSIGN -->|"3. token + ephemeral pubkey"| FULCIO
    FULCIO -->|"4. logs pre-cert"| CTLOG
    FULCIO -->|"5. signed cert (SCT embedded)"| COSIGN
    COSIGN -->|"6. signature + cert"| REKOR
    REKOR -->|"7. inclusion proof + SET"| COSIGN
    COSIGN -->|"8. push sig/cert as OCI artifact"| REG
    TUF -->|"root of trust for all keys"| COSIGN
    TSA -->|"optional signed timestamp"| COSIGN

```

Read the numbers as the signing path; the dashed edges are the trust infrastructure that makes the numbered steps verifiable. We now take each box apart.

Cosign — the client

Cosign is the CLI (and Go library) that engineers and CI actually invoke. It is the orchestrator: it drives the OIDC flow, talks to Fulcio and Rekor, computes digests, produces the signature, and — critically for the container world — **stores the signature in the OCI registry right next to the image**. Cosign signs three kinds of things: **container**

images (by digest), **arbitrary blobs** (cosign sign-blob), and **attestations** (in-toto statements wrapped in a DSSE envelope — SBOMs, SLSA provenance, VEX; see Book 5, Chapter 6 and Book 4, Chapter 3).

The essential Cosign verbs:

```
# Keyless sign a container image (by digest – always sign digests,
# never tags).
# In Cosign v2 keyless is the default; this triggers an OIDC flow.
cosign sign ghcr.io/acme/api@sha256:9b2a...c1

# Keyless verify – the identity flags are REQUIRED in v2 for keyless
# verification.
cosign verify \
  --certificate-identity "https://github.com/acme/api/.github/
workflows/release.yml@refs/heads/main" \
  --certificate-oidc-issuer "https://
token.actions.githubusercontent.com" \
  ghcr.io/acme/api@sha256:9b2a...c1

# Attach an attestation (e.g. an SBOM) as a signed in-toto statement.
cosign attest --type cyclonedx --predicate sbom.cdx.json \
  ghcr.io/acme/api@sha256:9b2a...c1

# Verify an attestation of a given predicate type against an identity.
cosign verify-attestation --type cyclonedx \
  --certificate-identity-regexp "^https://github.com/acme/.+" \
  --certificate-oidc-issuer "https://
token.actions.githubusercontent.com" \
  ghcr.io/acme/api@sha256:9b2a...c1
```

Two syntactic points matter. First, **you sign the digest, not the tag**. A tag is a mutable pointer; a `sha256:` digest is the content. Cosign resolves and signs the digest so the signature binds to immutable bytes. Second, in **Cosign v2** (released 2023) keyless verification *requires* `--certificate-identity` (or `--certificate-identity-regexp`) and `--certificate-oidc-issuer`. This is deliberate and is the single most important usability decision in the tool: **there is no "just verify it's signed."** You must state *whose* signature you expect. Verification is a policy statement about identity, and the CLI refuses to let you skip it. (Cosign v1 hid keyless behind `COSIGN_EXPERIMENTAL=1`; if you see that in old docs, it is obsolete.)

How Cosign stores signatures in the OCI registry

Cosign's cleverest piece of plumbing is that it needs **no separate signature database**. The OCI registry you already run is the signature store. There are two schemes, and both matter in practice.

The original scheme is a **tag convention**. For an image with digest `sha256:9b2a...c1`, Cosign computes a derived tag `sha256-9b2a...c1.sig` and pushes, under that tag, a small OCI image whose layer(s) carry the signature and certificate as **annotations** — `dev.cosignproject.cosign/signature` holds the base64 signature, and the associated certificate and Rekor bundle ride alongside. An attestation lands under `sha256-9b2a...c1.att`. This is a pure naming trick: replace the digest's `:` with `-`, append `.sig`, and any OCI 1.0 registry stores it with no special support. Its weakness is that the signature is not *cryptographically* linked to the image in the registry's own data model — it is linked only by the tag string.

The modern scheme is the **OCI 1.1 Referrers API** (Book 3, Chapter 5; Book 6, Chapter 1). OCI 1.1 added a `subject` field to manifests, letting an artifact declare "I am *about* that image digest," plus a `GET /v2/<name>/referrers/<digest>` API that returns everything referring to an image. Cosign can push signatures as referrers (`--registry-referrers-mode oci-1-1`), so the link is part of the registry's data model and a single Referrers query returns every signature, SBOM, and provenance for an image. Registries that lack native Referrers support fall back to a tag-schema index so clients get the same answer. Either way, the operational property is the same and it is a big deal at scale: **signatures travel with the image**. Pull the image to an air-gapped environment, mirror it to another registry, promote it between registries — the signature is right there under a derived reference, no out-of-band metadata service to keep in sync.

Fulcio — the short-lived certificate authority

Fulcio is a **certificate authority** with one unusual policy: it issues **X.509 code-signing certificates that are valid for roughly ten minutes**. You do not enroll, you do not prove key ownership over email for days, and you never renew. You present two things — an **OIDC identity token** and an **ephemeral public key** (freshly generated by Cosign for this one signing) — and Fulcio, after validating the token,

returns a certificate that **binds that identity to that public key**, valid from now until about ten minutes from now.

The validity window is the entire point. A ten-minute certificate is worthless to steal. There is nothing durable to exfiltrate: the private key lives only in Cosign's memory for the duration of one signature and is then discarded, and the certificate is expired before an attacker could do anything with it even if they grabbed it. Compare Chapter 2, where the whole game is protecting a key that must stay valid for one-to-three years.

What is *in* a Fulcio certificate is the mechanism that makes keyless verification identity-based:

- **Subject Public Key** — the ephemeral public key Cosign generated for this signing.
- **Subject Alternative Name (SAN)** — the **OIDC identity**. For a human authenticating with Google or GitHub, this is an **email address** (`rfc822Name`). For a **workload** — a CI job — it is a **URI SAN**, e.g. the GitHub Actions workflow identity `https://github.com/acme/api/.github/workflows/release.yml@refs/heads/main`. This is the field a verifier matches against `--certificate-identity`.
- **The OIDC issuer**, carried in a Fulcio custom X.509 extension (OID under the `1.3.6.1.4.1.57264.1` arc). This records *which identity provider* asserted the identity — `https://token.actions.githubusercontent.com` for GitHub Actions, `https://accounts.google.com` for Google, your corporate IdP for private Sigstore. A verifier matches this against `--certificate-oidc-issuer`. Identity *and* issuer together: `alice@example.com` via Google is a different signer than `alice@example.com` via a rogue IdP.
- **Additional workload claims**. For CI issuers, Fulcio maps extra OIDC claims into further custom extensions — the commit SHA, the workflow ref, the repository, the runner — so the certificate records not just "the release workflow" but the exact build context. These become policy surface in Chapter 4 and Book 4, Chapter 5.
- **A Signed Certificate Timestamp (SCT)**, embedded in the certificate, proving Fulcio logged this certificate to a **Certificate Transparency (CT) log** at issuance (see below).

Fulcio chains to a **Sigstore root CA** whose public key clients obtain through TUF (below). So the trust path a verifier walks is: *ephemeral*

signature → *Fulcio leaf cert* (identity is here) → *Fulcio intermediate* → *Sigstore root* (trusted via TUF).

Rekor — the transparency log

Rekor is an **append-only, tamper-evident transparency log** of signing events. When Cosign finishes signing, it uploads an **entry** to Rekor containing the **artifact digest**, the **signature**, and the **signing certificate** (the Fulcio cert, identity and all). Rekor stores that entry as a leaf in a **Merkle tree** (the cryptographic machinery is Chapter 5's subject; the one-line version is: every entry is hashed into a tree whose root commits to the entire log, so you cannot alter or remove a past entry without changing the root, and the log operator publishes signed roots so tampering is externally detectable). Rekor returns two things to Cosign:

- An **inclusion proof** — the Merkle audit path showing this specific entry is present in the tree under a particular **signed tree head** (checkpoint). Anyone can later recompute the path and confirm the entry is in the log.
- A **Signed Entry Timestamp (SET)** — Rekor's own signature over the entry's core fields, including its `logIndex`, the `logID`, and the **integratedTime** (when Rekor incorporated the entry). The SET is a **countersignature by the log**: it asserts, under Rekor's key, "this exact content existed in this log at this time."

Rekor serves two purposes, and the second is the subtle, load-bearing one.

Purpose 1 — transparency. Because every signing event is publicly logged, **misuse is detectable**. If an attacker compromises an identity and signs malware, that event lands in Rekor with the attacker-controlled identity in the certificate. The legitimate identity owner, or an automated monitor watching Rekor for a given identity or repository, can **see** it — a stark contrast with Chapter 2, where a stolen key signs in total silence and the first sign of compromise is often the malware itself in the wild.

Purpose 2 — verification after the certificate expires. This is the design's cleverest move, and it deserves its own section.

The Timestamp Authority and the CT log (briefly)

Two supporting services round out the architecture.

The **Certificate Transparency (CT) log** is where **Fulcio** logs every certificate it issues. It is a transparency log *for certificates*, not for signatures, and it is a **different log from Rekor** — a distinction that trips people up constantly. The CT log lets anyone audit what certificates Fulcio has issued (so a misbehaving or compromised CA is detectable), and Fulcio proves it logged a given cert by embedding an **SCT** in that certificate. Verifiers check the SCT to confirm the certificate was publicly logged.

The **Timestamp Authority (TSA)** is an optional **RFC 3161** service that issues a signed timestamp over the signature. Rekor's SET already provides a trusted signing time, so on the public instance the TSA is a belt-and-suspenders second source of time (and useful in deployments that want an RFC 3161 timestamp independent of the transparency log). Sigstore's newer **bundle** format can carry a TSA timestamp alongside the Rekor proof.

The Sigstore trust root — TUF

Everything above rests on clients knowing the **right public keys**: Fulcio's root CA cert, Rekor's public key, the CT log's key, the TSA's cert. If an attacker could feed a verifier a fake Fulcio root, they could forge the entire chain. Sigstore distributes these trust materials using **TUF — The Update Framework** (Book 5, Chapter 7), the same framework used to secure software update systems against a compromised distribution channel.

The Sigstore **TUF root** (served from a CDN, e.g. `tuf-repo-cdn.sigstore.dev`, and mirrored) is a set of TUF-signed metadata files whose top-level **root** role was established in a public **root-key signing ceremony** with multiple independent keyholders and a signing **threshold** — so no single person can forge the root, and compromise of one keyholder does not compromise the root. Cosign ships with a pinned copy of this trust root and updates it through TUF's secure workflow (`cosign initialize` refreshes it). The properties TUF buys here are exactly the ones Chapter 7 develops: **secure key rotation** (Fulcio or Rekor can roll keys and clients learn the new keys through signed metadata), **threshold trust** (no single key is fatal), and **resistance to a compromised delivery channel** (a malicious CDN cannot substitute keys without valid TUF signatures). For a private Sigstore deployment, you run your own TUF root and clients are pointed at it — this is how an

enterprise anchors verification in keys it controls rather than the public instance's.

Verify-after-expiry: how short-lived certs and a log combine

Here is the puzzle a skeptical reader should already be feeling. Fulcio certificates live for ten minutes. Artifacts live for years. If verification naively required the signing certificate to be *currently valid*, every Sigstore signature would become unverifiable ten minutes after it was made — which would be absurd. So how does a verifier, running a year later, trust a signature made by a certificate that expired 525,000 minutes ago?

The answer is that the verifier does **not** require the certificate to be valid *now*. It requires proof that the signature was made **while the certificate was valid** — and Rekor provides exactly that proof, in the form of the **integratedTime** carried inside the Rekor-signed **SET**.

```
flowchart LR
    subgraph win["Certificate validity window (~10 min)"]
        direction LR
        NB["NotBefore  
t0"]
        SIGN["Signature created  
at t0+30s"]
        LOG["Rekor entry  
integratedTime = t0+45s  
(inside window)"]
        NA["NotAfter  
t0 + 10 min"]
        NB --> SIGN --> LOG --> NA
    end
    LOG ==>|"SET signed by Rekor's key  
attests this time"| PROOF["Durable proof:  
'signed while cert valid'"]
    PROOF ==>|"checked years later"| VERIFY["Verifier (any time later):  
NotBefore ≤ integratedTime ≤ NotAfter  
⇒ signature is trustworthy"]
```

The logic the verifier runs is:

1. The signing **certificate** carries a validity window: **NotBefore** (t0) to **NotAfter** (t0 + ~10 min). The verifier does *not* check this against the current wall clock.

2. Rekor's **SET** provides a **trusted time**, `integratedTime`, signed by Rekor's key (obtained via TUF). This is not the verifier's clock and not the signer's clock — it is the log's attestation of when the entry was integrated.
3. The verifier checks that `NotBefore ≤ integratedTime ≤ NotAfter`. If the log's trusted timestamp falls inside the certificate's ten-minute window, then **the signature was made while the certificate was valid**, even though that window closed long ago.

That single inequality is what lets a short-lived certificate produce a *durable* signature. The certificate expiring is now irrelevant to verification; what matters is the **frozen, log-attested record** that the signing happened during its brief life. This is why "short-lived certs plus a transparency log" is not two independent features bolted together — it is one mechanism. The short-lived cert removes the long-lived key (the thing that gets stolen); the log's signed timestamp restores the "verify later" capability that a long-lived key used to provide. Take away the log and short-lived certs are useless for durable artifacts; take away the short lifetime and you are back to Chapter 2. Together they give you durable verifiability **with no long-lived key anywhere in the system**.

(The same reasoning explains why **revocation** largely disappears as a problem. In classic PKI, revoking a compromised long-lived cert via CRLs/OCSP is a notorious operational headache. A Fulcio cert is expired within minutes of issuance — there is essentially nothing to revoke. Compromise response shifts from "revoke the key" to "invalidate the *identity*, and audit Rekor for what that identity signed.")

The end-to-end keyless flow

We can now assemble the components into the two flows an engineer actually cares about.

Signing

```
sequenceDiagram
    autonumber
    participant Dev as Developer / CI job
    participant Cosign
    participant IdP as OIDC Provider
    participant Fulcio
    participant CT as CT Log
    participant Rekor
    participant Reg as OCI Registry

    Dev->>Cosign: cosign sign IMAGE@sha256:...
    Cosign->>Cosign: generate ephemeral key pair
    Cosign->>IdP: authenticate (browser flow, or ambient CI token)
    IdP-->>Cosign: OIDC ID token (identity + issuer)
    Cosign->>Fulcio: ID token + ephemeral public key
    Fulcio->>IdP: validate token signature + claims
    Fulcio->>CT: submit pre-certificate
    CT-->>Fulcio: SCT
    Fulcio-->>Cosign: short-lived cert (identity in SAN, SCT embedded)
    Cosign->>Cosign: sign artifact digest with ephemeral private key
    Cosign->>Rekor: upload {digest, signature, cert}
    Rekor-->>Cosign: inclusion proof + Signed Entry Timestamp
    Cosign->>Reg: push signature + cert (+ Rekor bundle) as OCI artifact
    Cosign->>Cosign: discard ephemeral private key
```

Step by step:

1. The developer or CI job runs `cosign sign` on an image *digest*.
2. Cosign generates a **fresh, ephemeral key pair** in memory for this one signing.
3. Cosign obtains an **OIDC ID token**. For a **human**, this is an interactive flow — Cosign opens a browser, you log in to Google/GitHub/your IdP, and the token comes back (Sigstore's OAuth flow uses a loopback/PKCE pattern so the token is bound to this session). For **CI**, it is **ambient**: the platform already exposes a workload OIDC token (GitHub Actions' `id-token: write` producing a token from `token.actions.githubusercontent.com`, or a cloud workload identity). No human, no secret — the runner's identity *is* the token. Chapter 4 is entirely about this half.
4. Cosign sends the **ID token and the ephemeral public key** to Fulcio.
5. Fulcio **validates the token** against the issuer's published keys and policy, then mints a **short-lived certificate** binding the token's

identity (into the SAN) and issuer to the ephemeral public key, and logs the cert to the **CT log** (embedding the SCT).

6. Cosign **signs the artifact digest** with the ephemeral private key.
7. Cosign **uploads** the signature, the certificate, and the artifact digest to **Rekor**, and receives an **inclusion proof and SET**.
8. Cosign **stores** the signature, certificate, and Rekor bundle in the **OCI registry** next to the image (tag scheme or Referrers).
9. Cosign **discards the ephemeral private key**. There is now no private key in existence anywhere. The durable outputs are: the signature and cert in the registry, and the entry in Rekor.

Verifying

```
sequenceDiagram
    autonumber
    participant V as Verifier (cosign verify / admission)
    participant Reg as OCI Registry
    participant TUF as TUF trust root
    participant Rekor

    V->>TUF: load trusted keys (Fulcio root, Rekor key, CT key)
    V->>Reg: fetch signature + cert (+ bundle) for IMAGE@sha256:...
    Reg-->>V: signature, Fulcio cert, Rekor bundle
    V->>V: verify signature over the artifact digest with cert's public key
    V->>V: verify cert chains to Fulcio root; check embedded SCT
    V->>V: extract identity (SAN) + issuer (OID) from cert
    V->>V: match identity + issuer against POLICY (expected signer)
    V->>Rekor: (or from bundle) verify inclusion proof + SET
    V->>V: check NotBefore <= integratedTime <= NotAfter
    V->>V: PASS only if every check holds
```

Step by step:

1. The verifier loads Sigstore's **trusted keys** from the **TUF** root (Fulcio root CA, Rekor's public key, the CT log key). This is the anchor for everything else.
2. It **fetches the signature and certificate** for the image digest from the registry (tag scheme or Referrers). In offline setups this comes from a **Sigstore bundle** shipped with the artifact.
3. It **verifies the signature** over the artifact digest using the public key **in the certificate**.

4. It **verifies the certificate chains** to the Fulcio root and checks the embedded **SCT** (confirming the cert was publicly logged, so a rogue-CA cert is detectable).
5. It **extracts the identity** (SAN) and **issuer** (OID extension) from the certificate.
6. It **matches identity and issuer against policy** — `--certificate-identity / --certificate-oidc-issuer` (or an admission-controller policy, or `--certificate-identity-regexp` for a family of workflows). **This is the step that makes verification meaningful.** The signer must be *the identity you expected* — "the `release.yml` workflow of `acme/api` on `main`, via GitHub's OIDC issuer" — not merely "some valid Sigstore signature."
7. It **verifies the Rekor inclusion proof and SET**, confirming the signing event is in the transparency log.
8. It checks that the log's `integratedTime` **falls inside the certificate's validity window** — the verify-after-expiry check from the previous section.
9. **Pass only if every check holds.** Fail closed on any failure.

The mental model to internalize — and the phrase to carry out of this chapter — is: **you verify the identity, not a pre-shared key**. In Chapter 2, verification answered "was this signed by key X?" and you had to have obtained key X's public half through some trusted side channel and pinned it. In Sigstore, verification answers "was this signed by **the identity my policy expects**, provably during a certificate's validity, and is that event in the public log?" You never pin a key. You pin an **identity and issuer**, which is far more meaningful to a human and far more stable operationally — the CI workflow that builds `acme/api` keeps the same identity string across thousands of signings and key rotations that no longer exist.

Why this fixes Chapter 2's failures

Chapter 2 catalogued the ways classic code signing breaks. Sigstore's design targets each one. Let us be explicit — and equally explicit about the one it does **not** fix.

Chapter 2 failure mode	Root cause	Sigstore mechanism	Fixed?
Stolen signing key (Stuxnet's Realtek/JMicron driver certs; the NVIDIA 2022 leak used to sign malware)	A long-lived private key is a durable, stealable secret	Keyless: ephemeral key exists for seconds, then discarded; ~10-min cert	Yes — nothing durable to steal
Silent misuse — a stolen key signs malware and nobody outside the signer can tell	Signing leaves no external record	Rekor transparency log records every signing event; monitorable per identity/repo	Yes — misuse is publicly detectable
Key management burden — HSMs, escrow, rotation, offline ceremonies (Chapter 2, Chapter 9)	Keys must be protected for years	No long-lived key to manage; identity is the durable trust anchor	Largely — burden shifts to IdP + log ops
Revocation is broken — CRL/OCSP rarely works in practice	Long-lived certs must be revocable	Certs expire in ~10 min; revoke the identity , audit Rekor	Mostly moots it
"Signed by key X" is not human-meaningful — you pin an opaque key	Trust anchor is a key blob	Trust anchor is an OIDC identity + issuer in the cert SAN	Yes — verify a named identity
Key distribution / pinning — verifiers must obtain the right public key out-of-band	Every verifier must pin every signer's key	TUF -served roots + Fulcio chain; verify by identity, not pinned key	Yes — no per-signer key pinning
Signing is hard, so people skip it	Ceremony, HSMs, secrets in CI	cosign sign with ambient CI identity; zero secrets	Yes — signing becomes the default

Chapter 2 failure mode	Root cause	Sigstore mechanism	Fixed?
Signed $\neq$ safe — a valid signature over malware is still a valid signature	Signing attests <i>who</i> , not <i>what's inside</i>	(none — out of scope)	No — see below

That last row is the honest limit and it is the same lesson Chapter 2 ended on. **Sigstore makes signing trustworthy; it does not make the signed thing good.** If an attacker compromises a CI workflow's identity — poisons the pipeline (Book 4, Chapter 7), steals the OIDC token's trust, subverts the build — they can produce a **perfectly valid** Sigstore signature over malware, because the malware really was signed by the real workflow identity. Verification passes. The xz-utils backdoor would have been *signed by the legitimate maintainer's identity*. Sigstore raises the bar from "steal a key" to "compromise an identity or a build," which is real and valuable progress, but it does not change the core truth: **a signature proves who/what signed, not that the artifact is safe.** What signing gives you is **accountability and non-repudiation**: if the release workflow signs malware, the transparency log names the release workflow, and you know exactly where to look. To assert more — that the artifact was *built the way you expect, from the source you expect* — you need **provenance and policy** layered on top: SLSA provenance (Book 4, Chapter 3), in-toto attestations (Chapter 6), verified against policy at admission (Chapter 10; Book 6, Chapters 5–6). Signing is the *transport* for those attestations, not a substitute for them. Keep the axes separate exactly as Book 4, Chapter 3 insisted: signing establishes identity; provenance establishes build integrity; neither is the other.

Distributed-systems lens

Keyless signing is, in a sense, a technology *designed for* large-scale CI — the properties that feel like conveniences for an individual are structural wins for a fleet.

Workload identity is the signer; there is no secret to distribute. The central pain of signing across a large organization has always been the key: to sign in CI you had to put a private key *somewhere* the pipeline could reach — a CI secret, a mounted HSM, a KMS grant — and then defend that key across hundreds of repos and runners, any one of which

leaking it compromises everything. Sigstore deletes the key. The **build's OIDC workload identity** is the signer (Book 4, Chapter 5 and Chapter 6; Chapter 4 here). There is no secret to provision, rotate, scope, or leak across the fleet. A thousand repositories on GitHub Actions each sign under their own workflow identity with zero key material configured. This is the difference between a control that scales and one that doesn't: the classic model's cost grows with the number of signers; the keyless model's cost is roughly constant.

Signing becomes a platform property, not a per-repo chore.

Because the signer is ambient identity and the client is a one-line `cosign sign`, the **paved-road build platform** (Book 4, Chapter 10) can sign **every** artifact automatically — image, SBOM, provenance — with the build's own identity, and the tenant team writes none of it. Every artifact leaving the platform is signed by construction. That is only feasible because there is no key to hand each team.

Verification is identity-and-policy, enforced at admission.

Downstream, the deploy gate does not ask "is this signed?" — it asks "is this signed by **the identity our policy allows?**" A Kubernetes admission controller (Sigstore's `policy-controller`, or Kyverno; Book 6, Chapters 5–6; Chapter 10) rejects any image whose signature does not chain to Fulcio *and* whose certificate identity does not match the expected workflow, failing closed:

```
# sigstore policy-controller ClusterImagePolicy (illustrative)
apiVersion: policy.sigstore.dev/v1beta1
kind: ClusterImagePolicy
metadata:
  name: require-acme-release-signature
spec:
  images:
    - glob: "ghcr.io/acme/**"
  authorities:
    - keyless:
        url: https://fulcio.sigstore.dev
      identities:
        - issuer: https://token.actions.githubusercontent.com
          subjectRegExp: "^https://github.com/acme/.+/\.\.github/
workflows/release\.\.yaml@.*"
```

The policy names an **identity pattern**, not a key. Rotate nothing; the identity is stable. (For private Sigstore, `url` points at the internal Fulcio and the `issuer` at the corporate IdP.)

Self-hosted Sigstore removes the public-instance dependency. At enterprise scale you generally do not want the *verification* path — which may run on every deploy and every admission decision — to depend on a public service's availability or rate limits, nor do you want your **signing events (which reveal what you ship and when) in a public log**. Running **private Fulcio, Rekor, CT, TSA, and TUF** (Chapter 9) keeps signing events internal, binds certificates to your own workload identities, and puts the trust root under your control. The architecture is identical; only the operator changes.

The transparency log is a fleet-wide signing audit. Rekor is not only a verification input — it is a **queryable record of everything your organization has signed**. A private Rekor becomes an audit substrate: which identities signed which digests, when, from which build context. You can monitor it for anomalies (a signature from an identity that should never sign, an artifact signed outside the release workflow), reconstruct incident timelines, and answer "what did this compromised identity ever sign?" — the question that is *unanswerable* in the classic model where signing leaves no trace. Book 4, Chapter 9 (build observability) and Book 8 (incident response) both draw on this.

The through-line: Sigstore is what lets the **build-integrity story of Book 4 actually leave the build**. Provenance and SBOMs are just data until something signs them with a verifiable identity and logs them so a deploy gate can trust them later. Sigstore is that something. It is the cryptographic transport that carries every other attestation in this suite from the platform that produces it to the gate that enforces it (Chapter 10; Book 6).

Key takeaways

- **Sigstore** (OpenSSF / Linux Foundation, launched ~2021) makes signing free and easy. Its innovation is not new crypto but a new **trust model**: replace the long-lived **key** with a durable **identity** plus a public **transparency log**. Three core components: **Cosign** (client), **Fulcio** (short-lived CA), **Rekor** (transparency log).
- **Cosign** signs images, blobs, and attestations, and stores signatures **in the OCI registry next to the image** — as a `sha256-<digest>.sig` tag artifact or, modern default, an **OCI 1.1 referrers**. Signatures travel with the artifact; no separate signature database. Cosign v2

requires `--certificate-identity` and `--certificate-oidc-issuer` for keyless verify.

- **Fulcio** issues **X.509 certs valid for ~10 minutes** binding an **OIDC identity** (in the **SAN** — email for humans, a **URI workload identity** for CI) and its **issuer** (a custom OID) to an **ephemeral** public key. There is no long-lived key to steal; the private key is discarded after signing.
- **Rekor** is an **append-only, tamper-evident Merkle-tree transparency log** of signing events. It returns an **inclusion proof** and a **Signed Entry Timestamp (SET)** carrying a trusted `integratedTime`. It provides both **transparency** (misuse is publicly detectable) and the **verify-after-expiry** timestamp.
- **Verify-after-expiry** is the linchpin: a verifier does *not* require the cert to be valid now; it checks `NotBefore ≤ integratedTime ≤ NotAfter` using Rekor's signed time. Short-lived certs remove the stealable key; the log's signed timestamp restores durable "verify later" — one mechanism, not two, with **no long-lived key anywhere**.
- There are **two logs**, routinely confused: the **CT log** logs the **certificates Fulcio issues** (proved by an embedded SCT); **Rekor** logs the **artifact-signing events**.
- **TUF** distributes Sigstore's own trust root (Fulcio/Rekor/CT/TSA keys) with threshold signing and secure rotation, so clients get and update the right keys through a compromise-resistant channel (Chapter 7). Everything is **self-hostable**; the public `sigstore.dev` instance is one deployment, and enterprises run **private Sigstore** (Chapter 9).
- **You verify an identity, not a pinned key.** Verification is a policy statement: signed by *the expected workflow identity*, via *the expected issuer*, logged in Rekor. This is more meaningful and more stable than pinning opaque public keys.
- Sigstore defeats Chapter 2's failure classes — stolen keys, silent misuse, key-management burden, broken revocation, key pinning, and the friction that made people skip signing — but **not the last one: signed ≠ safe**. A compromised CI identity can sign malware with a valid signature. Signing proves **who/what** signed; **provenance + policy** (Book 4, Chapter 3; Chapters 6, 10; Book 6) prove build integrity. Keep the axes separate.

- **Distributed-systems view:** the build's **workload identity** is the signer — **no secret to distribute** across the fleet; the **paved-road platform** signs every artifact automatically (Book 4, Chapter 10); verification is **identity + policy at admission** (Book 6, Chapters 5–6; Chapter 10); **private Sigstore** removes the public dependency and keeps events internal (Chapter 9); and **Rekor is a fleet-wide signing audit**.

Further reading

- **Sigstore documentation** — the project overview, `cosign` reference, and the Fulcio/Rekor service docs. <https://docs.sigstore.dev/>.
- **Cosign** — the CLI and signature-spec (OCI signature storage, tag scheme, and Referrers support). <https://github.com/sigstore/cosign> and the *Cosign Signature Specification*.
- **Fulcio** — the certificate authority: certificate profile, SAN identities, OID extensions, and the ~10-minute validity policy. <https://github.com/sigstore/fulcio> and its `docs/`.
- **Rekor** — the transparency log: entry types, inclusion proofs, Signed Entry Timestamp, and the Merkle-tree design. <https://github.com/sigstore/rekor> (Merkle internals in Chapter 5).
- **Sigstore: the internals of Sigstore** and the "**How Sigstore works**" guide — the end-to-end keyless sign/verify flow and the verify-after-expiry reasoning. <https://docs.sigstore.dev/>.
- **The Update Framework (TUF)** and Sigstore's **root-signing** repository — the trust root and key-rotation model. <https://theupdateframework.io/> and <https://github.com/sigstore/root-signing> (developed in Chapter 7).
- **Certificate Transparency (RFC 6962)** — the model the Fulcio CT log follows, and the SCT. <https://www.rfc-editor.org/rfc/rfc6962>.
- **sigstore/policy-controller** and **Kyverno** — admission-time keyless verification for Kubernetes (identity-based image policies). Applied in Chapter 10 and Book 6, Chapters 5–6.
- **sigstore/scaffolding** — Helm-based self-hosted Sigstore for private deployments (Chapter 9).
- Cross-references: Book 5, Chapter 1 (crypto foundations), Chapter 2 (classic code signing and its failures), Chapter 4 (keyless signing and

workload identity), Chapter 5 (transparency logs), Chapter 6 (in-toto attestations), Chapter 7 (TUF), Chapter 9 (key management / private Sigstore), Chapter 10 (deployment gates); Book 4, Chapters 3, 5-7, 10 (provenance, workload identity, pipeline poisoning, secure build platform); Book 3, Chapter 5 (attestation storage, OCI referrers); Book 6, Chapters 1, 5-6 (OCI artifacts, admission-time verification).

Chapter 4 — Keyless Signing and Workload Identity

What this chapter covers. Chapter 3 built the whole-system picture of Sigstore and named its central innovation **keyless signing**, but treated the identity that keyless signing binds to as a given. This chapter opens that box. First it dispels the word "keyless" itself — there *are* keys; what is gone is the **long-lived, human-managed** key that Chapter 2 spent a whole chapter watching get stolen. Then it goes deep on the mechanism that makes keyless signing work for the systems a backend engineer actually cares about: **automated pipelines with no human at the keyboard**. The answer is **workload identity** — a CI job proving *what it is* ("I am the `release.yml` workflow of `acme/api`, building commit `abc123` on `main`") through a short-lived, provider-signed token, without any stored secret. We take GitHub Actions OIDC apart claim by claim, survey the equivalent mechanisms on GitLab, AWS, Google Cloud, and other platforms, and give **SPIFFE/SPIRE** — the vendor-neutral workload-identity standard — the full treatment it deserves, because it is the same primitive that will show up again as the runtime service identity in zero-trust networking. Finally we return to verification, where the entire security value lives or dies on one thing: **getting the identity-matching policy right**, tight enough that "any workflow in the org" cannot masquerade as "the release workflow."

Learning goals — after this chapter you should be able to:

- Explain precisely what **"keyless"** does and does not mean: no long-lived user-managed key, an **ephemeral keypair per signing** used once and discarded, identity (not a key) as the durable trust anchor — and why this structurally removes Chapter 2's key-theft failure class.

- Give an **OIDC refresher** sufficient to reason about signing: the ID token as a signed JWT, its standard claims (`iss` , `sub` , `aud`), and how **Fulcio** validates that token and binds its identity into the certificate.
- Distinguish **human identity** (interactive browser OIDC → email in the cert) from **workload identity** (ambient machine token → workflow URI in the cert), and explain why CI needs the latter.
- Read a real **GitHub Actions OIDC token** claim by claim — `sub` , `repository` , `repository_owner` , `ref` , `workflow` , and especially `job_workflow_ref` — and explain the `id-token: write` permission and the token endpoint that produce it.
- Map the workload-identity mechanisms of **GitLab CI**, **AWS**, **Google Cloud Workload Identity Federation**, and others onto the same shape, and explain **SPIFFE/SPIRE** as the general fabric: the **SPIFFE ID**, **X.509-SVID** and **JWT-SVID**, and **node + workload attestation**.
- Write correct `cosign verify identity policies ( --certificate-identity / --certificate-identity-regexp , --certificate-oidc-issuer )` and recognize the **over-broad matching footgun**.
- State the honest security ledger: what keyless workload signing wins, and the new trust base it takes on (the IdP, Fulcio, Rekor, and the attestation of the workload identity itself).

"Keyless" is a misnomer — say what it really means

Start by killing the word. **Keyless signing is not signing without keys**. Signing is a mathematical operation over a private key; there is no such thing as a signature without one. What "keyless" names is the absence of a *particular kind* of key — the **long-lived, user-managed** private key that Chapter 2 built its entire failure taxonomy around.

Here is the mechanism, compressed. For each signing operation, the Cosign client generates a **fresh ephemeral keypair in memory** — typically ECDSA over P-256. It obtains an OIDC identity token, sends the token and the ephemeral *public* key to Fulcio, and receives back a certificate — valid for roughly ten minutes — that binds *this identity* to *this public key*. It signs the artifact digest with the ephemeral *private* key, uploads the signature and certificate to Rekor, and then **throws the private key away**. The keypair existed for the duration of one signature — seconds — and then ceased to exist. Nothing was written to disk, no HSM slot was provisioned, no secret was mounted into the runner.

Now re-read Chapter 2's central failure through that lens. The whole game in classic signing is protecting a private key that must stay valid for one to three years, because that key *is* the identity — anyone holding it can sign as you, forever, silently. Stuxnet's operators stole Realtek's and JMicron's legitimate driver-signing keys; the 2022 NVIDIA breach leaked code-signing certificates that were promptly reused to sign malware. Every one of those incidents is an instance of the same structural fact: **a durable secret is a durable liability**. It sits somewhere — a build server, an HSM, a developer laptop, a CI secret store — for the entire time it is valid, and that entire time is a window of theft.

Keyless signing closes the window by making it a few seconds wide and leaving nothing behind.

```
flowchart LR
    subgraph classic["Classic signing (Chapter 2)"]
        K1["Long-lived private key  
valid 1-3 years"]
        K1 -->|"lives on disk / HSM / CI secret"| STEAL["Stealable for the  
entire validity window"]
        K1 -->|"IS the identity"| SILENT["Stolen key signs  
silently, as you"]
    end
    subgraph keyless["Keyless signing (this chapter)"]
        E1["Ephemeral private key  
valid seconds"]
        E1 -->|"exists in memory only,  
then discarded"| NOTHING["Nothing durable to steal"]
        E1 -->|"identity comes from  
OIDC token, not the key"| IDENT["Identity is the anchor;  
key is disposable"]
    end
```

The trade is not "no keys" but **"key you keep" for "identity you prove."** The durable thing you now protect is not a secret file — it is an **account with an identity provider** and the integrity of a public log. That is a different, and for automated systems a far better, thing to defend, because the properties of "an account at an IdP" (revocable, auditable, MFA-protected, centrally managed) are exactly the properties "a private key on a build server" never had. The rest of this chapter is about where that identity comes from when there is no human to log in.

OIDC and identity — just enough to reason about signing

Keyless signing rests entirely on **OpenID Connect (OIDC)**, so we need a precise-but-compact model of it. (Book 9, Chapter 6 — OAuth 2.0 and OpenID Connect — is the full treatment; here we take only what signing needs.)

OIDC is an **identity layer on top of OAuth 2.0**. The cast:

- An **identity provider (IdP)** — Google, GitHub, GitLab, a corporate Okta/Entra tenant, or a CI platform's token service — that can authenticate a principal and vouch for who it is.
- An **ID token**: a **JWT** (JSON Web Token) the IdP issues, containing **claims** about the principal, and **signed by the IdP's private key**. Anyone can verify that signature using the IdP's public keys, which the IdP publishes at a well-known JWKS endpoint discoverable from its OIDC metadata (`/well-known/openid-configuration`).

A JWT is three base64url segments — header, payload, signature — joined by dots. The payload carries **standard claims** defined by the spec plus **provider-specific claims**. The three that matter most for signing:

- **iss** (issuer) — *who asserted this identity*. A URL, e.g. `https://token.actions.githubusercontent.com` for GitHub Actions or `https://accounts.google.com` for Google. This is the anchor for trust: you verify the token's signature against *this issuer's* published keys, and no other.
- **sub** (subject) — *who or what the token is about*, in the issuer's namespace. For a human this might be an opaque user ID or an email-bearing token; for a workload it is a **structured string describing the job** (we will dissect GitHub's shortly).
- **aud** (audience) — *who the token is for*. The intended recipient. A verifier must check that the token was minted for it and not replayed from somewhere else. Fulcio requires the audience to be `sigstore`.

Plus the housekeeping claims every JWT carries: `iat` (issued-at), `nbfi` (not-before), `exp` (expiry — OIDC ID tokens are short-lived, minutes), and `jti` (a unique token ID).

How Fulcio turns a token into a certificate identity

When Cosign sends Fulcio an ID token and an ephemeral public key, Fulcio does four things (Chapter 3 walked the flow; here is the identity-binding detail):

1. **Validates the token.** It fetches the issuer's public keys from the issuer's JWKS endpoint and verifies the JWT signature, then checks `exp / nbf` and that `aud == sigstore`. A token whose signature does not verify against the *claimed issuer's* keys is rejected — this is what stops an attacker minting their own tokens.
2. **Selects an identity mapping.** Fulcio has a configured set of accepted issuers, and for each issuer a rule for *which claim becomes the certificate's identity*. For a human Google/GitHub login it is the email; for GitHub Actions it is the `job_workflow_ref` claim; for GitLab it is the `ci_config_ref_uri` claim. This per-issuer mapping is the crux of the whole scheme and we return to it below.
3. **Mints a short-lived certificate** whose **Subject Alternative Name (SAN)** is that identity — an `rfc822Name` for an email, a **URI SAN** for a workload — and whose **custom X.509 extensions** (under the `1.3.6.1.4.1.57264.1` OID arc) record the issuer and, for CI issuers, a rich set of the other claims (repository, ref, commit SHA, runner environment, and so on).
4. **Logs the certificate** to the CT log and returns it (SCT embedded) to Cosign.

The upshot: **the OIDC token's identity becomes the certificate's identity**, cryptographically bound to the ephemeral key that will do the signing. Verification later (last section) is a check against *that identity*, never against a key you pinned.

Human identity versus workload identity

There are two ways to obtain the OIDC token in step 1, and they define two different modes of Sigstore use.

Human / interactive. A developer runs `cosign sign` at a terminal. Cosign opens a browser to the IdP's authorization endpoint (using an OAuth loopback + PKCE flow so the resulting token is bound to this local session), the human authenticates — possibly with MFA — and the token comes back with their **email** as the identity. The Fulcio certificate's SAN is `alice@example.com`; the issuer OID records `https://`

`accounts.google.com`. This is fine for a maintainer signing a release by hand, and it is how a lot of open-source signing happens. But it does not scale and it is not what a pipeline can do: **there is no human in a CI job to click a browser button.**

Workload / ambient. A CI job runs `cosign sign` with no browser and no human. The token is **ambient**: the platform the job runs on *is itself an OIDC provider*, and it has already made a signed token describing the running job available to that job through the environment. Cosign detects the CI environment, fetches the ambient token, and proceeds. The certificate's SAN is not an email — it is a **URI describing the workload**, e.g. `https://github.com/acme/api/.github/workflows/release.yml@refs/heads/main`. **This is the mode that matters for everything a backend engineer builds**, and it is the rest of this chapter.

Workload identity in CI/CD — the core

The problem statement

A CI job needs to prove a specific, verifiable claim about itself —

```
"I am the build produced by the release.yml workflow of the acme/api repository, running against commit abc123 on branch main, on a GitHub-hosted runner."
```

— to a party (Fulcio, a cloud IAM endpoint, a secret manager) that has never seen this job before, **without presenting any stored secret**. The "without a stored secret" clause is the whole difficulty. The obvious pre-keyless approach — put a long-lived credential (a signing key, a cloud access key, a service-account JSON) into a CI secret — is exactly the durable liability we are trying to eliminate. It sits in the secret store for years; it is copied into every runner that needs it; it leaks through log exposure, a malicious dependency in the build (Book 4, Chapter 7 — pipeline poisoning), a compromised action, or an insider; and once leaked it authenticates as the pipeline from anywhere until someone notices and rotates it.

The solution reframes the CI platform's role. **The platform becomes an OIDC identity provider.** Because the platform is the thing that actually schedules and runs the job, it is uniquely positioned to *know* what the job is — which repo, which workflow file, which ref, which commit, which

runner — and to **assert those facts in a signed, short-lived token** minted freshly for that job and delivered only to it. The job proves its identity not by holding a secret but by holding a token the platform made about it, on the fly, that expires in minutes. There is nothing durable to steal because the credential *is created per-run and dies with the run*.

```
sequenceDiagram
    autonumber
    participant Job as "CI job (no human, no stored key)"
    participant Plat as "CI platform OIDC IdP"
    participant Cosign
    participant Fulcio
    participant Rekor
    participant Reg as "OCI registry"

    Job->>Plat: request ID token (audience = sigstore)
    Plat->>Plat: assemble claims from job context
    (repo, ref, workflow, sha, runner...)
    Plat-->>Job: signed, short-lived ID token (JWT)
    Job->>Cosign: cosign sign IMAGE@sha256:...
    Cosign->>Cosign: generate ephemeral keypair
    Cosign->>Fulcio: ID token + ephemeral public key
    Fulcio->>Plat: fetch issuer JWKS, verify JWT signature
    Fulcio->>Fulcio: map job_workflow ref -> SAN identity
    Fulcio-->>Cosign: ~10-min cert (workload identity in SAN)
    Cosign->>Cosign: sign digest with ephemeral private key
    Cosign->>Rekor: upload {digest, signature, cert}
    Rekor-->>Cosign: inclusion proof + SET
    Cosign->>Reg: push signature + cert as OCI artifact
    Cosign->>Cosign: discard ephemeral private key
```

The signer, at the end of this, is provably and specifically **the workload** — not "someone with a key", not "the acme org", but *that exact build context*. Let us see how each platform produces that token.

GitHub Actions OIDC

GitHub Actions is the reference implementation and the one most readers will meet first, so we do it in detail.

Turning it on. A workflow requests an OIDC token by granting itself the `id-token: write` permission. This permission is **off by default** and must be explicitly granted — either at the workflow top level or per-job. Granting it is what makes the runner expose the token endpoint to the job.

```

name: release
on:
  push:
    tags: ["v*"]

permissions:
  contents: read
  id-token: write          # REQUIRED for OIDC / keyless signing

jobs:
  build-and-sign:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: sigstore/cosign-installer@v3
      # ... build and push image, capture its digest ...
      - name: Keyless sign
        run: cosign sign --yes ghcr.io/acme/api@$
    {{ steps.build.outputs.digest }}
  # No key, no secret. Cosign detects Actions, fetches the
  ambient
  # OIDC token, and drives the Fulcio -> Rekor flow.

```

Note the principle of least privilege even here: `contents: read`, `id-token: write`, nothing more. The `id-token: write` name is slightly misleading — it does not let the job *write* anything durable; it lets the job *request* an ID token be written for it. It is the switch that turns on OIDC for the job.

The token endpoint. When `id-token: write` is granted, the runner injects two environment variables into the job:

- `ACTIONS_ID_TOKEN_REQUEST_URL` — a per-job URL on GitHub's token service.
- `ACTIONS_ID_TOKEN_REQUEST_TOKEN` — a short-lived bearer token authorizing the request to that URL.

Code fetches an ID token by calling that URL, optionally passing an `audience` query parameter (the `aud` claim the returned token will carry). Cosign requests `audience=sigstore`; a job federating to a cloud would request that cloud's audience.

```
# What Cosign (and the GitHub toolkit's core.getIDToken) do under the hood.  
curl -sS -H "Authorization: bearer $ACTIONS_ID_TOKEN_REQUEST_TOKEN" \  
"$ACTIONS_ID_TOKEN_REQUEST_URL&audience=sigstore" | jq -r '.value'  
# -> eyJ... (a signed JWT, the OIDC ID token for THIS job)
```

The issuer is always `https://token.actions.githubusercontent.com`, whose JWKS Fulcio (and any verifier of the raw token) fetches to check the signature.

The claims. Decode that JWT's payload and you get the workload's self-description. A representative token from a tag-triggered release build (trimmed and annotated):

```
{
  "iss": "https://token.actions.githubusercontent.com",
  "aud": "sigstore",
  "sub": "repo:acme/api:ref:refs/heads/main",

  "repository": "acme/api",
  "repository_id": "123456789",
  "repository_owner": "acme",
  "repository_owner_id": "987654",
  "repository_visibility": "private",

  "ref": "refs/heads/main",
  "ref_type": "branch",
  "ref_protected": "true",
  "sha": "abc123def4567890abc123def4567890abc123de",

  "workflow": "release",
  "workflow_ref": "acme/api/.github/workflows/release.yml@refs/heads/main",
  "workflow_sha": "abc123def4567890abc123def4567890abc123de",

  "job_workflow_ref": "acme/api/.github/workflows/release.yml@refs/heads/main",
  "job_workflow_sha": "abc123def4567890abc123def4567890abc123de",

  "event_name": "push",
  "run_id": "7654321098",
  "run_number": "42",
  "run_attempt": "1",
  "runner_environment": "github-hosted",
  "actor": "release-bot",
  "actor_id": "555111",

  "iat": 1753900000,
  "nbf": 1753899700,
  "exp": 1753900300,
  "jti": "6f2a...c1"
}
```

The claims break into families:

- **What repo** — `repository`, `repository_id` (a stable numeric ID that survives repo renames), `repository_owner`, `repository_owner_id`, `repository_visibility`.
- **What code** — `ref`, `ref_type`, `ref_protected`, `sha`. The `ref_protected` flag is genuinely useful in policy: it tells you the ref is a protected branch.

- **What workflow** — `workflow` (the name), `workflow_ref` / `workflow_sha` (the *entrypoint* workflow file and its commit), and `job_workflow_ref` / `job_workflow_sha` (see below).
- **What run and who triggered it** — `event_name`, `run_id`, `run_number`, `run_attempt`, `actor`, `actor_id`, and `runner_environment` (`github-hosted` vs `self-hosted` — a claim you often want to pin, because a self-hosted runner is a different trust posture).

`job_workflow_ref` — **the load-bearing claim**. The distinction between `workflow_ref` and `job_workflow_ref` is subtle and it is the most important thing in this section.

- `workflow_ref` is the **top-level workflow that was triggered** — the entrypoint in the repo whose event fired.
- `job_workflow_ref` is the **workflow file that actually contains the running job's definition**. When a job calls a **reusable workflow** (uses: `acme/ci/.github/workflows/build.yml@v2`), the job runs *inside that reusable workflow*, and `job_workflow_ref` points at the reusable workflow's path, repo, and ref — while `workflow_ref` still points at the caller. When there is no reuse, the two coincide (as in the token above).

This is precisely why `job_workflow_ref` is the claim **Fulcio maps to the certificate SAN** for GitHub Actions, and the claim **SLSA Level 3 provenance** treats as the builder identity: it names the *code that actually did the work*, following into reusable workflows. If you run a hardened, centrally-owned reusable workflow — the paved-road signer of Book 4, Chapter 10 — then `job_workflow_ref` is `acme/ci-central/.github/workflows/sign.yml@refs/tags/v2`, and **every repo in the org that calls that shared workflow signs under the same, verifiable identity**. Downstream policy can require *that* identity and be certain the signing was done by the trusted shared workflow, not by arbitrary code in some tenant repo. This is the mechanism that turns "we have a golden signing pipeline" from an aspiration into an enforceable fact.

```

flowchart TB
  subgraph gh["GitHub Actions runtime"]
    JOB["Running job"]
    id-token: write granted
    EP["Token endpoint"]
    ACTIONS_ID_TOKEN_REQUEST_URL
    JOB -->|"request (audience=sigstore)"| EP
    EP -->|"signed JWT"| TOK
  end
  subgraph tok["OIDC ID token (JWT) claims"]
    TOK["iss = token.actions.githubusercontent.com"]
    sub = repo:acme/api:ref:refs/heads/main
    repository, repository_owner
    ref, sha, workflow
    job_workflow_ref = acme/ci/.../sign.yml@v2
    runner_environment
  end
  subgraph fulcio["Fulcio identity mapping"]
    MAP["issuer OID (57264.1.8) = iss"]
    SAN (URI) = job_workflow_ref
    + claims -> 57264.1.x extensions
    (repo, ref, sha, runner...)
  end
  subgraph cert["Short-lived certificate"]
    CID["SAN: https://github.com/acme/ci/.github/workflows/sign.yml@v2"]
    Issuer ext: token.actions...
    bound to ephemeral public key
  end
  TOK --> MAP --> CID

```

The Fulcio certificate that comes out embeds the issuer in the OID extension `1.3.6.1.4.1.57264.1.8` and maps a whole family of claims into further extensions in the same arc — the source repository URI and digest, the ref, the runner environment, the build trigger, and the run invocation URI. So the certificate is not just "signed by a GitHub workflow" — it is a durable record of the exact build context, all of which becomes *verifiable policy surface*. (You rarely match on every extension; `--certificate-identity` on the SAN plus `--certificate-oidc-issuer` on the issuer is the common case, with extension matching available for stricter policies.)

The same shape on other platforms

Every major CI platform and cloud implements the same primitive — the platform vouches for the workload via a short-lived, signed token — with different claim names and wiring.

GitLab CI/CD issues **ID tokens** via the `id_tokens` keyword; you declare the audience per job. GitLab is an accepted Fulcio issuer (<https://gitlab.com>, or your self-managed instance URL).

```
sign:
  image: alpine
  id_tokens:
    SIGSTORE_ID_TOKEN:
      aud: sigstore
  script:
    - cosign sign --yes "$IMAGE@$DIGEST" # cosign reads
      SIGSTORE_ID_TOKEN
```

Its claims mirror GitHub's with GitLab vocabulary: `namespace_path`, `project_path`, `project_id`, `ref`, `ref_type`, `ref_protected`, `pipeline_id`, `job_id`, `runner_id`, `environment`, `user_login`, `user_email`, a structured sub (e.g. `project_path:acme/api:ref_type:branch:ref:main`), and — the analogue of `job_workflow_ref` — `ci_config_ref_uri` and `ci_config_sha`, which Fulcio maps to the certificate SAN. So a GitLab keyless signature's identity looks like `https://gitlab.com/acme/api/.gitlab-ci.yml@refs/heads/main`.

AWS. AWS does not sign your artifacts, but it uses the identical OIDC-federation trick for the job to *assume an IAM role without a stored access key*. You register GitHub's issuer as an **IAM OIDC identity provider**, then write a role trust policy that conditions on the token's claims:

```
{
  "Effect": "Allow",
  "Principal": { "Federated": "arn:aws:iam::123456789012:oidc-provider/
token.actions.githubusercontent.com" },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
      "token.actions.githubusercontent.com:sub": "repo:acme/
api:ref:refs/heads/main"
    }
  }
}
```

The job calls `sts:AssumeRoleWithWebIdentity` with its GitHub token and gets back **short-lived AWS credentials** — no static `AWS_ACCESS_KEY_ID` in CI at all. The footgun here is exactly the one we meet again for signing: a **lazy sub condition** (`repo:acme/*:*`, or matching only `aud`) lets any workflow in any matched repo assume the role. Scope the `sub` to the exact ref/environment. (Inside EKS, the same idea appears as **IRSA — IAM Roles for Service Accounts** — where a pod's projected service-account token is the OIDC credential; Book 6, Chapter 9 develops the cloud side.)

Google Cloud Workload Identity Federation is the GCP analogue: you create a **workload identity pool** and add the CI platform as an **OIDC provider**, mapping token claims to Google attributes, then let the external identity impersonate a service account via STS token exchange — again, no downloaded service-account JSON key, which is the credential Google most wants you to stop shipping.

Buildkite, CircleCI, and others expose OIDC tokens through their own steps/contexts (Buildkite's agent OIDC token, CircleCI's OIDC context), with claims describing organization, pipeline, and job. The pattern is universal now: **the CI platform is an IdP for the jobs it runs**.

Platform	How the job gets a token	Fulcio-relevant identity claim	Issuer
GitHub Actions	<code>id-token: write</code> + token endpoint	<code>job_workflow_ref</code> -> URI SAN	<code>token.actions.gi</code>
GitLab CI/CD	<code>id_tokens:</code> keyword (per-job <code>aud</code>)	<code>ci_config_ref_uri</code> -> URI SAN	<code>gitlab.com</code> or self

Platform	How the job gets a token	Fulcio-relevant identity claim	Issuer
AWS (role assume)	AssumeRoleWithWebIdentity	sub in trust-policy condition	the federated IdP (e
Google Cloud WIF	pool + provider, STS exchange	mapped attributes/sub	the federated IdP
Buildkite / CircleCI	agent OIDC token / OIDC context	org/pipeline/job claims	platform issuer URL

SPIFFE and SPIRE — the vendor-neutral workload-identity fabric

Everything above is per-platform. The vendor-neutral standard for "give a workload a verifiable identity with no stored secret" is **SPIFFE** (Secure Production Identity Framework For Everyone), a CNCF project, with **SPIRE** (the SPIFFE Runtime Environment) as its reference implementation. SPIFFE matters here for two reasons: it generalizes the CI-OIDC pattern into a framework you can run yourself across a heterogeneous fleet, and — the architectural punchline of this chapter — **it is the same identity a service uses at runtime for mTLS in a zero-trust network** (Book 9, Chapter 10), so build-time signing identity and runtime service identity can be *one thing*.

The SPIFFE ID. A workload's identity is a URI:

```
spiffe://acme.example/ns/prod/sa/payments-api
      └─ trust domain ─┘ └─ workload path ─┘
```

The **trust domain** (`acme.example`) is an administrative root — one organization or environment — and the **path** names a workload within it, by whatever scheme you choose (here, Kubernetes namespace and service account). It is a name, not a secret. Compare the GitHub SAN `https://github.com/acme/api/.github/workflows/release.yml@...`: same idea — a URI that *names the workload* — which is exactly why the two worlds converge.

SVIDs — the credential. A SPIFFE ID is carried by an **SVID (SPIFFE Verifiable Identity Document)**, which comes in two forms, mirroring Sigstore's X.509-and-JWT duality:

- **X.509-SVID** — a short-lived X.509 certificate with the SPIFFE ID in its **URI SAN** (precisely the field Fulcio uses for the workload identity). Used for **mTLS**: two services present X.509-SVIDs and each verifies the other's SPIFFE ID against policy. This is the zero-trust building block.
- **JWT-SVID** — a JWT with the SPIFFE ID in `sub` and the intended audience in `aud`, signed by the trust domain's authority. Used where you cannot do mTLS (crossing an L7 proxy, calling an API that wants a bearer token) — structurally the same object as the CI OIDC tokens above.

SPIRE's architecture. SPIRE issues SVIDs through two components:

- A **SPIRE Server** per trust domain: the signing authority. It holds the trust domain's CA (which mints X.509-SVIDs and JWT-SVIDs), and a database of **registration entries** — the policy that maps *attestable properties of a workload* to the SPIFFE ID it should receive.
- A **SPIRE Agent** on every node: it authenticates to the server, then exposes the **Workload API** to local workloads over a Unix domain socket, handing out SVIDs and rotating them automatically.

The security rests on **two-stage attestation**, and this answers the question the CI platforms answered implicitly ("how does the issuer *know* the workload is what it claims?"):

```
sequenceDiagram
    autonumber
    participant Node as "Node / SPIRE Agent"
    participant Server as "SPIRE Server (trust domain CA)"
    participant WL as "Workload (process)"

    Note over Node,Server: Stage 1 - NODE attestation
    Node->>Server: node attestation evidence
    (AWS/GCP instance identity doc,
    k8s PSAT, x509 TPM, join token...)
    Server->>Server: verify evidence, derive node selectors
    Server-->>Node: agent X.509-SVID (this node's identity)

    Note over WL,Node: Stage 2 - WORKLOAD attestation
    WL->>Node: connect to Workload API (UDS), ask for SVID
    Node->>Node: interrogate caller via kernel:
    PID -> uid/gid, k8s ns/SA/pod labels,
    container image, docker labels
    Node->>Server: report workload selectors
    Server->>Server: match selectors to registration entry
    -> SPIFFE ID
    Server-->>Node: mint X.509-SVID / JWT-SVID for that SPIFFE ID
    Node-->>WL: deliver SVID (auto-rotated)
```

- **Node attestation** proves *which node* an agent runs on, without a pre-shared node secret. The agent presents platform evidence the server can independently verify: an **AWS/GCP/Azure instance identity document** signed by the cloud, a **Kubernetes projected service-account token (PSAT)**, a **TPM-backed x509pop** proof, or (weaker, for bootstrap) a one-time **join token**. The server verifies it and issues the agent its own SVID.
- **Workload attestation** proves *which workload* is asking, without the workload presenting any credential. When a process connects to the Workload API socket, the agent asks the **kernel** who the caller is (its PID), then runs **attestor plugins** that derive **selectors** from attestable properties: Unix `uid / gid / path`, or Kubernetes namespace, service account, and pod labels, or the container image digest. The server matches those selectors to a registration entry and returns the matching SPIFFE ID.

The crucial property, identical to keyless CI signing: **the workload holds no secret and presents no key to get its identity**. Its identity is *derived from what it verifiably is* — where it runs, as what user, in which pod — and re-issued continuously as a short-lived SVID. A stolen SVID is

useless in minutes; there is no long-lived credential to steal in the first place. SPIFFE also **federates** across trust domains: each domain publishes its CA public keys at a **bundle endpoint**, so a workload in `acme.example` can verify one in `partner.example` — the same rotate-and-distribute-keys problem Sigstore solves with TUF, solved here per trust domain.

The trust chain, assembled

Whether the platform is GitHub, GitLab, a cloud, or SPIRE, the chain a keyless workload signature rides is the same:

```
workload → short-lived OIDC token / SVID (signed by the platform
            IdP / trust-domain CA,
            asserting "this is workload W")           minted per-run, expires in minutes)
            → Fulcio verifies the token against the
            issuer's published keys,
            maps the identity claim into the certificate SAN
            → short-lived certificate binds workload identity ► ephemeral public key
            → signature over the artifact digest, logged in Rekor
```

At no point is there a durable secret. The signer, provably, is **the workload** — specific, named, and (via Rekor) recorded. That specificity is the entire point of the next section: it is only worth producing if verification actually *checks* it.

Verifying workload-identity signatures — policy is everything

Here is the mental discipline this whole book keeps returning to: **you do not verify a key — you verify an identity against a policy**. With a pinned key, "valid signature by key X" was the whole question. With keyless workload signing there is no key to pin; the certificate's key is ephemeral and different every time. What is stable, and what you must check, is the **identity and issuer** in the certificate. A "valid Sigstore signature" that you do not constrain to an expected identity is close to worthless — anyone in the world can produce a valid Sigstore signature over your image using *their own* GitHub identity. The signature is real; it just is not *yours*.

Cosign v2 enforces this by making the identity flags **mandatory** for keyless verification — there is no "just check it's signed" mode:

```
# Verify: this image MUST have been signed by the exact release
workflow,
# via GitHub's OIDC issuer. Both flags are required; order of checks is
fail-closed.
cosign verify \
  --certificate-identity "https://github.com/acme/api/.github/
workflows/release.yml@refs/heads/main" \
  --certificate-oidc-issuer "https://
token.actions.githubusercontent.com" \
  ghcr.io/acme/api@sha256:9b2a...c1
```

`--certificate-identity` matches the certificate **SAN** (the `job_workflow_ref`-derived URI); `--certificate-oidc-issuer` matches the **issuer OID extension**. Both together, because — as Chapter 3 stressed — `alice@example.com` via Google and `alice@example.com` via a rogue IdP are different signers, and `release.yml@main` in *your* repo versus in a fork are different signers only distinguished by the full URI.

When you legitimately need to accept a *family* of identities — say every repo in your org that calls a shared signing workflow — use the regexp forms, but **anchor them and make them specific**:

```
# Accept the SHARED signing workflow called from any repo in the acme
org,
# only from a version tag, via GitHub's issuer.
cosign verify \
  --certificate-identity-regexp '^https://github\.com/acme/ci-central/
\.github/workflows/sign\.yml@refs/tags/v[0-9]+' \
  --certificate-oidc-issuer 'https://
token.actions.githubusercontent.com' \
  ghcr.io/acme/api@sha256:9b2a...c1
```

The verify-by-identity flow, end to end:

```

flowchart TB
    START["cosign verify / admission controller"] --> FETCH["Fetch signature + cert (+ Rekor bundle) from registry"]
    FETCH --> SIG["Signature valid over artifact digest, cert chains to Fulcio root, SCT + Rekor proof ok?"]
    SIG -- no --> FAIL["REJECT (fail closed)"]
    SIG -- yes --> EXT["Extract identity (SAN) + issuer (OID) from cert"]
    EXT --> POLID["SAN matches --certificate-identity[-regexp]?"]
    POLID -- no --> FAIL
    POLID -- yes --> POLISS["issuer matches --certificate-oidc-issuer?"]
    POLISS -- no --> FAIL
    POLISS -- yes --> PASS["ACCEPT"]

```

The cryptographic checks (signature valid, chains to Fulcio, logged in Rekor, `integratedTime` inside the cert window — Chapter 3's verify-after-expiry) are necessary but **not sufficient**. They prove "a real Sigstore signature by *some* identity." The **identity/issuer match** is what turns that into "a signature by *the identity I trust to build this*." Skip it, or write it loosely, and you have built an elaborate no-op.

The footgun: over-broad identity matching

This is the same class of bug as the AWS `sub`-condition footgun above, and it is the single most common way real deployments get keyless verification wrong. Every one of these "works" — it accepts signatures and passes CI — while under-verifying:

- **Matching the org, not the workflow.** `--certificate-identity-regexp '^https://github\.com/acme/'` accepts a signature from *any workflow in any acme repo* — including a `test.yml` in a low-privilege repo, or a workflow a compromised insider added. If your security story is "only the hardened release workflow may sign production images," this regexp does not encode it.
- **Unanchored / dot-wildcards.** `--certificate-identity-regexp 'acme/api'` (no `^`, unescaped `.`) matches `evil-acme/api-backdoor` and more. Regex identity matching **must** be anchored (`^`) and escape literal dots and slashes. An unanchored substring match is an open door.

- **Ignoring the ref.** Matching `.../release.yml@` with any ref accepts a signature produced when the workflow ran on an attacker's branch or a PR fork. Pin the ref (`@refs/heads/main` , or `@refs/tags/v...`) — a workflow file's *contents* are whatever they were on that ref, so "release.yml on some random branch" is code you never reviewed.
- **Not pinning the issuer** (or accepting multiple issuers loosely). The issuer is half the identity.
- **Forgetting `runner_environment`** where it matters. If production images must be built on GitHub-hosted runners, the `runner_environment=github-hosted` extension is available to match; a self-hosted runner is a different (often weaker) trust boundary.

The correct instinct is **least-authority identity policy**: name the *exact* workflow path, the *exact* issuer, and the *exact* ref (or a tightly anchored pattern over versions), and add extension constraints (`runner_environment` , `ref_protected`) where your threat model needs them. In a Kubernetes admission policy (Sigstore `policy-controller` or Kyverno; Book 6, Chapters 5–6, and Chapter 10) the same discipline applies to the `subject/issuer` fields — a `subjectRegExp` of `^https://github.com/acme/.+` is a **finding**, not a policy.

Security properties and caveats — the honest ledger

Keyless workload signing is a large, real improvement, and it is important to state both sides plainly.

The wins.

- **No key management.** There is no signing key to generate, store, rotate, escrow, HSM-protect, or distribute across the fleet — the entire apparatus of Chapter 2 and much of enterprise PKI (Chapter 9) simply does not exist for the signer. The cost that used to grow with the number of signers is gone.
- **No stored secrets in CI.** The credential is minted per-run and dies with the run. There is nothing in the CI secret store to leak through pipeline poisoning, log exposure, or a malicious dependency in the build.
- **Specific, verifiable identities.** The signer is not "a key" but *this workflow, this repo, this ref, this runner* — human-meaningful and enforceable in policy.

- **Transparency.** Every signing event lands in Rekor (Chapter 5), so misuse of an identity is *detectable*, unlike a stolen key signing in silence.
- **It fits ephemeral CI natively.** Runners are cattle; they come up, build, sign, and vanish. Ambient identity with no provisioning step is the only signing model that matches that lifecycle.

The caveats — a different trust base, not a smaller one.

- **You now trust the OIDC IdP.** The security of every signature reduces to the integrity of the platform's token issuance. If GitHub, GitLab, or your corporate IdP mis-issues a token — a bug that lets one repo obtain another's `job_workflow_ref`, a signing-key compromise at the issuer — the identity binding is only as good as that issuance. You have traded "protect a key" for "trust an IdP's token minting," which is usually a *better* trade (IdPs are hardened, audited, MFA-gated) but it is a real dependency, not its absence.
- **You trust Fulcio and Rekor.** On the public-good instance, verification and (on the sign path) signing depend on services run by the OpenSSF/Linux Foundation. Their **availability** is on your critical path if you verify online, and their **integrity** underwrites the whole chain. This is a strong argument, at fleet scale, for **self-hosted Sigstore** (Chapter 9) so the dependency and the signing-event data are yours — and for offline verification via bundles so admission does not call a public service on every deploy.
- **A compromised CI identity still signs valid artifacts.** This is the persistent lesson, restated once more: signing proves *who/what* signed, **not that the thing is honest**. If an attacker subverts the release workflow — poisons a step, compromises a used action, exploits a `pull_request_target` misconfiguration (Book 4, Chapters 5 and 7) — the malware they produce is signed by the *real* workflow identity and verifies perfectly. Keyless signing raises the attacker's bar from "steal a key" to "compromise the build," which is genuine progress, but the axis is unchanged: **signed $\neq$ safe**. Build integrity is a separate axis carried by provenance and policy (SLSA, Book 4, Chapter 3; in-toto, Chapter 6), transported *by* signatures, not replaced by them.
- **The workload identity itself must be bootstrapped and attested.** "The platform vouches for the workload" only helps if the platform's vouching is itself sound. That is exactly what SPIRE's

node and workload attestation formalize — and it is why the *quality of attestation* matters: a weak node attestor (a shared join token, an unbounded selector) undermines every SVID derived from it, just as a lazy `job_workflow_ref` mapping or an over-broad verify policy undermines every signature. The identity is only as strong as the attestation that grounds it. There is no free lunch; there is a *better-shaped* lunch, and its shape is "attest what the workload verifiably is, issue a short-lived credential, and check it against a tight policy."

Distributed-systems lens

The reason this chapter sits at the center of the book is that **workload identity is the unifying primitive of the modern secure fleet** — the same identity, in the same shape, does four different jobs that used to require four different long-lived secrets.

Consider a single service, `payments-api`, across its lifecycle:

- **At build time (this chapter):** the build's workload identity — `.../ci-central/.github/workflows/sign.yml@v2` or a SPIFFE ID — signs the image, its SBOM, and its SLSA provenance. No signing key.
- **Authenticating to secret managers and clouds (Book 4, Chapter 6):** the *same* OIDC token federates to AWS/GCP/Vault to fetch runtime config, via `AssumeRoleWithWebIdentity` or Workload Identity Federation. No stored cloud key.
- **At runtime, service-to-service (Book 9, Chapter 10 — zero trust and mTLS):** the service presents an **X.509-SVID** carrying `spiffe://acme.example/ns/prod/sa/payments-api` and does mutual TLS; peers authorize by SPIFFE ID. No shared service password.
- **At admission (Book 6, Chapters 5-6; Chapter 10):** the deploy gate accepts the image *only if* its signature's certificate identity matches the trusted build identity. The verification policy references the *same identity strings* the build produced.

There are **no long-lived secrets anywhere in that lifecycle**. Every credential is short-lived, minted per-use from an attested identity, and named by a URI a human can read and a policy can match. This is the architectural convergence worth internalizing: **the identity that signs an artifact at build time and the identity a service uses at runtime are the same kind of thing** — a SPIFFE-style or OIDC-style workload

identity — and can, with SPIFFE/SPIRE as the org-wide fabric, be literally the same identity system. The certificate SAN Fulcio fills from `job_workflow_ref` and the URI SAN in an X.509-SVID are the same field doing the same job.

Two consequences for platform design:

Signing becomes automatic and free at the platform layer. Because the signer is ambient identity and the client is one line, the **paved-road build platform** (Book 4, Chapter 10) signs *every* artifact — image, SBOM, provenance — under the build's own identity, with no key handed to any team and no code written by any tenant. Route all signing through a **shared reusable workflow** so that `job_workflow_ref` is *one* stable, hardened identity across thousands of repos, and downstream policy can require exactly that identity. A thousand services, one signing identity to trust, zero keys to distribute — the classic model could never do this, because it priced in a key per signer.

Verification is identity-and-policy at the edges of every trust boundary. The admission controller, the deploy gate, the peer in an mTLS handshake, the cloud IAM endpoint — each is asking the same question in its own dialect: *"is this the identity my policy allows?"* Get that policy tight (the footgun section) and the fleet has a single, coherent, keyless identity story from `git push` to production traffic. Get it loose and you have spent a lot of cryptography to check almost nothing.

Key takeaways

- **"Keyless" is a misnomer.** There *is* an ephemeral keypair per signing — generated in memory, used once, then discarded. What is gone is the **long-lived, user-managed** key. Identity, not a key, is the durable anchor, which structurally removes Chapter 2's key-theft failure class: **nothing durable to steal**.
- **OIDC is the foundation.** An IdP issues a short-lived **ID token (a signed JWT)** with claims — `iss`, `sub`, `aud`, plus provider-specific ones. **Fulcio validates the token against the issuer's published keys and maps its identity claim into the certificate SAN**, binding identity to the ephemeral key.
- **Human vs workload identity.** Interactive signing yields an **email** in the cert via a browser flow; CI signing uses an **ambient** platform-

issued token yielding a **workload URI** in the cert. Automated pipelines need the latter — there is no human to click a browser.

- **The CI platform is an OIDC provider for its jobs.** GitHub Actions: grant `id-token: write`, fetch from the token endpoint (`ACTIONS_ID_TOKEN_REQUEST_URL`), get a JWT whose claims describe the job (`repository`, `ref`, `sha`, `workflow`, `runner_environment`, ...). `job_workflow_ref` — which follows into reusable workflows — is the claim Fulcio maps to the cert SAN and that SLSA L3 uses as the builder identity.
- **Same shape elsewhere.** GitLab `id_tokens (ci_config_ref_uri)`; AWS `AssumeRoleWithWebIdentity` and IRSA; Google Cloud **Workload Identity Federation**; Buildkite/CircleCI OIDC. All: the platform vouches for the workload via a short-lived signed token, no stored secret.
- **SPIFFE/SPIRE is the vendor-neutral fabric.** Identity is the **SPIFFE ID** (`spiffe://trust-domain/workload`), carried by an **X.509-SVID** (URI SAN, for mTLS) or **JWT-SVID** (bearer). SPIRE issues them via **node attestation** (prove the node with a cloud instance identity doc, k8s PSAT, TPM) then **workload attestation** (kernel-derived selectors → registration entry), so the workload holds no secret to get its identity.
- **Verify the identity, not a key.** Cosign v2 **requires** `--certificate-identity` (or `-regex`) and `--certificate-oidc-issuer`. The crux is getting this policy **tight**: name the exact workflow, ref, and issuer. **Over-broad matching** (org-wide instead of the specific workflow, unanchored regexes, ignoring the ref, missing issuer) under-verifies while appearing to work — the most common real-world failure, the same class as a lazy AWS `sub` condition.
- **Honest ledger.** Wins: no key management, no stored secrets, transparency, specific verifiable identities, native fit for ephemeral CI. Caveats: you now trust the **IdP + Fulcio + Rekor** (a different trust base, and a public-instance dependency — argues for self-hosting, Chapter 9); a **compromised CI identity still signs valid artifacts** (signed ≠ safe); and the **workload identity itself must be attested** (node/workload attestation) — the identity is only as strong as its attestation.
- **Distributed-systems view.** Workload identity is the **one primitive** that signs build artifacts (here), federates to clouds and secret

managers (Book 4, Chapter 6), and does runtime mTLS in a zero-trust mesh (Book 9, Chapter 10) — **no long-lived secrets anywhere**. The paved-road platform signs every artifact under a shared reusable-workflow identity automatically; admission policies reference those same identity strings. **Build-signing identity == runtime service identity** is the key convergence, with SPIFFE/SPIRE as the org-wide fabric.

Further reading

- **OpenID Connect Core 1.0** — the ID token, standard claims (`iss` , `sub` , `aud`), and discovery. https://openid.net/specs/openid-connect-core-1_0.html (full treatment in Book 9, Chapter 6).
- **GitHub Actions — OIDC** — `id-token: write` , the token endpoint, the full claim set, and hardening cloud trust with `sub` conditions. <https://docs.github.com/en/actions/security-for-github-actions/security-hardening-your-deployments/about-security-hardening-with-openid-connect>.
- **Fulcio certificate profile and OID extensions** — the `1.3.6.1.4.1.57264.1` arc, the issuer/SAN/ `job_workflow_ref` mappings, and per-issuer identity rules. <https://github.com/sigstore/fulcio> (`docs/oid-info.md` and the certificate spec).
- **Cosign verification** — `--certificate-identity` , `--certificate-identity-regexp` , `--certificate-oidc-issuer` , and extension matching. <https://docs.sigstore.dev/> and <https://github.com/sigstore/cosign>.
- **GitLab CI/CD — ID tokens and OIDC** — the `id_tokens` keyword, claims (`ci_config_ref_uri` , ...), and Sigstore integration. https://docs.gitlab.com/ee/ci/secrets/id_token_authentication.html.
- **AWS — IAM OIDC identity providers and AssumeRoleWithWebIdentity ; Google Cloud — Workload Identity Federation**. The cloud-federation side is developed in Book 6, Chapter 9.
- **SPIFFE and SPIRE** — the **SPIFFE ID** and **SVID** specs (X.509-SVID, JWT-SVID) and the SPIRE node/workload attestation model. <https://spiffe.io/docs/> and <https://github.com/spiffe/spire> (applied to runtime mTLS/zero trust in Book 9, Chapter 10).

- **SLSA v1.0 provenance** — how the builder identity (`job_workflow_ref` on GitHub) grounds Build L3. <https://slsa.dev/spec/v1.0/> (Book 4, Chapter 3).
- Cross-references: Book 5, Chapter 2 (classic code signing and its key-theft failures), Chapter 3 (Sigstore architecture — Cosign/Fulcio/Rekor, verify-after-expiry), Chapter 5 (transparency logs), Chapter 6 (in-toto attestations), Chapter 9 (private Sigstore and key management), Chapter 10 (attestation-based deployment gates); Book 4, Chapters 3, 5, 6, 7, 10 (SLSA provenance, hardening Actions, CI secrets/OIDC federation, pipeline poisoning, secure build platform); Book 6, Chapters 5, 6, 9 (image signing in Kubernetes, admission policy engines, cloud/serverless chain); Book 9, Chapter 6 (OAuth/OIDC), Chapter 10 (zero trust and service-to-service mTLS with SPIFFE).

Chapter 5 — Transparency Logs: Merkle Trees, Rekor, and CT Lessons

What this chapter covers. A signature answers *who signed*. It does not answer *who else has been signing, and what*. Chapter 2 showed the consequence: a stolen key or a compromised CA can sign malware, and with classic signing **nobody outside the signer can see it happen**. The signature is valid; the abuse is invisible. Chapter 3 introduced Sigstore's answer — short-lived certificates plus a **transparency log** — and deferred the log's internals to here. This chapter is that deferred deep dive. Transparency logs are the mechanism that converts "trust me" into "verify publicly": they make issuance of certificates, signatures, and attestations a matter of **public record** that is **tamper-evident** and **append-only**, so misuse becomes *detectable*. The slogan to carry through the chapter: *you can still be attacked, but you cannot be attacked silently*. We build the cryptographic spine first — RFC 6962 Merkle trees, inclusion proofs, consistency proofs, and signed tree heads, worked through concrete small examples — because everything else is an application of that spine. Then we study the proven precedent (**Certificate Transparency** and the Web PKI), the Sigstore application (**Rekor**), the two other major deployments (**Go's checksum database** and **binary transparency**), and finally the honest weaknesses:

transparency has no value without **monitoring**, and a log means nothing if different people can be shown different versions of it — the **split-view** problem.

Learning goals — after this chapter you should be able to:

- State precisely what a transparency log guarantees — **append-only**, **tamper-evident**, **publicly auditable**, **globally consistent** — and what each guarantee does and does *not* buy you.
- Construct an **RFC 6962 Merkle tree** by hand, including the `0x00 / 0x01` **domain separation** and why it exists, and compute an **inclusion proof** and a **consistency proof** on a concrete tree.
- Explain why the append-only property is **cryptographic**, not a matter of policy — how a consistency proof makes history-rewriting *detectable by anyone*.
- Describe **Certificate Transparency**'s real mechanics — logs, **SCTs**, **monitors**, **auditors** — the misissuance disasters that motivated it, and its limits.
- Describe what **Rekor** logs, how the **Signed Entry Timestamp** and inclusion proof enable **verify-after-expiry**, and how to query Rekor to *detect misuse* of your signing identities.
- Explain the **Go checksum database** and **binary/key transparency**, and why the Go design is unusually strong.
- Reason about the operational realities — the **monitoring requirement**, the **split-view/gossip** problem and **witnesses**, availability, and the **privacy** cost of public logging.

The problem: silent signing

Recall the failure class from Chapter 2. Classic code signing roots trust in a **long-lived private key**. When that key is stolen (Stuxnet's Realtek and JMicron driver certificates; the 2022 NVIDIA leak) or a certificate authority is subverted, the attacker produces signatures that are, by every cryptographic test a verifier can run, **valid**. Verification checks a chain to a trusted root and a signature over a digest; a stolen-but-not-yet-revoked key passes both. The defender's only recourse is **revocation**, and revocation is both slow and reactive: it can only act on abuse someone has already discovered.

The deeper problem is *discovery*. With classic signing, issuance is a **private act**: a CA that misissues a `*.google.com` certificate, or an attacker who signs a backdoored driver with a stolen key, leaves **no public trace**. The victim learns of it only if the malicious artifact surfaces and someone inspects it. This is the **silent-signing problem** — the signing infrastructure can be abused for weeks or months with no external signal.

Transparency logs attack this problem directly. They do **not** prevent a key from being stolen or a CA from being compromised — that is a different axis, covered by hardware protection and short-lived credentials (Chapters 2, 9). What they change is **observability**. If every certificate a CA issues, and every signing event a system produces, must be **published to a public, append-only, tamper-evident log** before it is honored, then abuse leaves an indelible, globally visible record. A domain owner watching the CT logs *sees* the fraudulent certificate; a platform team watching Rekor *sees* their CI identity sign an artifact they never built. The attack is no longer silent — the difference between a breach discovered in hours and one discovered, if ever, in months.

The four guarantees

A transparency log makes four distinct promises. Keep them separate, because different mechanisms enforce each and different attacks defeat each.

- **Append-only.** Entries are only ever added at the end. Nothing already logged is removed or altered. Enforced *cryptographically* by consistency proofs (below), not by the operator's good behavior.
- **Tamper-evident.** Any modification to any past entry changes the tree root, so it is *detectable*. Note the word: *tamper-evident*, not *tamper-proof*. The log does not stop tampering; it makes tampering impossible to hide from anyone checking.
- **Publicly auditable.** Anyone — not a privileged auditor — can fetch entries and proofs and verify inclusion and consistency for themselves, using only public data and a trusted root.
- **Globally consistent.** Everyone sees the *same* log. This is the hardest guarantee and the one most often glossed over: a log that shows one entry set to you and a different one to me has defeated the entire point. This is the **split-view problem**, and it is not solved by

the Merkle math alone — it needs gossip and witnesses (final section).

The first three are properties of a single log's internal structure and are enforced by the cryptography we build next. The fourth is a *distributed-systems* property about many observers agreeing, and it is where transparency logs are most subtle.

Merkle tree mechanics

This is the technical spine of the chapter. Chapter 1 sketched Merkle trees; here we pin down the exact RFC 6962 construction, because CT, Rekor, and the Go checksum database all use it (or a close variant), and because getting the details right is what separates "I've heard of inclusion proofs" from "I can verify one."

RFC 6962 tree construction and domain separation

RFC 6962 (Certificate Transparency, June 2013) defines the **Merkle Tree Hash (MTH)** of a list of n data entries $D[n] = d(0), d(1), \dots, d(n-1)$ recursively, using a hash function that in practice is SHA-256. Two rules do all the work:

- **Leaf hashing (a one-entry list).** For a single entry $d(i)$, the hash is $MTH = \text{HASH}(0x00 \parallel d(i))$. The single byte $0x00$ is prepended before hashing.
- **Internal nodes ($n > 1$ entries).** Let k be the **largest power of two strictly less than n** . Split the list into a left part $D[0:k]$ (the first k entries) and a right part $D[k:n]$. Then $MTH(D[n]) = \text{HASH}(0x01 \parallel MTH(D[0:k]) \parallel MTH(D[k:n]))$.

Two consequences of that k rule are worth internalizing. First, the **left subtree is always a complete (perfect) binary tree** of k leaves; the right subtree holds the remainder. The tree is not padded to a power of two — a log of 6 entries is a real 6-leaf tree, left-heavy. Second, because leaves are hashed with $0x00$ and internal nodes with $0x01$, a leaf hash and an internal node hash live in **disjoint domains**.

That domain separation is not decoration; it is a **second-preimage defense**. Without the prefixes, an attacker could take an internal node — which is itself just $\text{HASH}(\text{left} \parallel \text{right})$, i.e. the hash of two concatenated child hashes — and present it as if it were a *leaf* whose data

happened to be `left || right`. The verifier, unable to tell a leaf from a node, could be tricked into accepting a proof for an entry that was never actually logged, or into a tree that has two valid interpretations. Prefixing every leaf with `0x00` and every node with `0x01` makes the two hash inputs structurally impossible to confuse: no leaf hash can ever equal a node hash, so no node can masquerade as a leaf. This is the single most commonly omitted detail in hand-rolled Merkle implementations, and omitting it is a real vulnerability.

The **root hash** — the MTH of the whole list — is a single 32-byte value that commits to every entry and to their exact order. Change any entry, insert one, reorder two, and the root changes.

Inclusion proof: proof of presence

An **inclusion proof** (RFC 6962 calls it an *audit path*) demonstrates that a specific entry is a leaf under a given root, using only the **sibling hashes** along the path from the leaf to the root — about $\log_2(n)$ hashes for `n` entries. It reveals nothing about the other entries.

Work a concrete example: a perfect tree of **8 entries**, `d0 ... d7`, and prove that `d3` is present. Write `a_i = HASH(0x00 || d_i)` for the leaf hashes, and build up:

- Level 0 (leaves): `a0, a1, a2, a3, a4, a5, a6, a7`
- Level 1: `b01 = HASH(0x01 || a0 || a1)`, `b23 = HASH(0x01 || a2 || a3)`, `b45, b67`
- Level 2: `c0123 = HASH(0x01 || b01 || b23)`, `c4567 = HASH(0x01 || b45 || b67)`
- Root: `r = HASH(0x01 || c0123 || c4567)`

```

flowchart TD
  R["root = H(c0123 || c4567)"]
  R --> C0123["c0123 = H(b01 || b23)"]
  R --> C4567["c4567 = H(b45 || b67) ◀ proof #3"]
  C0123 --> B01["b01 = H(a0 || a1) ◀ proof #2"]
  C0123 --> B23["b23 = H(a2 || a3)"]
  C4567 --> B45["b45 = H(a4 || a5)"]
  C4567 --> B67["b67 = H(a6 || a7)"]
  B23 --> A2["a2 = H(0x00 || d2) ◀ proof #1"]
  B23 --> A3["a3 = H(0x00 || d3) TARGET"]
  style A3 fill:#2ea043,color:#fff
  style A2 fill:#1f6feb,color:#fff
  style B01 fill:#1f6feb,color:#fff
  style C4567 fill:#1f6feb,color:#fff

```

The proof for `d3` is exactly **three** hashes — the siblings encountered climbing from `a3` to the root:

1. `a2` — the sibling of `a3` (on the **left**, since `d3` is at an odd index).
2. `b01` — the sibling of `b23` (on the left).
3. `c4567` — the sibling of `c0123` (on the **right**).

The verifier, holding the entry `d3` and a **trusted root** `r`, recomputes:

```

a3    = HASH(0x00 || d3)           # verifier hashes the entry itself
b23   = HASH(0x01 || a2 || a3)     # a2 from proof, on the left
c0123 = HASH(0x01 || b01 || b23)   # b01 from proof, on the left
r'    = HASH(0x01 || c0123 || c4567) # c4567 from proof, on the right

```

and checks `r' == r`. Match means `d3` is in the tree committed to by `r`, in that position, and the log could not have forged the proof without breaking the hash's collision resistance. Note the **left/right orientation matters**: the verifier must know at each step whether the supplied sibling goes on the left or right of the running value, which is determined by the leaf's index and the tree size. Get the order wrong and you compute a different (wrong) root.

Three hashes prove membership among eight. The cost is logarithmic: a log of **one billion** entries needs only ~30 hashes for an inclusion proof. This is precisely what makes transparency *practical at scale* — a verifier confirms "my signature **is** recorded" by fetching ~30 hashes and doing ~30 SHA-256 computations, never the whole log.

Consistency proof: proof of append-only

Inclusion proves *presence*. **Consistency** proves *append-only-ness*: that a later version of the log is a strict superset of an earlier one, with the earlier entries **unchanged** and only new entries appended. This is the guarantee that makes the log tamper-evident against **the log operator itself**.

Given the root R_m at an earlier size m and the root R_n at a later size $n > m$, a **consistency proof** is a set of $\sim \log(n)$ node hashes that lets a verifier confirm two things at once from the *same* nodes: (a) those nodes recompute the old root R_m , and (b) those same nodes, plus the new material, recompute the new root R_n . Because the identical subtree hashes feed both computations, the old tree must sit inside the new one unchanged.

Concrete example: the log grows from $m = 6$ to $n = 8$. At size 6 the tree covers $d0 \dots d5$. By the k rule, the largest power of two below 6 is 4, so the size-6 root is built from the complete left subtree $c0123$ (over $d0\dots d3$) and a right subtree $b45$ (over $d4, d5$):

```
R6 = HASH(0x01 || c0123 || b45)          # b45 = HASH(0x01 || a4 || a5)
```

At size 8 the tree is the perfect tree from before, with $R8 = r = \text{HASH}(0x01 || c0123 || c4567)$ and $c4567 = \text{HASH}(0x01 || b45 || b67)$. The RFC 6962 consistency proof from 6 to 8 is the three nodes **[b45, b67, c0123]**.

```

flowchart TD
    subgraph new["Tree at size 8 (new)"]
        R8["R8 = H(c0123 || c4567)"]
        R8 --> C0123n["c0123 (unchanged prefix)"]
        R8 --> C4567n["c4567 = H(b45 || b67)"]
        C4567n --> B45n["b45 (was the old right edge)"]
        C4567n --> B67n["b67 (newly appended: d6,d7)"]
    end
    subgraph old["Tree at size 6 (old)"]
        R6["R6 = H(c0123 || b45)"]
        R6 --> C0123o["c0123"]
        R6 --> B45o["b45"]
    end
    C0123n -. same hash .-> C0123o
    B45n -. same hash .-> B45o
    style C0123n fill:#1f6feb,color:#fff
    style B45n fill:#1f6feb,color:#fff
    style B67n fill:#2ea043,color:#fff
    style C0123o fill:#1f6feb,color:#fff
    style B45o fill:#1f6feb,color:#fff

```

The verifier does two reconstructions from the proof nodes:

- **Old root:** `HASH(0x01 || c0123 || b45)` and checks it equals the trusted `R6`. This proves the log has not altered anything in the first six entries.
- **New root:** `HASH(0x01 || c0123 || HASH(0x01 || b45 || b67))` — i.e. `HASH(0x01 || c0123 || c4567)` — and checks it equals `R8`.

The load-bearing fact is that the **same** `c0123` and the **same** `b45` appear in both reconstructions. The verifier does not take the log's word that "the old tree is a prefix"; it proves it, because any change to entries `d0...d5` would have changed `c0123` or `b45`, which would break *either* the old-root check *or* the new-root check. An operator who tried to rewrite entry `d2` (say, to swap a logged certificate for a different one) cannot produce a size-8 tree that is consistent with the size-6 root it already published — every client that later requests a 6→8 consistency proof would get a failure. **History-rewriting is not forbidden by policy; it is made mathematically detectable by anyone.** That is what "append-only" means in a transparency log.

Signed tree head / checkpoint

The Merkle root is only meaningful if clients agree on *which* root is authoritative at a given time. That is the job of the **Signed Tree Head**

(STH) — RFC 6962's term — or **checkpoint**, the name used by the modern note-based format (Rekor and the Go checksum database). Periodically (in CT, at most every **Maximum Merge Delay** — typically 24 hours; in Rekor, continuously) the log computes its current root and **signs a small structure** committing to:

- the **tree size** (number of entries, `n`),
- the **root hash** at that size, and
- a **timestamp**.

In RFC 6962 the signed structure is `{ version, signature_type, timestamp, tree_size, sha256_root_hash }`, signed with the log's private key. The modern **checkpoint** ("note") serialization is human-readable and looks like:

```
rekor.sigstore.dev - 1193050959916656506
30707170
oS4TL5UWN0zBLTa1RgmXj2Rn+Dd0JUS5Fk9J0KtRTgU=

- rekor.sigstore.dev wNI9ajBFAiEA...base64-signature...
```

The first line is the log's **origin** (identity), the second the tree size, the third the base64 root hash; the `-` line carries the signature. This checkpoint is the log's **signed commitment** — "at this moment, I attest that my tree has exactly `n` entries and this root." A verifier trusts a checkpoint if the signature verifies against the log's public key (distributed out of band, in Sigstore via the TUF **trust root**; see Book 5, Chapter 7). Every inclusion and consistency proof is verified *relative to a checkpoint*: the proof shows an entry belongs to, or one tree extends to, the root that this signed checkpoint pins down.

The checkpoint is also the unit of the **global-consistency** problem: two clients who hold two *different* checkpoints for the *same* tree size, with *different* root hashes, have caught the log in a fork. We return to that.

Inclusion vs consistency at a glance

Property	Inclusion proof (audit path)	Consistency proof
Question answered	"Is this entry in the tree with root R?"	"Is the size- <code>m</code> tree an unmodified prefix of the size- <code>n</code> tree?"

Property	Inclusion proof (audit path)	Consistency proof
Inputs	The entry, its index, tree size, a trusted root R	Old size <code>m</code> + root <code>R_m</code> , new size <code>n</code> + root <code>R_n</code>
Proof contents	$\sim \log_2(n)$ sibling hashes leaf→root	$\sim \log(n)$ subtree node hashes spanning the m/n boundary
Verifier checks	Recompute root from entry + siblings == R	Recompute both <code>R_m</code> and <code>R_n</code> from shared nodes
Guarantees	Presence (tamper-evidence of one entry)	Append-only (tamper-evidence of all history)
Who runs it	Anyone proving their own entry is logged (monitors, signers)	Auditors , and any client tracking the log over time

Both are needed. Inclusion without consistency lets an operator rewrite the past between the time you saw an inclusion proof and now. Consistency without inclusion tells you the log is append-only but not that *your* entry is in it. Real clients — and real security — use both.

Certificate Transparency: the proven precedent

Transparency logs are not a Sigstore invention. They were designed, deployed, and battle-tested for the Web PKI a decade earlier, and Sigstore borrowed the model wholesale. Understanding CT is the fastest way to understand what a transparency log *is for*.

Why CT exists: the misissuance disasters

The Web PKI has a structural weakness: **any** trusted CA can issue a certificate for **any** domain. Your browser trusts on the order of a hundred root CAs (and the intermediates they delegate to); a certificate for `mail.yourcompany.com` from any one of them is accepted. So the security of *your* domain depends on the security of *every* CA in the store — a weakest-link system.

That weakness became catastrophe in **2011**. The Dutch CA **DigiNotar** was compromised; attackers issued fraudulent certificates for high-value domains, most infamously a wildcard for `*.google.com`, and used it to

man-in-the-middle an estimated ~300,000 Iranian Gmail users. DigiNotar had *no idea* which certificates the attacker had issued — the issuance was silent, and the CA could not even enumerate the damage. DigiNotar was removed from browser trust stores and went bankrupt within weeks. The same year, a reseller for **Comodo** was breached and issued fraudulent certificates for Google, Yahoo, Skype, and others. The lesson was stark: the Web PKI had **no mechanism to detect misissuance**, whether from a breached CA or a rogue insider.

Certificate Transparency (Google — Ben Laurie, Adam Langley, Emilia Kasper — standardized as **RFC 6962**) is the response. Its thesis: a CA cannot be prevented from misissuing, but it can be *required to publish* every certificate it issues to public append-only logs, so that misissuance is **detectable**. Chrome began requiring CT for Extended Validation certificates and then, from **April 30, 2018**, for **all** newly issued publicly trusted certificates. A certificate that is not logged is not trusted.

The mechanics: logs, SCTs, monitors, auditors

CT has four roles, and each maps to a piece of machinery you should be able to name.

- **Logs.** Independently operated append-only Merkle logs (RFC 6962 construction) run by Google, Cloudflare, Let's Encrypt (Sectigo, DigiCert, and others). CAs submit every certificate (in practice a **precertificate**, a poison-extended variant submitted before final issuance) to several logs.
- **SCTs (Signed Certificate Timestamps).** When a CA submits a (pre)certificate, the log returns an **SCT**: a signed promise, `{ timestamp, log_id, signature }`, that the log **will** incorporate this certificate into its tree within the **Maximum Merge Delay** (MMD, usually 24h). Crucially, the SCT is a **promise, not a proof of inclusion** — at SCT-issue time the entry may not yet be merged into the tree. Browsers require a certificate to carry **SCTs from multiple independent logs**, delivered by one of three channels: embedded as an **X.509 extension** in the certificate (by far the most common), via a **TLS extension** during the handshake, or via **OCSP stapling**.
- **Monitors.** Parties who **download the logs and watch for entries of interest** — most importantly, certificates for **their own domains**. A monitor for `yourcompany.com` fetches every new entry and flags any certificate naming your domains that your team did not request. This

is the role that actually *catches misissuance*. Services like Cloudflare's and Facebook's monitors, plus commercial and DIY tooling querying `crt.sh`, fill it.

- **Auditors.** Parties who verify the logs **behave correctly**: that they are genuinely append-only (consistency proofs check out), and that certificates promised by SCTs actually **appear** in the tree within the MMD (inclusion proofs check out). Lightweight auditing is built into browsers; standalone auditors and the log operators' peers do the heavier checking.

```
flowchart LR
    CA["CA issues cert /\nsubmits precert"] -->|submit"| L1["CT Log A"]
    CA -->|submit"| L2["CT Log B"]
    L1 -->|"SCT (promise)"| CA
    L2 -->|"SCT (promise)"| CA
    CA -->|"cert + embedded SCTs"| SRV["TLS server"]
    SRV -->|"TLS handshake:\ncert + SCTs"| BR["Browser"]
    BR -->|"require valid SCTs\nfrom N logs"| BR
    L1 --> MON["Monitor:\n\nwatches for\n\nyour-domain certs"]
    L2 --> MON
    L1 --> AUD["Auditor:\n\nconsistency +\n\nSCT-inclusion checks"]
    L2 --> AUD
    MON -->|"alert: unexpected\nncert for your domain"| OWN["Domain owner"]
```

The **monitor and auditor roles are both essential, and they answer different questions**. The auditor asks "is the log honest and append-only?" The monitor asks "did anything *bad for me* get logged?" A log that is perfectly append-only but that **nobody monitors** provides transparency *in principle* and none *in practice*: the misissued certificate is sitting there, cryptographically undeniable, and no one is looking. This is the single most important operational lesson from CT, and it transfers directly to Rekor: **transparency is a capability, not an outcome — it delivers security only when someone actually watches**.

CT's success and its limits

CT worked. Misissuance today is *detectable and routinely detected*: Symantec's large-scale misissuance was surfaced through CT-based analysis, and domain owners now catch unauthorized certificates within hours. It changed CA behavior, because every issuance is now public.

But CT's limits are instructive too. CT proves a certificate **exists and was logged**; it says nothing about whether the certificate **should** have been

issued — that judgment is the monitor's, and it requires the domain owner to actually run or subscribe to monitoring. The SCT is a *promise*, so a colluding or buggy log could issue an SCT and never merge the entry; catching that requires auditors to check SCT-to-inclusion, which browsers historically did only partially (fetching inclusion proofs raises privacy questions — the browser reveals which sites you visit to the log). And CT does nothing about the **split-view** problem on its own: a log could theoretically present different trees to different auditors. These gaps drove the gossip and witness work discussed at the end of the chapter.

Rekor: Sigstore's transparency log

Rekor is CT's model applied to **software signing events** rather than TLS certificates. Chapter 3 introduced it as one of Sigstore's three components; here we can be precise about what it stores and why it matters.

What Rekor logs

Rekor is an append-only RFC 6962-style Merkle log (built on Google's **Trillian**, with a newer tile-based backend, **Rekor v2/tessera**, emerging) of **signing events**. Each entry is a typed, canonicalized record. The common **entry types** include:

- **hashedrekord** — the default: the **hash of the signed artifact**, the **signature**, and the **public key or certificate** used. For keyless Sigstore signing, that certificate is the short-lived **Fulcio** certificate carrying the **OIDC identity** in its subject-alternative name (Chapter 3, 4).
- **intoto** and **dsse** — in-toto attestations / DSSE envelopes (Book 5, Chapter 6): the signed *statement* about an artifact (e.g., a SLSA provenance predicate), so provenance itself is transparency-logged.
- Others: **rekord**, **alpine**, **helm**, **jar**, **cose**, **rfc3161**, **tuf** — canonicalizations for specific ecosystems and formats.

Note what Rekor does **not** store: it does not store your artifact. It stores the artifact's **digest** plus the signature material. The log is a record of "*identity X signed something with digest D at time T*", not a copy of the something.

When you submit an entry, Rekor returns a `LogEntry` whose `verification` block carries two things that matter:

- an **inclusion proof** (the $\sim\log(n)$ hashes tying the entry to a signed **checkpoint**), and
- a **Signed Entry Timestamp (SET)** — Rekor's signature over a canonical subset of the entry (log index, log ID, `integratedTime`, and the entry body). The SET is the direct analog of CT's SCT: an immediately-returned, signed attestation that *this entry was integrated at this timestamp*. Unlike CT's SCT, the SET is issued at merge time and is backed by the inclusion proof, so it functions as a **trusted timestamp** you can carry with the signature.

The killer feature: verify-after-expiry

Here is where Rekor earns its place in the architecture. Fulcio certificates live for **about ten minutes** (Chapter 3). A naïve verifier checking a certificate that expired months ago would reject the signature — the certificate is long dead. Rekor's timestamp resolves the paradox.

```
sequenceDiagram
    autonumber
    participant CI as Signer (CI job)
    participant F as Fulcio (CA)
    participant R as Rekor (log)
    participant V as Verifier (later)
    Note over CI,F: t0 short-lived cert issued, valid t0 .. t0+10min
    CI->>F: OIDC identity token
    F-->>CI: cert binding identity->ephemeral key (exp t0+10m)
    CI->>CI: sign artifact digest at time t0+2m
    CI->>R: submit {digest, signature, cert}
    R-->>CI: inclusion proof + SET (integratedTime = t0+2m)
    Note over CI: key discarded; cert will expire at t0+10m
    Note over V: weeks later, cert long expired
    V->>R: fetch entry + inclusion proof + SET
    R->>V: verify SET signature (trusted Rekor key)
    V->>V: check integratedTime (t0+2m) falls INSIDE cert validity
    V->>V: verify signature + cert chain to Fulcio root
    Note over V: signature trusted: it was made while the cert was valid
```

The reasoning: the verifier trusts Rekor's SET as a **timestamping authority**. The SET says the entry — including the certificate and signature — existed in the log at `integratedTime = t0+2m`. That time falls **inside** the certificate's validity window (`t0 .. t0+10m`). Therefore the

signature was **made while the certificate was valid**, even though the certificate is now expired. No long-lived key exists anywhere in this system: the signing key evaporated after ~two minutes, the certificate died after ten, and the durable trust anchors are the *Fulcio root* and the *Rekor key* (both managed via TUF; Chapter 7). This is the precise mechanism by which "short-lived certificate + transparency log" replaces "long-lived signing key" — and it is impossible without the log's trusted timestamp.

Rekor as an auditability tool

The transparency guarantee is only realized if you **query** the log. Rekor exposes a REST API and `rekor-cli`. You can search by artifact digest, by public key, or by **identity**:

```
# Find every log entry for a signing identity (e.g. your CI workflow)
rekor-cli search --email builder@ci.yourcompany.com

# Fetch and verify a specific entry, including its inclusion proof
rekor-cli get --uuid <entry-uuid>

# By artifact hash: has THIS digest ever been signed, and by whom?
rekor-cli search --sha sha256:<digest>
```

Newer tooling (`rekor-monitor`, and identity-based verification in `cosign verify --certificate-identity / --certificate-oidc-issuer`) makes the monitoring pattern concrete: **watch Rekor for your own signing identities**. If your GitHub Actions workflow identity `https://github.com/yourorg/yourrepo/.github/workflows/release.yml@refs/tags/\*` appears in Rekor signing an artifact your team never released, you have **detected a compromise of your CI identity** — exactly the "not silent" property, applied to your own supply chain. Without this monitoring, Rekor is a beautifully-engineered log that provides you nothing; with it, Rekor is a fleet-wide **misuse detector** (Book 8 develops the detection story).

Other transparency applications

The same Merkle-log spine appears in several other systems. Two are worth real attention; a third deserves a mention.

Go's checksum database (sum.golang.org)

Go's module checksum database is, in many engineers' judgment, the **best-executed transparency log in production**, and it is worth studying as a design (Book 2, Chapter 1 covers the dependency-management side).

`go.sum` records a cryptographic hash for every module version your build depends on, in the form `module version h1:base64hash=` plus a second line for the module's `go.mod`. The question is: how do you trust the *first* time you fetch a module and populate `go.sum`? Go's answer is a **transparency log of module hashes** operated at `sum.golang.org`. When the `go` command fetches a new module version, it computes the hash **and** consults the checksum database, which returns the recorded hash together with an **inclusion proof** against a signed tree head. The `go` tool caches the log's tree state (under `$GOPATH/pkg/mod/cache/download/sumdb`) and, on every subsequent interaction, requires a **consistency proof** from the previously seen size to the new one — so the tool *itself* audits the log's append-only property, automatically, on ordinary `go build`s.

What makes the Go design unusually strong:

- **Verification is on by default and built into the standard toolchain.** There is no optional monitor to deploy; the `go` command enforces it (`GOSUMDB=sum.golang.org` by default; `GONOSUMCHECK / GONOSUMDB / GOPRIVATE` opt out for private modules). Contrast CT, where monitoring is a separate thing you must set up.
- **The client is a real auditor.** Every developer's `go` command checks consistency, so the log cannot rewrite history without some client's build failing. The auditing burden is distributed across the entire Go ecosystem's machines.
- **A tiled, efficient log format** (`golang.org/x/mod/sumdb/tlog`) makes proofs cheap enough to fetch on routine builds — the design that later influenced Sigstore's tile-based logs.
- The record is **content-addressed and append-only**: once `example.com/foo v1.2.3` maps to a hash in the log, that mapping is permanent and public, so a maintainer cannot silently swap the bits behind a published version without it being globally visible.

Go pairs this with `proxy.golang.org` (immutable module caching) so the module bits themselves are served consistently. The result: the *default* Go

experience gives you tamper-evident, transparency- backed dependency integrity with no configuration — the gold standard for "transparency that actually delivers because monitoring is not optional."

Binary transparency

Binary transparency applies the CT model to **released binaries**: require every distributed build of a product to be logged to a public append-only log, so consumers and monitors can detect **unexpected releases**. Mozilla implemented a form of it for Firefox (logging the binaries served by its update system); the general concept has been prototyped by several projects, and it underlies the "transparent releases" direction of tools like `cosign` + Rekor for release artifacts.

The threat it addresses is **targeted delivery**. Consider a supply-chain attacker who can serve a **different, backdoored binary to a specific set of victims** while everyone else gets the clean build — the most dangerous form of the SolarWinds-class attack, where malicious payloads are delivered selectively to evade broad detection. With binary transparency, a client refuses to run a binary unless it appears in the public log, and monitors watch the log for builds the vendor never announced. A build served to only a few victims must *either* be absent from the log (so those clients reject it) *or* present in the log (so monitors see it and raise the alarm). Selective, silent delivery becomes impossible: every binary that runs is a public binary. It does not stop the vendor from *building* a backdoor — that is provenance and build-integrity's job (Book 4) — but it removes the attacker's ability to hit a narrow target invisibly.

Key transparency (briefly)

Key transparency (KT) applies transparency logs to the **identity → public key** mapping used by end-to-end encrypted messaging. The problem: your provider tells you your contact's public key, and a malicious or coerced provider could hand you a *substitute* key to man-in-the-middle you silently. KT publishes the key directory to a transparency log so clients can verify their own key is correctly published and monitor for unexpected changes. Because looking a key up by *identity* efficiently needs more than an ordered log, KT typically combines a Merkle **log** with a **verifiable map / prefix tree** keyed by identity. Google's Key Transparency, Keybase, WhatsApp, and Apple's iMessage Contact Key

Verification are production examples. The through-line is identical to CT and Rekor: **make issuance public so substitution cannot be silent.**

The transparency-system landscape

System	What is logged	Who monitors (and how)	Enforcement
Certificate Transparency (RFC 6962)	Every publicly-trusted TLS certificate (as precerts)	Domain owners (via <code>crt.sh</code> , commercial/DIY monitors); auditors check consistency	Browsers require valid SCTs ; unlogged cert = untrusted
Rekor (Sigstore)	Signing events: artifact digest + signature + Fulcio cert / OIDC identity; in-toto & DSSE attestations	You, for your identities (<code>rekor-cli</code> , <code>rekor-monitor</code> , <code>cosign verify --certificate-identity</code>)	<code>cosign verify</code> requires a Rekor inclusion proof + SET
Go checksum DB (sum.golang.org)	<code>module@version</code> → hash records	The go command itself , on every build (consistency proofs), by default	Hash mismatch or missing entry fails the build
Binary transparency	Released/served binaries	Consumers + release monitors watching for unannounced builds	Client refuses to run unlogged binaries (where deployed)
Key transparency	<code>identity</code> → <code>public key</code> directory (log + verifiable map)	The key owner (self-audit) + monitors for unexpected key changes	Client flags/ rejects keys not consistently published

The pattern is one design applied five ways: an append-only Merkle log of *issuance events*, inclusion proofs for "mine is in there," consistency

proofs for "history is intact," and **monitoring** to turn public record into actual detection.

Weaknesses and operational realities

Transparency logs are powerful, but they are not magic, and treating them as magic is how teams end up with a false sense of security. Four honest limitations.

Transparency is worthless without monitoring

This bears repeating as a standalone point because it is the failure mode most likely to bite you. A transparency log makes misuse **detectable**; it does not make it **detected**. If you log every signing event to Rekor but no one ever queries Rekor for your identities, then when your CI identity is compromised and used to sign malware, the evidence sits in a public log that nobody on your team is reading. You have paid the full cost of transparency (see privacy, below) and collected none of the benefit.

The operational requirement is explicit: **run a monitor** for your signing identities. For Sigstore, that means `rekor-monitor` (or equivalent) watching for your CI workflow identities, your team's signing identities, and any Fulcio-issued identity mapping to your organization, with alerts wired into your detection pipeline (Book 8). For dependencies, Go's toolchain monitors for you by default — which is precisely why the Go design is held up as the model. The absence of a monitor is the difference between transparency-in-principle and transparency-in-practice, and it is entirely on you.

The split-view (fork) problem and gossip

The four guarantees included **global consistency**, and it is the one the Merkle math does *not* deliver on its own. Consistency proofs guarantee that a log is append-only **within a single view** — but they cannot, by themselves, stop a malicious or compromised log from maintaining **two views**: presenting checkpoint `A` (and a self-consistent history) to a victim, and checkpoint `B` (a different self-consistent history) to everyone else. This is the **split-view** or **fork** attack. Each view is internally consistent; the victim's inclusion and consistency proofs all verify; and yet the victim is being shown a fabricated reality — a certificate or signing

event that "the log" claims to record for the victim but that no one else can see.

The defense is not more Merkle math; it is **distributed agreement about which checkpoint is canonical**:

- **Gossip.** Clients, monitors, and auditors **exchange the checkpoints they have seen**. If two parties hold two checkpoints for the **same tree size with different roots**, or two checkpoints that cannot be reconciled by a consistency proof, the fork is exposed. Gossip converts a split-view from an invisible attack into a detectable inconsistency the moment two victims compare notes.
- **Witnesses (co-signing).** A stronger, proactive defense: a set of independent **witnesses** each track the log and **co-sign** a checkpoint only after verifying it is consistent with the last checkpoint they signed. A verifier then requires a checkpoint carrying signatures from a quorum of witnesses. To mount a split-view, the log would now have to **collude with a quorum of independent witnesses** to get them to co-sign two forking histories — a far higher bar than compromising the log alone. Sigstore and the Go/ `transparency-dev` community have been building exactly this **witness network** (the checkpoint "note" format carries multiple signatures for precisely this reason). The idea generalizes: consistency proofs make a single log honest; witnesses make the *set of observers* agree on one log.

For a distributed-systems engineer this should feel familiar: it is a consensus problem dressed in cryptography. The log alone is an append-only data structure; witnesses and gossip are the replication-and-agreement layer that keeps all replicas (all observers' views) converged.

Availability, scale, and the log as tier-1 infrastructure

If verification **requires** an inclusion proof from the log — as `cosign verify` and the `go` command do — then the log is on the **critical path of deploys and builds**. That has hard operational implications:

- **The log is tier-1 infrastructure.** If Rekor (or your self-hosted log) is down and your admission policy hard-requires a fresh inclusion proof, deploys stall. Verifiers therefore cache checkpoints and proofs, and policies must decide their behavior on log unavailability (fail-closed vs. fail-open — a real trade-off, discussed at admission in Book 6).

- **Scale is a genuine engineering problem.** The public Rekor instance has grown to tens of millions of entries and has repeatedly pushed the limits of its Trillian/MySQL backend, driving the move to a **tile-based, more horizontally scalable design** (the same tlog/tile approach Go pioneered). A log that everyone must write to on every sign and read from on every verify is a high-throughput, append-only, globally-read datastore — exactly the sort of system backend engineers respect and know is hard to run well.

Privacy: public logs expose what you sign

Transparency is, by construction, **public**. Every entry in Rekor is world-readable: the artifact digests you sign, the identities that signed them, the timestamps, and — for in-toto/DSSE entries — potentially the **attestation contents** (build system, source repo, workflow paths). For open-source software this is a feature. For an enterprise, it can be a **metadata leak**: the public Rekor log reveals your internal artifact naming, your build cadence, the existence of products before announcement, and the structure of your CI. You may not want the world to know that `internal-fraud-detection-service` was built 40 times last Tuesday.

This is a primary motivation for **self-hosting Sigstore** (private Fulcio + private Rekor; Chapter 9): you keep the transparency guarantees — append-only, tamper-evident, monitorable — but the log is **internal**, so metadata stays inside your trust boundary and you control availability and retention. The trade-off is that you now operate a tier-1 log and its monitoring apparatus yourself. For most large organizations running private CI anyway this is the right call; the point is that "public" is not a free property, and the privacy cost is real.

Distributed-systems lens

A transparency log is, stripped of the security framing, a **distributed data structure** — and a particularly elegant one, which is why it rewards a backend engineer's attention.

- **It is a verifiable, replicated, tamper-evident append-only log.** The Merkle tree is the authenticated data structure; inclusion and consistency proofs are $O(\log n)$ membership and prefix queries with cryptographic integrity; the checkpoint is a signed vector-clock-like

commitment to a specific state. If you have built event-sourced systems or append-only ledgers, this is that, with the added property that **clients can verify the log's integrity without trusting the operator.**

- **Consistency proofs are the local invariant; witnesses/gossip are the global consensus.** The math makes any *single* log honest with itself; the split-view problem is the classic distributed-systems difficulty of getting many observers to agree on one history, and the solution — a quorum of independent witnesses co-signing checkpoints — is recognizably a consensus/quorum design. Do not mistake the cryptography for the whole system; the interesting hardness is in the agreement layer.
- **Monitors are watchers; at fleet scale, watch your own log.** Just as you run alerting on your own infra, run **Rekor (or self-hosted log) monitoring for all your signing identities** — every CI workflow, every release identity, every service that signs. This is how transparency becomes *detection* (Book 8): an unexpected entry for one of your identities is a high-signal compromise alert. A log you do not monitor is telemetry you are not collecting.
- **The log lets you trust artifacts across untrusted intermediaries.** Content-addressing (digests) plus a transparency log means a **registry, mirror, CDN, or proxy in the middle need not be trusted**: the consumer verifies the digest against a publicly-logged signing event, and a tampering intermediary is caught because it cannot forge an inclusion proof against a witnessed checkpoint. This is the property that makes global software distribution over untrusted infrastructure safe — the same reason Go can serve modules through `proxy.golang.org` without you trusting the proxy.
- **Treat the log as tier-1.** If it gates deploys, it has the availability and scale requirements of any critical dependency: cache aggressively, decide your fail-open/fail-closed posture deliberately, capacity-plan the write and read paths, and — if privacy or availability demands it — **self-host** (Chapter 9). The log is not a nice-to-have side service; when verification depends on it, it is production infrastructure.

Key takeaways

- **Transparency logs convert silent abuse into detectable abuse.** They do not stop a stolen key or a compromised CA from signing; they make every issuance a **public, append-only, tamper-evident record**, so misuse leaves an indelible trace. *You can still be attacked; you cannot be attacked silently* — but only if someone is watching.
- **The RFC 6962 Merkle construction is the spine.** Leaves hashed with `0x00`, internal nodes with `0x01` (domain separation — a real second-preimage defense, not decoration); the tree is left-complete, not power-of-two-padded. **Inclusion proofs** (~log n sibling hashes) prove *presence*; **consistency proofs** (~log n shared subtree hashes) prove *append-only* by recomputing both old and new roots from the same nodes. Append-only is **cryptographic**, not policy: rewriting history breaks a consistency proof that anyone can check.
- **The signed tree head / checkpoint** is the log's signed commitment to (size, root, timestamp); every proof is verified relative to it, and two forking checkpoints at the same size are how you catch a lying log.
- **Certificate Transparency is the proven precedent.** Born from DigiNotar/Comodo (2011), RFC 6962 requires CAs to log all certs; browsers require **SCTs** (a *promise*, not proof of inclusion); **monitors** watch for misissuance of *your* domains; **auditors** check the log is honest. Both roles are needed — a log nobody monitors is transparent in principle only.
- **Rekor is CT for signing events.** It logs the digest + signature + Fulcio cert/OIDC identity (and in-toto/DSSE attestations), returns an **inclusion proof + Signed Entry Timestamp (SET)**, and the SET's trusted timestamp is what makes **verify-after-expiry** work: a ten-minute certificate is trusted forever because the log proves the signature was made while it was valid. Query Rekor for your identities to **detect misuse**.
- **Go's checksum database is the model execution** because verification and consistency-auditing are **built into the default toolchain** — monitoring is not optional. Binary transparency defeats *targeted* delivery (every running binary is a public binary); key transparency defeats silent key substitution.

- **The honest limits:** transparency needs **monitoring** to have value; **split-view** attacks need **gossip and witnesses** (a quorum-consensus layer) to defeat; the log is **tier-1 infrastructure** with real availability/scale demands; and public logs **leak metadata**, which is why enterprises self-host (Chapter 9).
- **Distributed-systems view:** a transparency log is a verifiable, replicated, append-only Merkle ledger; consistency proofs are the local invariant and witnesses the global consensus; monitors are watchers you should run for your whole fleet; and content-addressing + a witnessed log let you trust artifacts across **untrusted** registries, mirrors, and proxies without trusting them.

Further reading

- **RFC 6962 — Certificate Transparency.** The normative Merkle Tree Hash, inclusion (audit) and consistency proof algorithms, SCTs, and STH. The single most important source for this chapter. <https://www.rfc-editor.org/rfc/rfc6962>.
- **Certificate Transparency project** — the ecosystem, log list, monitors, and the `certificate-transparency` tooling. <https://certificate.transparency.dev/> and the CT policy in the Chrome/Apple root programs.
- **`crt.sh`** — a public CT log search interface; the fastest way to see monitoring in practice for your own domains. <https://crt.sh/>.
- **Sigstore Rekor** — entry types, inclusion proofs, the Signed Entry Timestamp, the REST API, and `rekor-cli`. <https://docs.sigstore.dev/logging/overview/> and <https://github.com/sigstore/rekor>. See also `rekor-monitor` for the monitoring pattern.
- **Sigstore verify-after-expiry** — "How Sigstore works" / the internals docs on short-lived certs plus the log timestamp. <https://docs.sigstore.dev/>.
- **Go Module Mirror, Index, and Checksum Database** — the design of `sum.golang.org`, the tlog/ tile format, and how the `go` command audits it. <https://go.dev/ref/mod#checksum-database> and Russ Cox's "*Transparent Logs for Skeptical Clients*" — the clearest exposition of tiled transparency logs anywhere. <https://research.swtch.com/tlog>.

- **transparency-dev** — the witness network, tessera (tile-based log), and gossip/consistency tooling that Sigstore and Go share. <https://github.com/transparency-dev>.
- **Google Key Transparency and Apple iMessage Contact Key Verification** — the KT variant (log + verifiable map). <https://github.com/google/keytransparency> and Apple's security docs.
- **CONIKS** (Melara et al., USENIX Security 2015) — the academic foundation for key transparency and the split-view/gossip reasoning. Worth reading for the fork-detection argument.
- Cross-references: Book 5, Chapter 1 (Merkle trees, inclusion/consistency proofs, hashing), Chapter 2 (classic signing and silent misuse), Chapter 3 (Sigstore/Rekor architecture, the two logs), Chapter 4 (keyless signing and OIDC identity), Chapter 6 (in-toto/DSSE attestations, the `intoto / dsse` Rekor entry types), Chapter 7 (TUF and the Sigstore trust root/witness keys), Chapter 9 (private Sigstore, self-hosted Rekor, key management); Book 2, Chapter 1 (Go modules, `go.sum`, checksum DB); Book 8 (monitoring transparency logs as fleet-wide detection).

Chapter 6 — in-toto: Attestations, Layouts, and Policies

What this chapter covers. The last four chapters were about signing an artifact and proving *who* released it. Chapter 2 dissected classic code signing; Chapters 3–5 rebuilt the trust model around keyless signing, ephemeral identities, and a transparency log. But a valid signature on a container image answers exactly one question — *who signed this blob* — and it is not the question that matters most. The question that matters is: **did the right steps happen, in the right order, performed by the right actors, with each step consuming exactly what the previous step produced?** A signature on the final artifact is silent about the *process* that produced it. SolarWinds (Book 1, Chapter 3) shipped a perfectly, legitimately signed DLL; the signature was valid because the attacker had corrupted a *step in the middle of the build*, not the signing key. This chapter is about **in-toto**, the framework built precisely to secure the whole chain rather than its last link. We cover the modern, dominant core — the **in-toto Attestation Framework**, whose `Statement`

is the universal container that SLSA provenance, SBOMs, VEX, test results, and vulnerability scans all ride inside — and then the original, still-conceptually-important **layout / link** model that verifies an entire pipeline against a signed policy. We tie attestations to DSSE (Chapter 1) and Rekor (Chapter 5), show how `cosign` produces and verifies them, and end at the policy gate that turns a pile of signed metadata into an admit/deny decision.

Learning goals — after this chapter you should be able to:

- Explain **why signing the final artifact is insufficient** and what in-toto adds: attestation of *each step* plus verification that the steps connect (this step's output was that step's input) and were performed by *authorized* parties — defending against a tampered, skipped, or inserted step.
- State the **in-toto Statement v1** structure exactly — `_type`, `subject`, `predicateType`, `predicate` — and explain that `predicateType` is the **extensibility point** that makes one framework carry provenance, SBOMs, VEX, tests, and custom claims uniformly.
- Draw the **layering**: `predicate` → `Statement` → DSSE envelope (signed) → optionally logged in Rekor — and place SLSA provenance correctly as *one predicate type*, not a separate system.
- Describe the classic **layout model**: a signed policy defining steps, authorized **functionaries**, m-of-n **thresholds**, and **artifact rules** (`MATCH`, `CREATE`, `DELETE`, `REQUIRE`, `ALLOW`, `DISALLOW`) that connect step outputs to step inputs; and how **link metadata** records materials and products per step.
- Explain the **VSA** (Verification Summary Attestation) and how it enables *delegated* verification — verify once centrally, let consumers trust a signed summary.
- Use `cosign attest / cosign verify-attestation` to produce and policy-check attestations stored in an OCI registry, and describe the tooling landscape (Python in-toto, Witness/go-witness, Archivist).

Why signing the final artifact is not enough

Consider the minimal supply chain for a service: a developer's commit is **cloned** from source control, **tested**, **built** into a binary, and **packaged** into a container image that is signed and pushed. Chapters 2–5 let you

sign that final image and prove an identity signed it. Now enumerate what a valid image signature does *not* tell you:

- Was the image built **from the commit it claims** — or from a tampered working tree?
- Did the **tests actually run and pass**, or did someone route around them?
- Was a **malicious step inserted** between build and package — a script that patched the binary after compilation but before signing?
- Did the tarball the build consumed **match** the tarball the clone produced, or was an artifact **swapped** between steps?
- Was each step performed by an **authorized** actor, or did a compromised runner masquerade as the builder?

These are all *process integrity* questions, and a signature on the output is structurally incapable of answering them. SolarWinds is the canonical demonstration: the SUNBURST backdoor was injected by **SUNSPOT**, malware resident on the build servers that replaced source during compilation. The resulting artifact was signed with SolarWinds' *legitimate* code-signing certificate, so every downstream check of the signature passed. The failure was not a stolen signing key; it was a **corrupted build step** inside an otherwise trusted pipeline. Signing the output cannot catch that, because the output *is what the corrupted process produced*.

in-toto's thesis is that you must attest to the **steps**, not just the result. Each step, when performed, emits a signed statement of *what it consumed* (materials) and *what it produced* (products), keyed by cryptographic digest. A separate signed **policy** declares which steps are supposed to happen, in what order, who may perform each, and — crucially — how the artifacts must **flow**: the products of step N must be the materials of step $N+1$. A verifier checks the whole graph. If a step was skipped, its link metadata is missing. If a step was inserted, there is no authorized functionary for it. If an artifact was swapped mid-chain, the digest in one step's products won't match the digest in the next step's materials, and the artifact-flow rule fails. This is **provenance for the entire pipeline**.

in-toto came out of **NYU's Secure Systems Lab** — Santiago Torres-Arias, Justin Cappos, and colleagues — and was presented at **USENIX Security 2019** ("in-toto: Providing farm-to-table guarantees for bytes

and binaries"). It is a **Cloud Native Computing Foundation (CNCF)** project (it entered the CNCF in 2019) and has become the substrate for the broader attestation ecosystem: SLSA provenance (Book 4, Chapter 3) is *defined as* an in-toto attestation, and Sigstore's `cosign attest` speaks the in-toto Statement format natively.

```
flowchart LR
    subgraph pipeline["Supply chain steps (each attested by its performer)"]
        C["clone"] --> T["test"] --> B["build"] --> P["package"]
    end
    P --> IMG["final image + signature"]
    IMG -. "signature answers only: WHO signed this blob" .-> Q1["? built from the right commit"]
    IMG -. "cannot answer" .-> Q2["? tests actually passed"]
    IMG -. "cannot answer" .-> Q3["? no step inserted or swapped"]
    classDef q fill:#fdd,stroke:#900;
    class Q1,Q2,Q3 q;
```

The in-toto Attestation Framework

Modern in-toto splits into two related but distinct things. The **layout/link** model (covered later) verifies a full pipeline against a signed policy — it is the original 2019 design. The **Attestation Framework** is the newer, and today far more widely deployed, core: a **general, typed container for any signed claim about software**. If you have touched SLSA provenance, an SBOM attestation, or `cosign attest`, you have used the Attestation Framework whether you knew it or not. We start here because it is what you will actually encounter.

The Statement

The heart of the framework is the **Statement**, the standardized envelope contents. Statement **v1** has exactly four top-level fields:


```
{
  "_type": "https://in-toto.io/Statement/v1",
  "subject": [
    {
      "name": "ghcr.io/acme/api",
      "digest": { "sha256": "d34db33f0c0ffee5eaf00d1234abcd...9f21" }
    }
  ],
  "predicateType": "https://slsa.dev/provenance/v1",
  "predicate": {
    "buildDefinition": { "...": "..." },
    "runDetails": { "...": "..." }
  }
}
```

Read the four fields precisely — they are the whole model, and inventing extra ones or renaming them is a common source of broken tooling:

- **_type** — a constant identifying the object as an in-toto Statement of a given version: `https://in-toto.io/Statement/v1`. (The older schema used `https://in-toto.io/Statement/v0.1`; v1 is the current one and the field is literally spelled `_type`, with the leading underscore.)
- **subject** — an **array** of `{ name, digest }` objects naming the artifacts the statement is *about*. `digest` is a map from algorithm to hex value (`sha256`, `sha512`, `gitCommit`, ...). The subject is the **binding**: it ties the claim to specific bytes by content hash, so a verifier can confirm *this attestation is about the artifact in front of me* by recomputing the digest. A single statement may cover multiple subjects.
- **predicateType** — a **URI naming the schema** of the `predicate` field. This is the single most important design decision in the framework: it is the **extensibility point**. The predicate type tells a consumer how to parse and interpret the predicate, and lets a verifier select attestations by kind ("give me the SLSA provenance for this image").
- **predicate** — the arbitrary, typed payload: the actual claim, whose shape is defined by the `predicateType`. For SLSA provenance it is `{ buildDefinition, runDetails }`; for an SBOM it is an SPDX or CycloneDX document; for a VSA it is the verification summary; for a custom attestation it is whatever your schema says. The predicate

may be **omitted entirely** when the subject and predicate type carry all the meaning (a "this artifact is of type X" assertion).

The insight to internalize is that **the container is generic and the meaning lives in predicateType**. SLSA Provenance is not a different file format from an SBOM attestation — both are in-toto Statements. They differ only in the `predicateType` string and the shape of `predicate`. One framework, one parser, one verification pipeline, arbitrarily many claim types.

The layering: predicate → Statement → DSSE → Rekor

A Statement is just JSON; on its own it is unsigned and unauthenticated. in-toto attestations are **always** wrapped and signed, and the signing envelope is **DSSE** — the Dead Simple Signing Envelope from Chapter 1. The full stack, from innermost claim to outermost log record:

```
flowchart TB
    PRED["predicate  
(the claim: provenance / SBOM / VSA / test / custom)"]
    STMT["in-toto Statement  
_type + subject + predicateType + predicate"]
    DSSE["DSSE envelope  
payloadType = application/vnd.in-toto+json  
payload = base64(Statement)  
signatures = [ { keyid, sig } ... ]"]
    REKOR["Rekor transparency log entry (optional)  
tamper-evident, publicly auditable"]
    OCI["stored in OCI registry next to the image  
(or in Archivista, or alongside the artifact)"]

    PRED --> STMT
    STMT --> DSSE
    DSSE --> REKOR
    REKOR --> OCI
```

- **DSSE** wraps the Statement. Its `payloadType` is the constant `application/vnd.in-toto+json`, its `payload` is the base64-encoded Statement, and `signatures` is an array of signatures. The signature is computed over the **PAE** (Pre-Authentication Encoding) of `(payloadType, payload)`, not over the raw JSON — this is what stops an attacker from reinterpreting the same bytes under a different type (Chapter 1 covers PAE in detail). Because DSSE carries an *array* of signatures, an attestation can be co-signed — the mechanical basis for the m-of-n thresholds we return to below.

- **Rekor** (Chapter 5) optionally records the signed envelope in a transparency log. Sigstore supports a dedicated `intoto / dsse` entry type; logging makes the attestation **discoverable and tamper-evident**, and — combined with keyless signing's short-lived certificates — lets a verifier trust it long after the signing certificate expired.
- **Storage** is usually the **OCI registry**, next to the image it describes, addressed by the image's digest (Book 3, Chapter 5, on OCI referencers / attestation storage). The attestation travels with the artifact.

Place SLSA provenance correctly in this picture: it is **an in-toto attestation whose predicateType is `https://slsa.dev/provenance/v1`** (Book 4, Chapter 3). SLSA does not define a new envelope, a new signature format, or a new transport. It defines *one predicate schema* (`buildDefinition` + `runDetails`) and the *requirements and levels* around producing it honestly. in-toto is the general framework; SLSA provenance is a specific, standardized predicate plus the build-integrity program built on top. Confusing "in-toto" with "SLSA" — treating them as competitors or as the same thing — is the single most common error in this area. They are a container and one of its contents.

One framework, many predicate types

The predicate ecosystem is where the generality pays off. A handful of standardized predicate types cover most of the secure-supply-chain metadata you will emit, and custom types cover the rest:

```

flowchart TB
    STMT["in-toto Statement  
(same container for all)"]
    STMT --> PROV["slsa.dev/provenance/v1  
build provenance"]
    STMT --> VSA["slsa.dev/verification_summary/v1  
VSA – 'a verifier checked this'"]
    STMT --> SPDX["spdx.dev/Document  
SPDX SBOM"]
    STMT --> CDX["cyclonedx SBOM"]
    STMT --> VEX["OpenVEX / vuln  
exploitability, scan results"]
    STMT --> TEST["in-toto test result"]
    STMT --> LINK["in-toto.io/Link/v1  
(legacy link metadata)"]
    STMT --> CUSTOM["your-org/custom/v1  
anything"]

```

Predicate	predicateType (representative)	What it asserts
SLSA Provenance	https://slsa.dev/provenance/v1	how/from-what/by-what an artifact was built
SLSA VSA	https://slsa.dev/verification_summary/v1	a verifier checked an artifact against a policy and it passed
SPDX SBOM	https://spdx.dev/Document	the components/dependencies in the artifact (SPDX)
CycloneDX SBOM	https://cyclonedx.org/bom	components/dependencies (CycloneDX)
Vulnerability scan	https://in-toto.io/attestation/vulns/v0.1	scanner, timestamp, findings
VEX	OpenVEX / CSAF predicate	exploitability of specific CVEs in this artifact
Test result	https://in-toto.io/attestation/test-result/v0.1	which tests ran and their outcome

Predicate	predicateType (representative)	What it asserts
Link (legacy)	<code>https://in-toto.io/Link/v1</code>	materials/products of a pipeline step (layout model)
Custom	<code>https://acme.example/attestation/review/v1</code>	anything: manual approval, license check, deploy record

The `in-toto/attestation` repository on GitHub is the canonical home for these predicate specs and their JSON schemas; the SLSA site owns the provenance and VSA schemas; SPDX and CycloneDX own their SBOM formats. When you invent a custom predicate, you mint your own `predicateType` URI (ideally under a domain you control and versioned) and document the `predicate` shape — nothing else in the stack changes. The container, the DSSE signing, the Rekor logging, and the OCI storage are all reused verbatim.

Producing and verifying attestations with cosign

`cosign` (Chapter 3) is the workhorse for attestations in the container world. It wraps a predicate in an in-toto Statement, signs it in a DSSE envelope (keyless by default), optionally logs it to Rekor, and stores it in the OCI registry attached to the image digest.

```
# Attach a signed SLSA-provenance attestation to an image (keyless /
OIDC).
cosign attest \
  --predicate provenance.json \
  --type slsaprovenance \
  --yes \
  ghcr.io/acme/api@sha256:d34db33f...9f21

# Attach an SBOM attestation (SPDX predicate) to the same image.
cosign attest \
  --predicate sbom.spdx.json \
  --type spdxjson \
  --yes \
  ghcr.io/acme/api@sha256:d34db33f...9f21

# A custom predicate type by full URI.
cosign attest \
  --predicate review.json \
  --type https://acme.example/attestation/review/v1 \
  --yes \
  ghcr.io/acme/api@sha256:d34db33f...9f21
```

`--type` accepts shorthands (`slsaprovenance`, `slsaprovenance1`, `spdx`, `spdxjson`, `cyclonedx`, `vuln`, `link`) or an arbitrary predicate-type URI. Cosign takes your `--predicate` file as the `predicate`, computes the image's digest as the `subject`, sets `predicateType` from `--type`, assembles the Statement, DSSE-signs it, records it in Rekor, and pushes it to the registry as an attestation associated with `sha256:d34db33f...`. Multiple attestations of different types coexist on the same image.

Verification is a **policy** operation, and this is where the distributed-systems payoff lives:

```
cosign verify-attestation \
  --type slsaprovenance \
  --certificate-identity-regexp '^https://github.com/acme/.+/.github/
workflows/build\.yaml@refs/heads/main$' \
  --certificate-oidc-issuer 'https://
token.actions.githubusercontent.com' \
  --policy provenance-policy.rego \
  ghcr.io/acme/api@sha256:d34db33f...9f21
```

This one command checks several independent things, all of which must hold:

1. There **exists** an attestation of type `slsa-provenance` for this exact image digest.
2. It was signed by the **expected identity** — here, a specific GitHub Actions workflow on `main`, via the expected OIDC issuer (Chapter 4 covers identity-based verification; the identity is bound in the Fulcio certificate's SAN).
3. The signing certificate and the entry are consistent with **Rekor** (transparency, Chapter 5).
4. The **predicate satisfies the policy** — the CUE or Rego file asserts things about the predicate content, e.g. the builder ID equals your platform's, the source repo matches, the build was on the protected branch.

A minimal Rego policy body that `cosign verify-attestation` will evaluate against the decoded predicate:

```
package sigstore

default allow = false

allow {
    input.predicateType == "https://slsa.dev/provenance/v1"
    input.predicate.runDetails.builder.id == "https://github.com/acme/
build-platform/.github/workflows/builder.yml@refs/heads/main"
    startswith(input.predicate.buildDefinition.externalParameters.workf
low.repository, "https://github.com/acme/")
}
```

The mental model to carry forward: **producing** an attestation is one signed statement about an artifact; **verifying** is "does an attestation of the right *type*, from the right *identity*, logged in the right *place*, whose *content* satisfies my *policy*, exist for these exact bytes?" That compound check is the deployment gate (Chapters 8 and 10; Book 6, Chapters 5–6).

The layout model: verifying a whole pipeline

The Attestation Framework gives you signed claims about artifacts. It does **not**, by itself, verify that a *sequence* of steps was followed correctly — that the build consumed exactly what the clone produced, that no step

was skipped or inserted. That end-to-end verification is the job of the original in-toto **layout** model. It is less ubiquitous than the Attestation Framework in cloud-native tooling today (SLSA provenance from a trusted builder often substitutes for a multi-step layout), but it is the *complete* expression of in-toto's thesis and remains the right mental model for full-chain verification, so it is worth understanding precisely.

The model has three roles:

- **The project owner** authors and signs a **layout** — the policy. Its key is the root of trust.
- **Functionaries** perform steps and sign **link metadata** attesting to what each step consumed and produced.
- A **verifier** (often the end user, or an admission gate) checks the link metadata against the layout.

The layout: a signed policy for the pipeline

A **layout** is a signed document that declares the expected shape of the supply chain. Its principal fields:

- **steps** — the sequence of operations the chain must perform (`clone` , `run-tests` , `build` , `package`). For each step the layout specifies:
- **pubkeys** / **threshold** — which functionary keys are authorized to sign this step's link metadata, and how many distinct authorized signatures are required (the **m-of-n threshold**).
- **expected_command** — the command the functionary is expected to have run (advisory but recorded).
- **expected_materials** and **expected_products** — the **artifact rules** governing this step's inputs and outputs (below).
- **inspections** — commands the *verifier* runs at verification time to inspect the final products (e.g., untar the package and confirm its contents match the build's products), each with its own artifact rules.
- **keys** — the public keys of all functionaries, referenced by the steps.
- **expires** — an expiration, so a layout cannot be trusted forever.

The whole layout is signed by the project owner (optionally with its own threshold of owner keys). The owner's public key is the trust anchor the verifier must obtain out of band — this is the one key whose authenticity the whole scheme depends on.

Link metadata: what each step actually did

When a functionary performs a step, it records **link metadata** — a signed statement of that step's execution. A link's fields:

- **name** — the step it corresponds to in the layout.
- **materials** — the **inputs**: a map from file path to digest ({ "src.tar.gz": { "sha256": "..."} }) of everything the step consumed.
- **products** — the **outputs**: the same shape, for everything the step produced.
- **byproducts** — captured stdout/stderr/return value (advisory).
- **command** — the command actually run.
- **environment** — optional execution context.

The link is signed by the functionary's key. **materials** and **products** are the load-bearing fields: they are the *measured* record of what flowed in and out, by content hash, which the layout's artifact rules will be checked against. In the modern framework, link metadata is expressed as a Statement with **predicateType** <https://in-toto.io/Link/v1> — the layout model and the Attestation Framework are the same machinery viewed at different granularity.

Artifact rules: connecting outputs to inputs

Artifact rules are the mechanism that makes the chain a *chain* rather than a set of disconnected steps. Each rule matches file paths and either constrains what a step may touch or, most importantly, asserts that an artifact **flows** correctly from one step to another. The rule verbs:

- **CREATE <pattern>** — the step must have **produced** files matching the pattern (they are in **products** but not **materials**). A build step *creates* the binary.
- **DELETE <pattern>** — the step must have **removed** matching files (in **materials** , not **products**).
- **MODIFY <pattern>** — matching files must appear in both, with **different** digests.
- **ALLOW <pattern>** — matching files are permitted (no constraint beyond permission).

- **DISALLOW <pattern>** — matching files must **not** appear; a catch-all **DISALLOW *** at the end of a rule list rejects anything not explicitly allowed.
- **REQUIRE <pattern>** — matching files must be present.
- **MATCH <pattern> WITH (MATERIALS|PRODUCTS) [IN <path>] FROM <step>** — the **cross-step** rule and the crux of the whole model. It asserts that files matching the pattern in *this* step correspond, by digest, to the materials or products of *another* step.

The **MATCH** rule is what stitches steps together. Its full grammar is:

```
MATCH <pattern> [IN <src-prefix>] WITH (MATERIALS | PRODUCTS) [IN <dst-prefix>] FROM <step-name>
```

Read a concrete example. The **build** step's materials must be exactly the **clone** step's products:

```
# In the layout, for the "build" step:
expected_materials:
- ["MATCH", "src/*", "WITH", "PRODUCTS", "FROM", "clone"]
- ["DISALLOW", "**"]
```

This says: every file under **src/*** that the build step consumed must, by digest, equal a file the clone step produced — and nothing else may be consumed. If an attacker swaps the source tarball between clone and build (a SUNSPOT-style substitution), the digests diverge, the **MATCH** fails, and verification rejects the chain. The trailing **DISALLOW *** forbids any *unexpected* input from sneaking in. Chaining **MATCH** rules across every adjacent pair of steps encodes the full artifact flow: clone→test→build→package, each link pinned by hash to its predecessor.

```

flowchart LR
  subgraph L["Layout (signed by project owner)"]
    LS["steps: clone, test, build, package
+ functionary keys + thresholds
+ artifact rules (MATCH ...)"]
  end
  subgraph links["Link metadata (one per step, signed by functionary)"]
    L1["clone.link
products: src/* @ H1"]
    L2["test.link
materials: src/* @ H1"]
    L3["build.link
materials: src/* @ H1
products: bin @ H2"]
    L4["package.link
materials: bin @ H2
products: image @ H3"]
  end
  L1 -->|"MATCH src/* WITH PRODUCTS FROM clone"| L3
  L3 -->|"MATCH bin WITH PRODUCTS FROM build"| L4
  LS --> V{"Verifier"}
  L1 --> V
  L2 --> V
  L3 --> V
  L4 --> V
  V -->|"all steps present, authorized,
thresholds met, artifacts flow"| PASS["PASS"]
  V -->|"missing / unauthorized step,
or digest mismatch"| FAIL["FAIL"]

```

Verification: checking the chain against the layout

The verifier's algorithm, given the owner's public key, the layout, and the collected link metadata:

1. **Verify the layout's signature** with the owner's public key (and check `expires`). If this fails, stop — the policy itself is untrusted.
2. **Load the functionary keys** declared in the layout.
3. **For each step**: collect the link metadata named for that step, discard any not signed by an **authorized** functionary key, and verify that at least **threshold** distinct authorized functionaries signed it. A missing step (no valid links) or an unauthorized signer fails here.
4. **Apply the artifact rules** for every step's `expected_materials` and `expected_products` against the links' `materials` and `products`. Every **MATCH** must resolve to equal digests in the referenced step; every

`CREATE / DELETE / DISALLOW` must hold. A swapped or tampered artifact fails here.

5. **Run the inspections** from the layout, apply their artifact rules to the results, and confirm the final products match what the last step produced.

If every check passes, the verifier has cryptographic evidence that **the declared steps happened, in order, each performed by an authorized functionary, with artifacts flowing intact from step to step — none skipped, inserted, or swapped**. That is a materially stronger guarantee than any signature on the final artifact.

Functionaries and thresholds

A **functionary** is any party authorized to perform a step and sign its link — a CI runner, a build service, a human reviewer. The layout binds each step to a set of functionary public keys and a **threshold** m : the step is only satisfied if **m distinct authorized functionaries** independently performed it and signed matching link metadata. This is the same **m -of- n** idea as threshold signing in Chapter 1, applied to *steps* rather than to a single signature. Setting `threshold: 2` on the `build` step means an attacker must subvert *two* independent builders that produced *identical products* to forge that step — a strong defense for high-value stages, at the cost of running the step redundantly. Most steps run at threshold 1; reserve higher thresholds for the stages whose compromise is catastrophic (the build itself, a release-approval step).

Delegated verification: the VSA

Full verification — pull every attestation, check every signature and identity, evaluate every policy, maybe re-run inspections — is expensive, and it is wasteful for *every consumer* of an artifact to repeat it. The **Verification Summary Attestation (VSA)** solves this by making the *result of verification* itself an attestation.

A VSA is an in-toto attestation with `predicateType` `https://slsa.dev/verification\_summary/v1`. Its predicate records that **a specific verifier checked a specific artifact against a specific policy, and what the outcome was**. Its notable fields:

- **verifier** — the identity of the party that performed the verification (`{ "id": "https://acme.example/verifier" }`).

- **timeVerified** — when.
- **resourceUri** — the artifact verified (the subject binds the digest).
- **policy** — a reference to the policy that was applied.
- **inputAttestations** — the attestations that were consumed (provenance, SBOM, scans...).
- **verificationResult** — **PASSED** or **FAILED**.
- **verifiedLevels** — the SLSA levels the artifact was confirmed to meet (e.g. ["SLSA_BUILD_LEVEL_3"]).
- **slsaVersion**, **dependencyLevels** — supporting detail.

The point is **delegation of trust in verification**. A central, trusted verifier — your platform security team's service — does the full expensive check once, and emits a signed VSA. Downstream consumers (an admission controller in a cluster that does not run your full policy engine, a partner org, a fleet of thousands of pods) then only need to check **one** thing: does a VSA from *the verifier they trust* exist for this artifact, with `verificationResult: PASSED` and the required `verifiedLevels`? They trust the summary rather than re-deriving it. This is exactly the kind of tiered trust that scales: verify deeply in one place, propagate a cheap signed assertion everywhere else.

```

flowchart LR
    subgraph central["Central verifier (deep check, once)"]
        A1["provenance"] --> V["verify: identity + Rekor + policy"]
        A2["SBOM"] --> V
        A3["vuln scan"] --> V
        A4["test results"] --> V
        V --> VSA["VSA"]
    end
    verificationResult: PASSED
    verifiedLevels: SLSA_BUILD_LEVEL_3"
    end
    VSA --> C1["cluster A admission"]
    VSA --> C2["cluster B admission"]
    VSA --> C3["partner org"]
    C1 -->|"trust the summary"| OK1["admit"]
    C2 -->|"trust the summary"| OK2["admit"]
    C3 -->|"trust the summary"| OK3["consume"]

```

Tooling

The attestation ecosystem has several reference and production implementations; you will mix them depending on where you sit in the pipeline.

- **Python in-toto** (`in-toto/in-toto`) — the reference implementation from NYU. Provides `in-toto-run` (wrap a command and emit signed link metadata), `in-toto-record` (for interactive or multi-command steps), and `in-toto-verify` (check links against a layout). This is the canonical home of the **layout/link** model.
- **Witness / go-witness** (`in-toto/witness` , from TestifySec) — a Go tool that runs **in-pipeline**: you wrap a build command with `witness run` , and it observes the execution (via attestors for git, the environment, materials/products, GitLab/GitHub context, SLSA provenance, and more) and emits a signed in-toto attestation capturing what happened. Witness is designed to generate rich attestations *as the pipeline runs*, rather than after the fact.
- **Archivista** (`in-toto/archivista`) — a GraphQL-fronted **storage and retrieval** service for attestations, letting you store the DSSE envelopes Witness produces and query the attestation graph (find all attestations for a subject digest, traverse materials→products links across steps). It is the persistence layer for attestation-heavy pipelines.
- **cosign** (`sigstore/cosign`) — the dominant tool for *producing and verifying attestations in the OCI world*, as shown above: keyless DSSE signing, Rekor logging, registry storage, and policy-based `verify-attestation` .
- **The in-toto/attestation repository** — not a tool but the **spec home**: the Statement schema, the predicate-type registry, and the JSON schemas for the standard predicates. When in doubt about a field name, this is the authority; do not guess.

A common production shape: Witness emits attestations during the build and stores them in Archivista; the paved-road platform *also* has `cosign attest push` SLSA provenance and an SBOM to the OCI registry; a central verifier evaluates the lot and emits a VSA; admission trusts the VSA.

Verifying attestations in practice: the deployment gate

Everything in this chapter converges on a single operational moment: **the gate that decides whether an artifact may be deployed**. This is policy-based verification, and it is where attestations stop being metadata and start being enforcement. It previews Chapters 8 and 10 and Book 6, Chapters 5–6.

A representative fleet policy: *an image may be admitted to production only if it has (1) SLSA Build-Level-3 provenance from our build platform's identity, (2) an SBOM, (3) a passing vulnerability scan no older than 24 hours, and (4) all of the above logged in Rekor*. An admission controller evaluates that at deploy time against the attestations attached to the image:

```
flowchart LR
    subgraph reg["OCI registry"]
        IMG["image @ sha256:..."]
        AP["provenance attestation"]
        AS["SBOM attestation"]
        AV["vuln-scan attestation"]
    end
    DEP["deploy request"] --> GATE["admission policy verify"]
    IMG --> GATE
    AP --> GATE
    AS --> GATE
    AV --> GATE
    GATE -->|"provenance L3 from our builder  
+ SBOM present  
+ scan passing and fresh  
+ Rekor-logged  
+ signed by expected identity"| ADMIT["ADMIT"]
    GATE -->|"any check fails"| DENY["DENY"]
```

The controller does not care *how* the attestations were produced — Witness, cosign, a bespoke signer — only that they are the right *types*, from the right *identities*, logged in the right *place*, and that their *content* satisfies policy. That uniformity is the entire reason the in-toto container is generic: one verification engine consumes provenance, SBOMs, VEX, and test results identically, selecting by `predicateType` and evaluating each against the relevant rule. Chapter 8 builds the policy engine; Chapter 10 and Book 6 wire it into Kubernetes admission (`policy-controller`, Kyverno, sigstore image policies).

Distributed-systems lens

Attestations are the **universal metadata currency** of the secure supply chain, and in-toto is the mint. At fleet scale — hundreds of services, dozens of teams, thousands of deploys a day — the architecture that actually works has three properties, and in-toto supplies all three:

- **Uniformity through one container.** Every stage — source, build, scan, test, deploy — emits a signed in-toto Statement, differing only in `predicateType`. Provenance, SBOM, VEX, test results, and your custom "SRE-approved-this-release" claim all flow through the *same* signing (DSSE), the *same* transport (OCI referrers, Book 3, Chapter 5), the *same* transparency log (Rekor, Chapter 5), and the *same* verification engine. You do not build one pipeline for provenance and another for SBOMs; you build one attestation pipeline and vary the predicate. That is the only way the metadata problem stays tractable across hundreds of services.
- **Attestations travel with the artifact.** Because they are stored next to the image by digest, an attestation is portable across registries, clusters, and organizational boundaries. The artifact and its provenance move as a unit; verification is possible anywhere the artifact lands, with no side channel to the pipeline that built it.
- **The paved road produces them for free; the gate consumes them.** The secure build platform (Book 4, Chapter 10) auto-generates a *standard set* of attestations for **every** artifact — no team opts in, no team can opt out — and admission (Book 6) verifies them. This is how you enforce supply-chain requirements across an entire org without asking every team to become a security expert: the platform emits, the gate checks, and non-conforming artifacts simply cannot deploy. **Attestations plus policy are the enforcement mechanism**; in-toto is what makes them composable.

And **VSA closes the scaling loop**: full verification is expensive and does not need to run in every cluster and every consumer. Verify deeply, once, in a central trusted service; emit a signed summary; let thousands of downstream consumers trust the summary. This is the same tiered-trust pattern that lets any large distributed system scale a costly operation — do the hard work in one authoritative place, and propagate a cheap, verifiable assertion of the result everywhere else.

The through-line from Chapter 2 to here: a signature answers *who signed this blob*; in-toto answers *was the right process followed by the right actors to produce it*. Signing proves identity; attestation proves **process**. Keep the axes separate — and note the honest limit that carries over from Chapter 3: an attestation is only as trustworthy as the step that produced it. A compromised, authorized builder can emit a perfectly valid provenance attestation for a malicious artifact. in-toto raises the bar from "steal a signing key" to "subvert an attested, authorized, threshold-gated step whose products are hash-pinned to its neighbors" — a dramatically harder attack — but it does not make "attested" mean "benign." Provenance and policy tell you *how* something was built and by *whom*; whether that *how* is trustworthy is the subject of build-integrity levels (SLSA, Book 4) and of the policies you write at the gate.

Key takeaways

- **Signing the final artifact proves who signed a blob; it says nothing about the process.** SolarWinds shipped a legitimately signed DLL corrupted by a subverted build step. in-toto secures the **whole chain**: each step is attested by whoever performed it, and a policy verifies the steps connect (output → input, hash-pinned) and were done by authorized parties.
- **The in-toto Statement v1 has exactly four fields:** `_type` (`https://in-toto.io/Statement/v1`), `subject` (array of `{name, digest}` — the artifacts, bound by content hash), `predicateType` (a URI naming the predicate schema), and `predicate` (the typed payload). **predicateType is the extensibility point** — one generic container carries provenance, SBOMs, VEX, tests, and custom claims uniformly.
- **The layering is:** `predicate` → `Statement` → **DSSE** envelope (signed, `payloadType application/vnd.in-toto+json`) → optionally **Rekor** → stored in the **OCI** registry next to the image. **SLSA provenance is one predicate type** (`https://slsa.dev/provenance/v1`), **not a separate system** — in-toto is the framework; SLSA is a specific predicate plus the levels around it.
- **The classic layout model** verifies a full pipeline: a project owner signs a **layout** (steps, authorized **functionary** keys, m-of-n **thresholds**, and **artifact rules**); each step emits signed **link** metadata (`materials`, `products`); the verifier confirms every step

ran, was authorized, met threshold, and that artifacts flow intact. The **MATCH ... WITH PRODUCTS FROM <step>** rule is the crux — it hash-pins one step's inputs to another's outputs, defeating mid-chain artifact swaps.

- **The VSA** (https://slsa.dev/verification_summary/v1) records that a verifier checked an artifact against a policy and the result (**PASSED / FAILED** , **verifiedLevels**). It enables **delegated verification**: verify deeply once, centrally; let downstream consumers trust the signed summary.
- **cosign attest --predicate ... --type ...** produces attestations; **cosign verify-attestation --type ... --certificate-identity-regexp ... --policy ...** checks type + identity + Rekor + policy content in one gate. Tooling: reference **Python in-toto** (layout/link), **Witness/go-witness** (in-pipeline attestation), **Archivista** (storage), and the **in-toto/attestation** spec repo.
- **Distributed-systems view**: attestations are the universal signed-metadata currency; one framework covers every claim type uniformly; the paved-road platform auto-emits a standard attestation set for every artifact and admission verifies them at the gate; VSA lets you verify once and propagate a cheap signed summary across the fleet. **Attestations + policy are how supply-chain requirements are enforced across hundreds of services.**

Further reading

- **in-toto specification** — the layout, link, and artifact-rule model. <https://github.com/in-toto/docs> and the in-toto website. The formal grammar of **MATCH / CREATE / DELETE / ALLOW / DISALLOW / REQUIRE** lives here.
- **in-toto Attestation Framework** (**in-toto/attestation**) — the Statement v1 schema, the DSSE envelope binding, and the **predicate-type registry** with JSON schemas for the standard predicates. <https://github.com/in-toto/attestation>. The authority on field names; do not guess.
- **"in-toto: Providing farm-to-table guarantees for bytes and binaries"** — Torres-Arias, Afzali, Kuppusamy, Curtmola, Cappos, *USENIX Security 2019* — the original design and threat model.

- **SLSA v1.0** — provenance (<https://slsa.dev/provenance/v1>) and the **VSA** (https://slsa.dev/verification_summary/v1) specs, and the levels/requirements around producing provenance. <https://slsa.dev/spec/v1.0/> (developed in Book 4, Chapter 3).
- **DSSE — Dead Simple Signing Envelope** — the envelope and PAE that wrap and sign every attestation. <https://github.com/secure-systems-lab/dsse> (mechanics in Chapter 1).
- **Cosign attestations** — `cosign attest` / `cosign verify-attestation`, predicate types, and OCI storage. <https://docs.sigstore.dev/> and <https://github.com/sigstore/cosign> (Chapter 3).
- **Witness / go-witness and Archivist** (TestifySec, in the in-toto org) — in-pipeline attestation generation and storage. <https://github.com/in-toto/witness> and <https://github.com/in-toto/archivist>.
- **SPDX 3.0 and CycloneDX 1.6** — the SBOM predicate formats carried as attestations (Book 3).
- **sigstore/policy-controller** and **Kyverno** — admission-time attestation verification for Kubernetes. Applied in Chapter 10 and Book 6, Chapters 5-6.
- Cross-references: Book 5, Chapter 1 (DSSE, PAE, thresholds), Chapters 3-5 (Sigstore, keyless identity, Rekor), Chapter 8 (policy engines), Chapter 10 (deployment gates); Book 4, Chapters 3, 10 (SLSA provenance, secure build platform); Book 3, Chapter 5 (SBOMs, OCI attestation storage); Book 1, Chapter 3 (SolarWinds); Book 6, Chapters 5-6 (admission-time verification).

Chapter 7 — TUF: The Update Framework

What this chapter covers. Every chapter so far has been about proving something true of a single artifact: this blob was signed by that identity (Chapters 2-4), this signing event was logged (Chapter 5), these steps produced this output (Chapter 6). This chapter changes the question. It is not "is this artifact authentic?" but "**can I still get authentic artifacts when the system that serves them has been compromised?**" A software repository — PyPI, a container registry, an internal mirror — is a *distributed system that many parties can attack*: its web servers, its CDN,

its storage, its signing keys. Naive signing assumes the signing key is safe and everything else is untrusted. That assumption is exactly backwards for the threats that matter. **The Update Framework (TUF)** is a design that assumes the repository *will* be partially compromised — a stolen key here, a hijacked CDN there — and bounds the damage so that, in most compromise scenarios, an attacker still cannot make you install malware, and in the scenarios where they can slow you down, you can *detect* it. This is **compromise resilience**, and it is TUF's one big idea. We take apart the four roles and their metadata precisely, walk the client update algorithm in order, show exactly how online/offline key separation and thresholds contain a key compromise, and then look at where TUF actually runs in production — most importantly, *inside Sigstore itself* (Chapter 3), where TUF is the root of trust for a root of trust.

Learning goals — after this chapter you should be able to:

- Explain why **signing alone is not enough** for an update system, and enumerate the repository-level attacks TUF is designed to survive — arbitrary-package, rollback, freeze, fast-forward, mix-and-match, endless-data, slow-retrieval — and the mechanism that defeats each.
- State the **four top-level TUF roles** — root, targets, snapshot, timestamp — with their exact responsibilities and the metadata each signs, without confusing their duties.
- Describe **online/offline key separation** and **m-of-n thresholds**, and explain precisely how they *bound the damage* of a key compromise: why stealing the frequently used online keys still does not let an attacker forge malicious content.
- Walk the **client update workflow** in its mandated order — root → timestamp → snapshot → targets → download-and-verify — and justify why the order (authority, then freshness, then consistency, then content) is load-bearing.
- Explain **delegation** of the targets role and why it scopes trust to sub-maintainers, and how **key rotation and recovery** through the root role let a TUF repository recover from compromise rather than collapse.
- Place TUF in the real world accurately: **PEP 458/480** for PyPI, **Notary v1 / Docker Content Trust, Sigstore's TUF root** (go-tuf, root-signing), and **Uptane** for automotive — and articulate how

TUF underpins the Sigstore trust you have been relying on since Chapter 3.

Why signing an artifact is not enough

Start from where Chapter 2 left off. You sign a package with a private key; clients hold your public key and verify the signature. If the key stays secret, an attacker who compromises the repository cannot forge a valid signature over malware, and you are done. That is the naive model, and it fails for two independent reasons.

The first is that it defends against exactly one attack — content substitution — and a repository is vulnerable to *many* attacks that have nothing to do with forging a signature. An attacker who controls the repository or its CDN does not need to sign anything to hurt you. They can serve you a **genuinely signed but old** version of a package — one with a known, exploitable vulnerability — and your signature check passes, because the old package really was signed. They can **withhold updates** entirely, pinning you to a stale, vulnerable state while a CVE is being actively exploited, and again every signature you see is valid. They can serve a package whose signature is fine but whose *dependencies* are an inconsistent, never-tested combination. None of these are defeated by "is the signature valid?", because in each case it is.

The second reason is the one that defines TUF's threat model: **the signing key is not categorically safer than the repository**. Book 1's case studies are a catalogue of stolen and misused signing keys — the NVIDIA and stolen-driver-certificate incidents in Chapter 2, the long-lived key that *is* the single point of failure. A repository is a high-value target (Book 2, Chapter 1 on the dependency attack surface; Book 6, Chapter 2 on registries), and its signing infrastructure is *part of that target*. If your entire security model is "one key, kept safe," then the compromise of that one key is total: the attacker signs whatever they want and every client accepts it. There is no containment, no detection, and no recovery short of manually re-pinning a new key at every client. For infrastructure that ships code to millions of machines, "one key compromise equals game over" is not an acceptable failure mode.

TUF's thesis is to invert the assumption. **Assume the repository will be partially compromised** — some server, some CDN edge, some key — and design so that the *set* of things an attacker must simultaneously

compromise to make you install malware is large, well-defended, and mostly offline; and so that the attacks achievable with a lesser compromise are limited to *denial and delay*, which are **detectable**. This property has a name: **compromise resilience**. It is not "the repository cannot be hacked" — nothing offers that — it is "hacking the repository, up to and including stealing its most-exposed keys, does not get the attacker arbitrary code execution on your machines, and cannot silently freeze you forever."

TUF came out of **NYU's Secure Systems Lab** — Justin Cappos, Trishank Karthik Kuppusamy, Sebastien Awwad, and colleagues — building on Cappos's earlier "package management security" work (the *Thandy*/Stork lineage and the 2008 study of how badly real package managers handled repository compromise). It is a **Cloud Native Computing Foundation (CNCF)** graduated project. Its design has been adopted by Python (**PEP 458/480** for PyPI), **Docker/Notary v1**, **Sigstore** (as its root of trust), the automotive industry (**Uptane**), RubyGems, and others. The framework is a *specification* with multiple implementations — `python-tuf` (the reference implementation) and `go-tuf` chief among them — not a single program.

The attacks TUF is designed to survive

TUF's design reads backwards from a threat enumeration. Every role and every metadata field exists to defeat a specific named attack, so it is worth stating the attacks first — they *are* the requirements.

- **Arbitrary software / arbitrary package.** The attacker gets you to install a file of their choosing. This is the attack ordinary signing already stops *if the signing key is safe*. TUF keeps that property but no longer depends on a single key staying safe.
- **Rollback attack.** The attacker serves an *older, legitimately signed* version of a package or of metadata — often one with a known vulnerability — to downgrade you into an exploitable state. The signature is valid; the problem is that it is *stale*.
- **Freeze attack (indefinite freeze).** The attacker prevents you from ever seeing new updates by continuing to serve the last metadata you accepted. You never learn a patch exists. Every byte is validly signed.

- **Fast-forward attack.** A subtler cousin: an attacker who has (temporarily) compromised an online key inflates a metadata **version number** to an enormous value. After the key is rotated and legitimate publishing resumes at a normal version number, clients that saw the inflated version reject the *legitimate* new metadata as a rollback — a self-inflicted denial of service planted during the compromise window.
- **Mix-and-match attack.** The attacker serves a combination of packages (and delegated metadata) that were each individually signed but were **never published together** and never tested as a set — e.g., a new `libfoo` with an old, incompatible `libbar` chosen to reopen a vulnerability.
- **Endless data attack.** The server responds to a metadata or target request with an endless stream of bytes to exhaust the client's disk or memory.
- **Slow retrieval attack.** The server trickles bytes just fast enough to keep the connection alive, stalling the client indefinitely without ever technically timing out.
- **Key compromise.** The headline. The attacker steals one or more of the repository's signing keys. TUF's defining contribution is that the *consequences* of a key compromise depend on *which* key and *how many* — and that the keys most exposed to theft are precisely the ones whose theft grants the least power.

The following table maps each attack to the TUF mechanism that defeats it. The mechanisms are introduced in the next sections; treat this as the map you are about to walk.

Attack	What the attacker achieves	TUF defense
Arbitrary package	Install attacker's file	Target hashes/length in targets metadata, signed by an offline key; verified after download
Rollback	Downgrade to old vulnerable version	Monotonic version numbers in every metadata file; timestamp pins the current snapshot version

Attack	What the attacker achieves	TUF defense
Freeze / indefinite freeze	Withhold updates silently	Short expiration timestamps, especially on timestamp metadata; clients reject expired metadata
Fast-forward	Poison version numbers to block recovery	Rotating the compromised keys via root re-establishes trust; version-number reset on re-keying
Mix-and-match	Serve an inconsistent, untested set	snapshot metadata commits one consistent set of metadata versions
Endless data	Exhaust client resources	Exact length (and hashes) recorded in the referring metadata; client stops at the declared length
Slow retrieval	Stall the client	Client-side download-rate/timeout enforcement (implementation requirement in the spec)
Key compromise	Steal a signing key	Role separation + online/offline split + m-of-n thresholds bound the damage

The four roles and their metadata

TUF's core is a set of **signed metadata files**, one per **role**. A role is a named responsibility with an associated set of authorized keys and a signing **threshold** (how many of those keys must sign for the metadata to be valid). There are four top-level roles. Getting their responsibilities exactly right is the whole game; they are frequently confused, and the security argument collapses if you blur them.

root — the trust anchor

The **root** role is the anchor from which everything else is derived. Root metadata (`root.json`) does one job: it **lists which public keys are authorized for each of the four roles**, and the threshold for each. Concretely, root's metadata contains a `keys` dictionary (key ID → public key) and a `roles` dictionary mapping each of `root`, `targets`, `snapshot`,

and `timestamp` to the set of key IDs authorized to sign that role and the number of signatures required. It also carries a `version`, an `expires` timestamp, and a `consistent_snapshot` flag (discussed below).

Root is signed by the **root keys** themselves, at a threshold — typically several keys held by several people, e.g. 3-of-5. These keys are kept **offline**: on hardware tokens or HSMs, in a safe, brought together only for a deliberate signing ceremony. Root metadata **rarely changes** — only when a role's keys are added, removed, or rotated. It is the metadata a client bootstraps from: a TUF client ships with a pinned copy of a trusted `root.json` and, from that anchor, learns which keys to trust for every other role. Because root defines *all other trust*, **root key compromise is the worst case** — an attacker who controls a threshold of root keys can re-delegate every other role to keys they hold. The entire design of root — offline storage, a high threshold across multiple holders, infrequent use — exists to make reaching that threshold as hard as possible.

targets — what is actually available, and delegation

The **targets** role describes the actual files clients can download — the "targets." Targets metadata (`targets.json`) lists each available target file by path, recording for each its **length** and one or more **cryptographic hashes**, plus an optional free-form `custom` dictionary (where deployments stash things like the package version, or in Sigstore the *role* of a given key file). A client that trusts targets metadata can download a file and verify it byte-for-byte against the recorded length and hash: this is the mechanism that actually defeats arbitrary-package substitution. Targets is signed by the **targets keys**, and — like root — these are high-value keys that should be kept **offline**, because forging a malicious *target* requires them.

Targets' second power is **delegation**. Instead of one targets file listing every package in a large repository, the targets role can **delegate** authority over a subset of the target namespace to other roles — *delegated targets* roles — each with its own keys and threshold. Delegations are scoped: a delegated role is granted authority over specific paths (glob patterns like `django/*`) or over targets whose path hashes fall under given prefixes (`path_hash_prefixes`, used to shard a huge namespace evenly). Delegations can be **terminating** or non-terminating, which controls whether the client keeps searching other delegations if the target is not found in this one. The metadata expresses

this in a `delegations` object carrying the delegated roles' keys and their `roles` list (name, key IDs, threshold, paths).

Delegation is what makes TUF usable at repository scale and what makes *end-to-end* developer signing possible. A registry can delegate `numpy/*` to the NumPy maintainers' keys and `django/*` to the Django maintainers' keys, so that each project's releases are signed by *that project's* offline keys. The top-level targets key never signs individual packages; it only signs the delegation structure. This directly bounds compromise: stealing the Django delegated key lets an attacker forge Django releases and *nothing else* — the delegation's path scope confines the blast radius to `django/*`. PEP 480 (below) uses exactly this to give PyPI maintainers their own signing keys.

snapshot — one consistent set

The **snapshot** role solves mix-and-match. Snapshot metadata (`snapshot.json`) records the **version number of every targets metadata file** in the repository — the top-level `targets.json` and every delegated targets file — as a single, atomically published set. Its `meta` field maps each metadata filename to (at least) its current version number. The guarantee snapshot provides is that **every client sees the same consistent set of metadata versions**: the exact combination that the repository operator published and (implicitly) intends to be used together. An attacker cannot splice a new `libfoo.json` together with an old `libbar.json` of their choosing, because a valid snapshot commits *one* version of each, and the client checks the metadata it downloads against the versions snapshot names. Snapshot is signed by the **snapshot key**, which is an **online** key — it must be re-signed whenever *any* targets metadata changes, which on a busy repository is constantly, so it cannot live in a safe.

timestamp — freshness

The **timestamp** role solves freeze and rollback of the whole tree. Timestamp metadata (`timestamp.json`) is tiny: it references the **current snapshot** file, recording snapshot's version number, length, and hashes. It is **re-signed frequently and expires quickly** — on the order of a day for public repositories, sometimes far less. Because it is the first thing a client checks and it carries a short `expires`, a client that is being served stale metadata *notices*: either the timestamp has expired (freeze

detected) or it names a snapshot version lower than one the client already trusts (rollback detected). Timestamp is signed by the **timestamp key**, the **most frequently used and most exposed** key in the system — it lives online and re-signs continuously. And here is the crux of the whole design: **compromising the timestamp key alone buys the attacker almost nothing**. They can serve a stale-but-not-yet-expired timestamp (a brief freeze) or point at an older snapshot (a rollback among already-published states), both of which are bounded and detectable — but they *cannot forge new malicious content*, because that requires the offline targets key, which timestamp compromise does not grant.

The following table is the one to remember.

Role	Responsibility	Key exposure	Signs how often	Damage if this key alone is compromised
root	Authorizes keys/thresholds for all roles	Offline, high threshold (m-of-n)	Rarely	Catastrophic <i>only if threshold reached</i> : attacker re-delegates every role. The anchor is the hardest target by design.
targets	Lists targets with hash+length; delegates	Offline	On release / delegation change	Can forge malicious targets — but the file must also be admitted into a signed snapshot and timestamp; still bounded by delegation scope

Role	Responsibility	Key exposure	Signs how often	Damage if this key alone is compromised
snapshot	Commits one consistent set of metadata versions	Online	On any metadata change	Rollback/mix-and-match among <i>already-published</i> metadata; cannot forge new targets
timestamp	Points at the current snapshot; short expiry	Online (most exposed)	Continuously	Freeze / rollback of the whole tree, bounded and detectable ; cannot forge new targets

The picture

```

flowchart TB
    subgraph offline["OFFLINE keys – kept in HSM / air-gapped / safe"]
        ROOT["root.json"]
        ROOT -->|authorizes keys + thresholds  
for all four roles| TARGETS["targets.json"]
        TARGETS -->|target paths + hashes + lengths  
+ delegations| DELEG["delegated targets"]
        DELEG -->|django/*, numpy/*, ...  
per-maintainer offline keys| end1["end"]
    end
    subgraph online["ONLINE keys – connected signing service"]
        SNAP["snapshot.json"]
        SNAP -->|version of every  
targets metadata file| TS["timestamp.json"]
        TS -->|points at current snapshot  
short expiry| end2["end"]
    end

    ROOT -->|"authorizes keys for"| TARGETS
    ROOT -->|"authorizes keys for"| SNAP
    ROOT -->|"authorizes keys for"| TS
    ROOT -->|"authorizes keys for"| ROOT
    TARGETS -->|"delegates paths to"| DELEG
    SNAP -->|"commits versions of"| TARGETS
    SNAP -->|"commits versions of"| DELEG
    TS -->|"commits version+hash of"| SNAP

```

Read the arrows as trust flow. Root authorizes the keys for every role (including itself, for rotation). Targets delegates sub-namespaces to delegated targets roles. Snapshot commits *which version* of every targets file is current. Timestamp commits which version of snapshot is current. The dashed line between the two boxes is the single most important architectural fact in TUF: the two roles whose keys are exposed online — snapshot and timestamp — are the two roles that cannot, by themselves, introduce new content.

How key compromise is contained

Now make the containment argument explicit, because it is the reason TUF exists. Ask the precise question: *what does an attacker have to compromise, simultaneously, to make a client install a file of the attacker's choosing?*

To get you to accept a malicious file, the attacker needs three things to line up. (1) A **valid targets metadata entry** for that file — the right hash and length, signed by an authorized targets or delegated-targets key. (2) That targets file admitted into a **valid snapshot** — signed by the snapshot key. (3) That snapshot referenced by a **valid, unexpired timestamp** — signed by the timestamp key. The client checks all three, in order, and each check is against keys that *root* authorized.

The snapshot and timestamp keys are online and comparatively easy to steal — but stealing them gets the attacker only (2) and (3). They still lack (1): a *fresh, malicious targets entry* signed by the offline targets key. Without it, the best they can do with online keys is replay, rollback, or freeze *content that was already legitimately signed* — annoying and worth detecting, but not arbitrary code execution. To get (1), they must reach the **offline** targets key (or an offline delegated key, whose damage is confined to that delegation's path scope). And for the absolute worst case — re-delegating everything — they must reach a **threshold** of the **offline** root keys held by multiple people.

flowchart TB

```

    A["Attacker steals ONLINE keys
(timestamp + snapshot)"] --> B["Can they forge
a NEW malicious target?"]
    B -->|"NO – needs offline targets key"| C["Limited: freeze / rollback
among already-signed content"]
    C --> D["DETECTABLE: expired timestamp
or lowered version number"]

    E["Attacker also steals the
OFFLINE targets key"] --> F["Enough to serve
arbitrary malware?"]
    F -->|"Only within delegation scope,
and only if online keys also held"| G["Bounded blast radius
(e.g. one project's paths)"]

    H["Attacker reaches THRESHOLD of
OFFLINE root keys (m of n, many holders)"] --> I["Worst case:
re-delegate every role"]
    I --> J["Requires compromising multiple
people's hardware keys at once"]

```

This is a strict improvement over flat single-key signing on two axes. First, **the easy-to-steal keys are the low-power keys**: online exposure is inversely correlated with the damage a stolen key does. Second, **the high-power keys require multi-party, offline, threshold**

compromise: there is no single key whose theft ends the game. A flat model has neither property — its one key is both maximally exposed (it signs every release, so it is used constantly) and maximally powerful (it is the trust). Thresholds add the final margin: even holding one offline root key is not enough; you need m of them, from m different holders' hardware, before the anchor moves.

The client update workflow

A TUF client does not just "download and check a signature." It runs a fixed, ordered algorithm over the metadata, and the **order is part of the security argument**. The client begins with a locally trusted `root.json` (shipped with the client, pinned) and proceeds:

```
sequenceDiagram
    participant C as Client (trusted root pinned)
    participant R as Repository / CDN
    Note over C: 1. Update ROOT (authority)
    C->>R: fetch N+1.root.json, N+2.root.json, ...
    R-->>C: newer root metadata (if any)
    Note over C: verify each with OLD threshold AND NEW threshold;
    walk the chain to latest; check expiry
    Note over C: 2. Update TIMESTAMP (freshness)
    C->>R: fetch timestamp.json
    R-->>C: timestamp metadata
    Note over C: verify sig vs root-authorized timestamp keys;
    reject if version < trusted; reject if expired
    Note over C: 3. Update SNAPSHOT (consistency)
    C->>R: fetch snapshot.json (version from timestamp)
    R-->>C: snapshot metadata
    Note over C: verify length+hash vs timestamp;
    verify sig; reject any targets version rollback; check expiry
    Note over C: 4. Update TARGETS (content metadata)
    C->>R: fetch targets.json (version from snapshot)
    R-->>C: targets metadata
    Note over C: verify length+hash vs snapshot; verify sig; check
    expiry;
    follow delegations to locate the target
    Note over C: 5. DOWNLOAD + VERIFY the target
    C->>R: fetch target file
    R-->>C: file bytes
    Note over C: verify length + hashes vs targets metadata;
    stop at declared length (endless-data defense)
```

Step by step, and why the order is what it is:

1. **Update root first (authority before anything).** The client repeatedly fetches the next version of root (`2.root.json` , `3.root.json` , ...) and verifies each against **both** the *previous* root's threshold **and** the *new* root's threshold. This double check is what makes root rotation safe: a new root is trusted only if the currently trusted root's keyholders signed off on it *and* the new root is self-consistent. The client walks this chain to the latest root, then checks expiry. Root is updated first because it defines which keys are valid for every subsequent step — you cannot check any other signature until you know whose keys to trust.
2. **Update timestamp (freshness before consistency).** Fetch `timestamp.json` , verify its signature against the (now up-to-date) root-authorized timestamp keys, reject it if its version is lower than the timestamp the client already trusts (rollback), and reject it if it has expired (freeze). Timestamp is small and cheap, and doing it early means the client detects a freeze or rollback *before* spending effort on the larger metadata.
3. **Update snapshot (consistency).** The trusted timestamp names the current snapshot's version, length, and hash. Fetch `snapshot.json` , verify its length and hash against what timestamp promised, verify its signature against root-authorized snapshot keys, and check that **no targets metadata version has decreased** relative to the snapshot the client already trusted (rollback of any individual targets file), then check expiry. After this step the client knows the exact, consistent set of targets-metadata versions it is allowed to use.
4. **Update targets (content metadata).** For each targets file it needs (starting with top-level `targets.json` , following delegations toward the specific target), the client fetches the version snapshot named, verifies length and hash against snapshot, verifies the signature against the authorized (possibly delegated) keys, and checks expiry. Delegation search respects path scoping and the terminating flag.
5. **Download and verify the target.** Only now does the client fetch the actual file. It verifies the file's length and hashes against the trusted targets metadata, and — crucially — it stops reading at the declared length, which is what makes the endless-data attack impossible.

The overarching logic of the order is **authority** → **freshness** → **consistency** → **content**. You establish *whose keys to trust* before

checking any signature; you check *whether the metadata is current* before trusting its contents; you pin *a consistent set* before selecting individual files; and you verify the *bytes* last, against metadata you have already fully validated. Reorder these and you open a hole — e.g., checking content before freshness would let a freeze attack feed you validly signed but stale targets.

Consistent snapshots

One implementation detail makes the whole thing survive concurrent publishing. With **consistent snapshots** enabled (the `consistent_snapshot` flag in root), metadata and targets are written under **version- or hash-prefixed filenames** — `3.snapshot.json`, `<hash>.targets.json`, `<hash>.django-4.2.tar.gz` — rather than being overwritten in place. A client fetching a set of metadata during a publish therefore never sees a half-updated repository: the timestamp it reads names specific prefixed filenames that are immutable once written. This turns "publish an update" into an atomic swap (write all new prefixed files, then write the new timestamp last) and removes a class of races that would otherwise let a client assemble an inconsistent view. Public deployments (PyPI's design, Sigstore's root) run with consistent snapshots on.

Key management, rotation, and recovery

Compromise resilience is only half the promise; the other half is **recovery**. A flat single-key model has none — once the key is out, your only move is to generate a new key and somehow deliver it, out of band, to every client, invalidating everything ever signed. TUF makes recovery a routine, *in-band* operation because the root role can re-delegate keys.

Root signing ceremonies. Because root is the anchor, establishing and updating it is done as a deliberate, auditable **ceremony**: the m-of-n root keyholders — ideally in different organizations, on different hardware tokens — each sign the new root metadata. Sigstore's root (below) does this in public, with recorded ceremonies and community keyholders, precisely so that the anchor's provenance is itself transparent. Offline hardware (HSMs, YubiKeys) and geographic and organizational distribution of the holders are what make reaching the threshold hard.

Rotating a role's keys. If a *snapshot* or *timestamp* key (or a targets/delegated key) is compromised or simply due for rotation, the fix is a **new root** that removes the old key ID and adds the new one for that role, signed by the root threshold. Clients pick this up through the normal root-update step (step 1 above): the next time they walk the root chain, they learn the old key is no longer authorized and the new one is. No client re-pinning, no flag day. This is why the online keys can be online: their compromise is *recoverable* by an offline action, and their limited power means the window before recovery is bounded to freeze/rollback rather than malware.

Rotating root itself. Root rotation is the double-signed chain from step 1: the new root must be signed by a threshold of the *old* root keys (proving the current holders authorize the change) and satisfy the *new* root's own threshold (proving the new keys are present and consistent). A client that trusts version N follows $N \rightarrow N+1 \rightarrow N+2$ verifying each hop against both, so it can move its anchor forward safely even across a complete change of root keyholders — as long as each step was blessed by the previous set.

The fast-forward wrinkle. Recall the fast-forward attack: an attacker with a stolen online key sets a metadata version to something enormous, so that after recovery the *legitimate* lower versions are rejected as rollbacks. TUF's answer is that **rotating the compromised keys via root resets the trust chain for that metadata** — when the role is re-keyed, clients begin trusting metadata signed by the *new* keys and the poisoned version number no longer binds them, allowing the version counter to be reset to a sane value under the new keys. The general principle: any damage an online key can do is undone by an offline re-key, which is exactly the recovery a flat model lacks.

The genuine limit. Recovery bottoms out at root. If a threshold of root keys is *lost* (holders gone, hardware destroyed) the repository cannot produce a new valid root and must re-bootstrap all clients out of band — the one scenario TUF cannot rescue you from in-band. And if a threshold of root keys is *stolen*, the attacker can re-delegate everything, which is why root is offline, thresholded, and multi-party. TUF does not claim invulnerability; it claims that the invulnerable- in-practice case (multi-party offline threshold) is the *only* path to total compromise, and every lesser compromise is bounded and recoverable.

TUF in the real world

TUF is a specification with several serious deployments. They differ in *how much* of the model they use and *who holds which keys*, and those differences matter.

PyPI — PEP 458 and PEP 480

Python packaging has two accepted/proposed PEPs built on TUF. **PEP 458** applies TUF to PyPI so that clients can verify the **integrity of downloaded distributions** against signed target hashes, with the repository (PyPI/Warehouse) operating the online snapshot and timestamp roles and holding the targets keys. Its threat model is primarily a **compromised mirror or CDN**: even if the file- serving layer is hostile, it cannot substitute or roll back a distribution without valid TUF metadata, and consistent snapshots prevent mix-and-match. Note the honest limit of PEP 458 alone: because PyPI holds the targets keys, a full compromise of PyPI's *signing* infrastructure is not defended by 458 — it protects the distribution channel, not against PyPI itself being subverted at the signing layer.

PEP 480 extends this with the "maximum security model": **maintainer (developer) keys**, using TUF **delegation** so that individual project maintainers sign their own releases with their own offline keys, and PyPI's online keys can no longer forge those projects' packages. That is genuine end-to-end signing, and it is the more ambitious, harder-to-operate model.

A candid status note, per this book's accuracy rules: **PEP 458 was accepted (2019) but has not been fully deployed in production on PyPI as of this writing, and PEP 480 remains a draft/deferred proposal**. In practice PyPI's integrity efforts have in recent years also moved toward Sigstore- based **digital attestations / provenance (PEP 740)** — a different mechanism than a full TUF rollout. Treat "PyPI runs TUF end-to-end" as aspirational, not deployed; the PEPs are the important *design* reference regardless of rollout state.

Notary v1 and Docker Content Trust

Notary v1 is a from-scratch implementation of TUF for signing content in container registries; **Docker Content Trust (DCT)** is the Docker CLI experience built on it (`DOCKER_CONTENT_TRUST=1`). Each repository gets its

own TUF trust hierarchy — a root key held by the publisher, plus targets, snapshot, and timestamp roles (with the registry-side Notary server holding the online timestamp key). It is a faithful TUF deployment and it *works*, but it saw **limited adoption**, largely because of the operational burden it pushed onto users: publishers had to generate and safeguard a per-repository root key locally, key loss meant losing the ability to sign that repository, and the delegation UX for teams was awkward. For most users the ceremony outweighed the perceived benefit.

Much of the container-signing world has since moved to **Sigstore/cosign** (Chapters 3–4), whose keyless model removes the user-held long-lived root key that made DCT painful. Note a naming trap: **Notary v2 / notation** (the OCI Notary Project) is a *different* design — X.509 signatures stored as OCI artifacts — and is **not** TUF-based. "Notary" alone is ambiguous; only Notary v1 is TUF.

Sigstore's TUF root — the root of trust for a root of trust

This is the deployment you have been depending on since Chapter 3, whether or not you knew it. A Sigstore client (cosign) verifies signatures by trusting Fulcio's root CA, Rekor's public key, the CT log's key, and the timestamp authority's key. **Those keys are distributed and rotated via TUF.** Sigstore runs a public TUF repository (served from a CDN, e.g. `tuf-repo-cdn.sigstore.dev`, and mirrored) whose targets are the *trust materials* — the Fulcio/Rekor/CT/TSA public keys and their validity metadata. The root role was established in a **public root-signing ceremony** with a threshold of community keyholders across multiple organizations, using **go-tuf** and the `sigstore/root-signing` repository. Cosign ships a pinned copy of this TUF root and refreshes it through the normal TUF update workflow (`cosign initialize` fetches the latest trust root).

```

flowchart TB
    subgraph tuf["Sigstore TUF repository (public, threshold-signed)"]
        SROOT["root.json"]
        community["keyholders, m-of-n, offline"]
        TARGETS["targets = TRUST MATERIALS"]
        Fulcio["root CA key, Rekor key, CT log key, TSA key (+ validity)"]
    end
    COSIGN["cosign / sigstore client ships pinned root, runs cosign initialize"]
    FULCIO["Fulcio (issues signing certs)"]
    REKOR["Rekor (transparency log)"]

    SROOT -->|"authorizes"| TARGETS
    TARGETS -->|"delivers verified keys via TUF workflow"| COSIGN
    COSIGN -->|"trusts cert chain to"| FULCIO
    COSIGN -->|"verifies inclusion against"| REKOR

```

The layering is worth stating plainly because it is easy to get tangled. Sigstore is a root of trust for *artifact signatures* (Chapter 3). But Sigstore's own trust anchors — the keys that make Fulcio and Rekor trustworthy — need their own secure distribution and rotation, and *that* is what TUF provides. **TUF is the root of trust for Sigstore's root of trust.** The properties it buys are exactly TUF's: a malicious CDN cannot substitute Fulcio's or Rekor's key without a valid threshold- signed TUF update; keys can be rotated (a Rekor key rollover, a Fulcio CA change) without re-pinning every client; and the whole thing survives compromise of the delivery channel. For a *private* Sigstore deployment (Chapter 9), you run your own TUF root and point your clients at it — same architecture, your keys. See Book 5, Chapter 3 — Sigstore Architecture, for how cosign consumes this root during verification, and Chapter 5 for how the Rekor key it distributes anchors log inclusion proofs.

Uptane — TUF for cars

Uptane is a TUF adaptation for **automotive over-the-air updates**, and it is a genuine standard in that industry. It extends TUF for the realities of a vehicle: two repositories — an **Image repository** (offline-signed, holds the firmware images and their targets metadata, the long-term trust) and a **Director repository** (online, decides which ECUs get which images and can respond to per-vehicle state) — with the vehicle verifying both. It adds concepts for **primary and secondary ECUs** and **partial**

verification for resource-constrained ECUs that cannot run the full metadata check, while a more capable primary ECU does full verification on their behalf. Uptane exists because the vehicle threat model is severe (an attacker on the update path could brick or weaponize cars) and because ECUs are heterogeneous and constrained — a good demonstration that TUF's role/threshold/freshness model generalizes beyond package registries.

Others and the implementations

RubyGems has explored TUF integration (research and prototypes from the same NYU lineage); treat production status as partial/uncertain rather than assuming a full rollout. The two reference implementations to know are **python-tuf** (the specification's reference implementation, with the modern **ngclient** update API) and **go-tuf** (used by Sigstore and historically related to the Notary lineage). When you build on TUF, you build on one of these rather than re-implementing the update algorithm — the ordering and edge cases (double-signed root chains, delegation search, consistent-snapshot filenames) are exactly the kind of thing you do not want to get subtly wrong.

Distributed-systems lens

TUF is, at bottom, a **compromise-resilient distributed-trust design**, and its ideas transfer directly to any system that distributes artifacts to many consumers — which describes most of the infrastructure a senior backend engineer owns.

- **Internal artifact distribution is a repository too.** If you run an internal registry, a language-package mirror, or a binary/model distribution service (Book 2, Chapter 8), it has the same threat surface as PyPI: high-value target, exposed CDN/storage, signing keys that live *somewhere*. The naive posture — one signing key, kept safe — has the same single-point-of-failure failure mode internally as externally. Adopting TUF (or its principles) makes an internal mirror compromise-resilient: a stolen mirror or a stolen online key cannot silently ship your fleet a malicious build.
- **Online/offline separation is a key-infrastructure design pattern.** The rule "the keys you use constantly are the keys whose theft must be survivable; the keys that grant real power are used

rarely and kept offline" is the exact discipline you want when you design signing infrastructure at scale (Chapter 9). TUF is a worked example of that pattern with a clean security proof attached.

- **Freshness and rollback protection matter for auto-updating fleets.** Anything that pulls updates automatically — agents, sidecars, node daemons, edge components (Book 1, Chapter 9; Book 6) — is vulnerable to freeze (an attacker pins your fleet to a vulnerable version by withholding updates) and rollback (downgrading you into a known CVE). TUF's timestamp/version machinery is the general-purpose answer: short-expiry freshness metadata plus monotonic versions turn "you silently stopped getting patches" from an invisible failure into a detectable one.
- **Thresholds and delegation are org-structure primitives.** Delegation maps to how large orgs actually work — each team owns and signs its own namespace, and a compromised team key is confined to that team's paths. Thresholds map to the reality that you should not let any one person or one machine be a total trust singularity for tier-0 distribution.
- **The philosophy is the takeaway even if you never run TUF.** "Assume the repository and some keys *will* be compromised; bound the damage; make what you can't prevent detectable; make recovery in-band." That is the mature security posture for tier-0 distribution infrastructure, and it is a strictly higher bar than "we signed it." Signed, as Chapter 3 kept insisting, is not the same as safe — and TUF is the framework that takes the next step: *safe even when the thing doing the signing is partly owned.*

Key takeaways

- TUF secures **software update/distribution systems**, not just individual artifacts. Its goal is **compromise resilience**: even if the repository infrastructure or *some* keys are compromised, an attacker cannot make you install arbitrary malware, and the attacks they *can* mount (freeze, rollback) are bounded and **detectable**.
- It defends a specific, enumerated attack set — arbitrary-package, rollback, freeze, fast-forward, mix-and-match, endless-data, slow-retrieval, and above all **key compromise** — with a specific mechanism for each. The design reads backwards from that threat list.

- There are **four top-level roles**: **root** (authorizes all keys/thresholds; offline anchor), **targets** (lists files by hash+length; delegates to sub-maintainers; offline), **snapshot** (commits one consistent set of metadata versions; defeats mix-and-match; online), and **timestamp** (points at the current snapshot; short expiry; defeats freeze/rollback; online). Do not confuse their duties.
- **Online/offline separation plus thresholds is the core insight.** The most-exposed keys (timestamp, snapshot) are the *least powerful* — their theft cannot forge new content, only freeze or roll back already-signed content, detectably. Forging malware requires the **offline targets** key (scoped by delegation) and, for total compromise, a **threshold of offline root keys** held by multiple people.
- The client update algorithm runs in a **mandatory order** — root → timestamp → snapshot → targets → download+verify — which is **authority → freshness → consistency → content**. Each step checks signatures against root-authorized keys, plus version (no rollback) and expiry (no freeze).
- The root role makes **rotation and recovery in-band**: a new root re-keys any compromised role and clients pick it up automatically; a flat single-key model has no equivalent. Recovery bottoms out only at loss/theft of a *threshold* of offline root keys.
- Real deployments: **PEP 458/480** for PyPI (accepted/proposed; not fully deployed — hedge), **Notary v1 / Docker Content Trust** (faithful but low-adoption; superseded by Sigstore for many), **Uptane** (automotive standard), and — most importantly for this book — **Sigstore's own TUF root**, which distributes and rotates the Fulcio/Rekor/CT/TSA keys your cosign verifications depend on. TUF is the **root of trust for Sigstore's root of trust**.

Further reading

- **The Update Framework specification** — the authoritative definition of the four roles, metadata formats, delegation, consistent snapshots, and the detailed client workflow. <https://theupdateframework.github.io/specification/latest/> and the project site <https://theupdateframework.io/>. Do not guess metadata field names; the spec is the authority.

- **"Survivable Key Compromise in Software Update Systems"** — Samuel, Mathewson, Cappos, Dingledine (ACM CCS 2010) — the foundational paper establishing role separation, thresholds, and the online/offline split. The threat enumeration in this chapter derives from it.
- **"A Look in the Mirror: Attacks on Package Managers"** — Cappos et al. (ACM CCS 2008) — the study of how real package managers mishandled repository compromise, which motivated TUF.
- **PEP 458** ("Secure PyPI downloads with signed repository metadata") and **PEP 480** ("Surviving a compromise of PyPI: maximum security model") — <https://peps.python.org/pep-0458/> and <https://peps.python.org/pep-0480/>. The target-hash model and the maintainer-key delegation model, respectively; read alongside the deployment-status caveat above.
- **Uptane standard** — <https://uptane.org/> — the automotive adaptation, with the Director/Image two-repository split and partial verification for constrained ECUs.
- **Sigstore root-signing** — `sigstore/root-signing` and **go-tuf** (<https://github.com/theupdateframework/go-tuf>); the public TUF root that distributes Fulcio/Rekor/CT/TSA keys (Book 5, Chapter 3). See how `cosign` consumes it via `cosign initialize`.
- **python-tuf** — the reference implementation and `ngclient` update API. <https://github.com/theupdateframework/python-tuf>. The place to read a correct implementation of the ordered update algorithm.
- **Notary v1 / Docker Content Trust** — <https://github.com/notaryproject/notary> (v1, TUF-based) — and note the contrast with **Notary v2 / notation**, which is *not* TUF-based.
- Cross-references: Book 5, Chapter 2 (single-key failure modes TUF avoids), Chapter 3 (Sigstore architecture and its TUF root), Chapter 5 (the Rekor key TUF distributes), Chapter 9 (key management, private trust roots, signing infra at scale); Book 2, Chapter 1 (dependency attack surface) and Chapter 8 (internal registries/

mirrors); Book 6, Chapter 2 (registries) and the fleet auto-update chapters; Book 1's key-compromise case studies.

Chapter 8 — Provenance Verification in Practice

What this chapter covers. The last five chapters built the *production* side of the trust machine: cryptographic primitives (Chapter 1), classic and keyless signing (Chapters 2–4), the transparency log that makes ephemeral keys auditable (Chapter 5), and the in-toto attestation framework that carries provenance, SBOMs, tests, and scans (Chapter 6). Producing all of that is necessary but, on its own, worth almost nothing. A signature that no one checks changes no attacker's calculus. Provenance that never gates a deploy is a log file with extra steps. The value of every mechanism in this book is realized at exactly one moment: when a **verifier** reads the metadata, compares it against a **policy**, and makes an **admit/deny** decision that stops bad code from running. This chapter is the practical, hands-on treatment of that moment — what to check, in what order, with which tools, where in the pipeline, and how to reason about the inevitable operational failure: what to do when the verifier itself cannot decide.

The single most important idea here is deceptively simple and constantly botched: **verifying that something is validly signed is nearly useless; you must verify that it was signed by the *expected* identity, and that the signed thing is *this* artifact.** Most of this chapter is spent making that distinction concrete and enforceable.

Learning goals — after this chapter you should be able to:

- Articulate the "**produce AND verify**" principle and name the six properties verification establishes: identity, integrity, provenance, policy compliance, transparency, and freshness/non-revocation.
- Execute the **layered verification checklist** — signature validity → *signer identity vs policy* → subject/digest match → provenance content vs policy → transparency-log inclusion → required-attestation set → freshness — and explain what each step defends against.
- Run real verification with `cosign verify`, `cosign verify-attestation`, and `slsa-verifier`, using the correct flags (--

`certificate-identity`, `--certificate-oidc-issuer`, `--type`, `--policy`, `--source-uri`, `--builder-id`) and knowing what each enforces.

- Write admission-time policy with **sigstore/policy-controller** (`ClusterImagePolicy`) and **Kyverno** (`verifyImages`) that requires keyless signatures from a specific identity *plus* named attestations, and understand why `verify-by-digest` matters.
- Place verification gates across the SDLC — CI, registry ingest, CD, and **admission as the enforcement backstop** — and recognize the trust boundary each gate defends.
- Diagnose the common failure modes (identity-blind verification, over-broad identity policy, sign-but-don't-verify, subject mismatch, TOCTOU, skipped transparency checks) and reason explicitly about **fail-open vs fail-closed**.

The verification mindset

Security in a signing system is not enforced where signatures are produced. It is enforced where they are *checked*. This is worth stating baldly because the industry's incentives push the other way: producing signatures is visible, demo-able, and satisfying — you run `cosign sign`, a green check appears, and the ticket closes. Verification is invisible when it works and a production outage when it's too strict, so it is chronically underinvested. The result is an ecosystem full of diligently signed artifacts that nothing on the consuming side ever inspects.

That gap is the attacker's dream. A pipeline that signs everything and verifies nothing has *more* attack surface than one that does neither, because it manufactures a false sense of assurance while providing no actual gate. The attacker does not need to forge a signature; they need only get their artifact into the flow at a point where no one is checking — and if nothing checks, every point qualifies. The recurring warning across this suite (Book 4, Chapter 3 on SLSA provenance; Book 3, Chapter 9 on operationalizing SBOMs) is the same: **an attestation you produce but never verify is security theater**. The metadata has to *gate* something.

So the mental model to internalize is a pipeline of two halves. The first half — everything in Chapters 1-6 — *emits* signed claims. The second half — this chapter — *consumes* them and converts them into a decision. Verification is where the following six properties actually get established:

Property	Question it answers	Established by
Identity	<i>Who or what produced this?</i>	Signature over an identity-bound certificate (Ch 4)
Integrity	Has it been modified since?	Digest binding, signature check (Ch 1)
Provenance	Built <i>how</i> , from <i>what source</i> ?	SLSA provenance predicate (Book 4, Ch 3)
Policy compliance	Does it meet <i>our</i> requirements?	Policy engine over the attestations (Ch 6)
Transparency	Is it publicly logged and auditable?	Rekor inclusion proof (Ch 5)
Freshness / non-revocation	Is the trust still valid <i>now</i> ?	Timestamps, revocation checks

Note that a bare signature check — the thing most people mean when they say "we verify our images" — establishes only the first two, and only partially. The rest require reading the *content* of attestations and comparing it against policy. That is the work.

The layered verification checklist

Verification is not one check; it is an ordered gate of them. Each layer answers a different question, and skipping any one leaves a specific, exploitable hole. Run them in this order — cheap cryptographic checks first, then identity, then content — and deny on the first failure.

```

flowchart TD
    A["Artifact + attestations  
referenced BY DIGEST"] --> B{"1. Signature  
cryptographically valid?"}
    B -->|no| DENY1["DENY"]
    B -->|yes| C{"2. Signer identity ==  
EXPECTED identity?  
(issuer + subject)"}
    C -->|no| DENY2["DENY"]
    C -->|yes| D{"3. Subject digest ==  
THIS artifact's digest?"}
    D -->|no| DENY3["DENY"]
    D -->|yes| E{"4. Provenance content  
meets policy?  
(source, builder, params)"}
    E -->|no| DENY4["DENY"]
    E -->|yes| F{"5. Rekor inclusion proof  
valid + timestamped?"}
    F -->|no| DENY5["DENY"]
    F -->|yes| G{"6. ALL required  
attestations present?  
(prov + SBOM + scan + tests)"}
    G -->|no| DENY6["DENY"]
    G -->|yes| H{"7. Fresh /  
not revoked?"}
    H -->|no| DENY7["DENY"]
    H -->|yes| ADMIT["ADMIT – cross trust boundary"]

```

1. Signature validity

The cheapest and most fundamental check: does the signature verify against the public key or certificate? For keyed signing this is a raw signature verification (Chapter 1); for keyless Sigstore signing it means the certificate chains to a trusted Fulcio root, the certificate's validity window covered the signing time (proven by the Rekor timestamp — Chapter 5), and the signature verifies against the certificate's embedded public key. This layer establishes that *some* private key controlled by *some* identity produced this signature. It says nothing about *which* identity. Passing this and stopping is the single most common verification mistake in the wild.

2. Signer identity versus policy — the layer everyone botches

A valid signature by *anyone* is worth essentially nothing. Sigstore's Fulcio will issue a signing certificate to *any* authenticated identity — your CI, my CI, an attacker's GitHub Actions workflow, a random Gmail account via

the browser OIDC flow. The transparency log will happily log all of them. So "this image is validly signed and in Rekor" is true of an image an attacker signed five minutes ago with their own throwaway identity. The signature is real. The identity is wrong.

Verification must therefore assert the **expected identity**, not merely *an* identity:

- For **keyless** signing: the certificate's **OIDC issuer** must be the expected issuer (e.g., `https://token.actions.githubusercontent.com` for GitHub Actions), *and* the certificate's **subject / SAN** must be the expected workload — the specific repository, workflow file, and ref (Chapter 4). "Signed by a GitHub Actions workflow" is not enough; it must be *our* workflow at *our* ref.
- For **keyed / KMS** signing: the signature must verify against the *specific* public key you trust, and that key's identity in your KMS must be the expected one.

```
flowchart LR
    subgraph bad["Identity-BLIND verification (WRONG)"]
        A1["cosign verify img"] --> A2["Valid signature by anyone?"]
        A2 -->|yes| A3["ADMIT x attacker's throwaway identity accepted"]
    end
    subgraph good["Identity-PINNED verification (RIGHT)"]
        B1["cosign verify img --certificate-identity=... --certificate-oidc-issuer=..."] --> B2["Signed by the EXPECTED workflow at the EXPECTED issuer?"]
        B2 -->|no| B3["DENY ✓"]
        B2 -->|yes| B4["ADMIT ✓"]
    end
```

This is the load-bearing layer. Everything else is defense in depth around it. If you take one operational lesson from this chapter: **pin the identity**.

3. Subject / digest match

An attestation or signature is *about* a specific artifact, identified by cryptographic digest (content addressing — Chapter 1). Verification must confirm the metadata you validated actually describes the artifact you are about to run. Two failure modes hide here. First, an attestation for image

A can be presented alongside image B — if you validate the attestation's signature and identity but never check that its `subject.digest` equals B's digest, you've verified a claim about a *different* artifact. Second, the whole verification must be **anchored to a digest, not a mutable tag**. `registry.example.com/app:latest` is a pointer that can be repointed after you check it. `registry.example.com/app@sha256:abcd...` is immutable. Verify the digest; deploy the digest.

4. Provenance content versus policy

Now we read the *inside* of the provenance predicate (Book 4, Chapter 3) and compare it to policy. A signature proves who signed; the provenance predicate says *how the artifact was built*, and policy decides whether that's acceptable:

- **Source:** built from the expected repository and ref — e.g., `main`, not an attacker's fork or feature branch.
- **Builder:** produced by a trusted builder ID — your hosted CI, the SLSA GitHub generator, your internal build platform — not an unknown runner.
- **Build parameters:** meets policy — e.g., SLSA Build Level 3, hermetic build, no injected parameters, expected entry point.

This is where "signed by our CI" becomes "built from `main` of the right repo, on our L3 builder, with the expected workflow." A compromised-but-legitimate identity (say, a workflow that an attacker triggered from a malicious branch) can pass the identity check and still be caught here if your policy pins the ref.

5. Transparency-log inclusion

The Rekor entry must exist and carry a valid **inclusion proof** and a signed **timestamp** (Chapter 5). This does two things. It makes the signing event **auditable** — anyone monitoring the log can detect a signature by an unexpected identity. And it enables **verify-after-expiry**: the Rekor timestamp proves the short-lived Fulcio certificate was valid *at signing time*, so you can verify the signature years later even though the certificate expired minutes after issuance. Skip this and you lose both auditability and the ability to trust anything past the ~10-minute certificate lifetime. Cosign checks the log by default; disabling it (`--insecure-ignore-tlog`) is a deliberate downgrade.

6. Required-attestation set

Policy rarely wants *one* attestation. A mature gate requires a **set**: SLSA provenance *and* an SBOM (Book 3) *and* a passing vulnerability scan *and* test results (Chapter 6). Verification must check that *all* required predicate types are present, each individually valid and identity-pinned. A missing SBOM is a policy failure, not a warning. This turns "is it signed?" into "does it satisfy our full supply-chain contract?"

7. Freshness and revocation

Where applicable: is the trust still current? A vulnerability-scan attestation from six months ago may no longer reflect known CVEs; policy may require a scan attestation no older than N days. If you run your own PKI (Chapter 9) rather than ephemeral keys, revocation matters — a compromised signing key's artifacts must stop verifying. With keyless Sigstore the ephemeral-key model sidesteps classic revocation, but freshness of *content* attestations remains a live concern.

Tools and concrete commands

cosign verify — signature + identity + transparency, in one command

For keyless verification, cosign 2.0 and later **require** you to pin the identity — the flags are not optional, precisely because identity-blind verification was such a common footgun in the 1.x era:

```
cosign verify \  
  --certificate-identity "https://github.com/myorg/myrepo/.github/  
workflows/release.yml@refs/heads/main" \  
  --certificate-oidc-issuer "https://  
token.actions.githubusercontent.com" \  
  registry.example.com/app@sha256:3f7a...c19d
```

Note the reference is a **digest**, not a tag. What each part enforces:

- `--certificate-identity` — the exact SAN in the Fulcio cert; here, the specific workflow file at a specific ref. This is checklist layer 2. Use `--certificate-identity-regex` when you must match a family of identities, but keep the regex tight (see the over-broad footgun below).

- `--certificate-oidc-issuer` — the OIDC issuer that minted the token; pins the *source* of the identity. An attacker signing via Google OAuth cannot satisfy an issuer pinned to GitHub Actions.
- By default, cosign also verifies the **Rekor** inclusion proof (layer 5). Add `--rekor-url` for a private log; `--offline` verifies from a bundle without contacting Rekor live.

For **keyed** verification, swap identity flags for a key:

```
cosign verify --key cosign.pub registry.example.com/
app@sha256:3f7a...c19d
# or a KMS reference:
cosign verify --key aws kms:///alias/prod-signing registry.example.com/
app@sha256:3f7a...c19d
```

A successful run prints the verified DSSE/signature payloads (trimmed):

```
Verification for registry.example.com/app@sha256:3f7a...c19d --
The following checks were performed on each of these signatures:
- The cosign claims were validated
- Existence of the claims in the transparency log was verified offline
- The code-signing certificate was verified using trusted
certificate authority certificates
Certificate subject: https://github.com/myorg/myrepo/.github/workflows/
release.yml@refs/heads/main
Certificate issuer URL: https://token.actions.githubusercontent.com
```

cosign verify-attestation — checking the *content* of attestations

`verify-attestation` does everything `verify` does (signature + identity + Rekor) *plus* selects a predicate type and evaluates a **policy** over its content — checklist layers 4 and 6 in one shot:

```
cosign verify-attestation \
--type slsaprovenance \
--certificate-identity-regexp "^https://github.com/myorg/" \
--certificate-oidc-issuer "https://
token.actions.githubusercontent.com" \
--policy provenance-policy.cue \
registry.example.com/app@sha256:3f7a...c19d
```

- `--type` selects the predicate. Cosign understands shorthands: `slsaprovenance` (SLSA v0.2), `slsaprovenance1` (SLSA v1.0),

`spdxjson`, `cyclonedx`, `vuln`, `link`, or a full predicate-type URI for custom types.

- `--policy` points at a **CUE** or **Rego** file evaluated against the predicate. This is where "built from `main`" becomes an enforced assertion rather than a hope.

A minimal CUE policy pinning the source and builder for a SLSA v0.2 provenance predicate:

```
// provenance-policy.cue
predicateType: "https://slsa.dev/provenance/v0.2"

predicate: {
  builder: id: =~"^https://github.com/myorg/"
  invocation: configSource: {
    uri: =~"^git\\+https://github.com/myorg/myrepo@refs/heads/
main$"
  }
}
```

If the provenance says it was built from a fork, a feature branch, or an unexpected builder, CUE evaluation fails and cosign exits non-zero. That non-zero exit is the entire point: it is what a CI step or admission controller gates on.

slsa-verifier — purpose-built SLSA provenance verification

`slsa-verifier` (from the SLSA framework) is a focused tool that verifies SLSA provenance against a source and builder, doing the source/builder/ref checks (layer 4) with SLSA-aware defaults. For a released binary with detached provenance:

```
slsa-verifier verify-artifact app-linux-amd64 \
--provenance-path app-linux-amd64.intoto.jsonl \
--source-uri github.com/myorg/myrepo \
--source-tag v1.4.2 \
--builder-id "https://github.com/slsa-framework/slsa-github-
generator/.github/workflows/generator_generic_slsa3.yml@refs/tags/
v2.0.0"
```

For a container image (provenance stored in the registry):

```
slsa-verifier verify-image "registry.example.com/
app@sha256:3f7a...c19d" \
  --source-uri github.com/myorg/myrepo \
  --builder-id "https://github.com/slsa-framework/slsa-github-
generator/.github/workflows/builder_container-based_slsa3.yml@refs/
tags/v2.0.0"
```

What it checks: the provenance signature and its transparency-log entry, that the `--source-uri` **matches** the repository recorded in the provenance, that the `--builder-id` **matches** the builder that produced it (this is the trust anchor — you are asserting *which* builder you trust), and, when supplied, that `--source-tag` / `--source-branch` match. It rejects on any mismatch. The crucial mental model: `--builder-id` and `--source-uri` are *your policy inputs*. Passing them is how you say "I only trust artifacts built by *this* builder from *this* repo." Omitting the pins, or accepting any builder, reduces the tool to a signature check.

Policy engines for attestations at admission

The commands above are what you run in a pipeline step. At the runtime platform, verification is expressed as **policy** enforced by an admission controller. Two dominant approaches on Kubernetes:

sigstore/policy-controller with a `ClusterImagePolicy` — this is the Sigstore-native admission controller. It requires matching images to be signed by a pinned keyless identity *and* to carry named attestations, evaluating a CUE/Rego policy over each:

```

apiVersion: policy.sigstore.dev/v1beta1
kind: ClusterImagePolicy
metadata:
  name: require-prov-and-sbom
spec:
  images:
    - glob: "registry.example.com/**"
  authorities:
    - keyless:
        url: https://fulcio.sigstore.dev
      identities:
        - issuer: https://token.actions.githubusercontent.com
          subjectRegExp: "^https://github.com/myorg/[^/]+/.github/workflows/release.yml@refs/heads/main$"
      ctlog:
        url: https://rekor.sigstore.dev
      attestations:
        - name: must-have-slsa-provenance
          predicateType: https://slsa.dev/provenance/v1
          policy:
            type: cue
            data: |

predicate: buildDefinition: externalParameters: workflow: ref: "refs/heads/main"
  - name: must-have-sbom
    predicateType: https://spdx.dev/Document

```

This one policy asserts identity (`issuer` + `subjectRegExp`), transparency (`ctlog`), a provenance *content* check (the CUE block), *and* the presence of a second required attestation (the SBOM). A pod referencing a non-conforming image is rejected at admission.

Kyverno with a `verifyImages` rule — Kyverno is a general-purpose policy engine whose image verification does the same job with an important extra: it can **rewrite the tag to a digest** at admission, closing the TOCTOU gap:

```

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: verify-image-and-attestations
spec:
  validationFailureAction: Enforce
  webhookTimeoutSeconds: 30
  rules:
    - name: verify-signature-and-provenance
      match:
        any:
          - resources:
              kinds: ["Pod"]
            verifyImages:
              - imageReferences:
                  - "registry.example.com/*"
                  mutateDigest: true      # resolve tag -> digest, pin it into
the Pod spec
                  verifyDigest: true
                  required: true
                  attestors:
                    - count: 1
                      entries:
                        - keyless:
                            subject: "https://github.com/myorg/myrepo/.github/
workflows/release.yml@refs/heads/main"
                            issuer: "https://
token.actions.githubusercontent.com"
                            rekor:
                                url: https://rekor.sigstore.dev
                          attestations:
                            - type: https://slsa.dev/provenance/v1
                              attestors:
                                - entries:
                                    - keyless:
                                        subject: "https://github.com/myorg/
myrepo/.github/workflows/release.yml@refs/heads/main"
                                        issuer: "https://
token.actions.githubusercontent.com"
                                    conditions:
                                      - all:
                                          - key: "{{ buildDefinition.externalParameters.workf
low.ref }}"
                                          operator: Equals
                                          value: "refs/heads/main"

```

`mutateDigest: true` is doing security work, not convenience: Kyverno resolves the tag to a digest, verifies against *that* digest, and pins the digest into the admitted Pod spec, so the kubelet pulls exactly what was

verified. `validationFailureAction: Enforce` makes this a hard gate rather than an audit log. The `attestations.conditions` block is the content check (layer 4) expressed in Kyverno's JMESPath.

in-toto / Witness verification (Chapter 6) rounds out the toolset for full-pipeline layout verification — `in-toto-verify` against a signed layout, or `witness verify` against a policy — when you need to assert that an entire chain of steps ran in order and connected by hash, not just that a final artifact is signed.

Where verification happens: defense in depth

Verification is not a single gate; it is a series of gates along the path an artifact travels from "untrusted/external" to "trusted/will-run." Each gate defends a **trust boundary** — the point where an artifact crosses from a lower-trust zone to a higher-trust one. The rule of thumb: **verify at every boundary crossing, and treat admission as the non-negotiable backstop.**

```
flowchart LR
    ext["External deps  
(untrusted)"] -->|"verify provenance  
before use (aspirational)"| ci["CI build"]
    ci -->|"cosign verify /  
slsa-verifier on ingest"| reg["Internal registry  
(Book 2, Ch 8)"]
    reg -->|"verify before promote"| cd["CD pipeline"]
    cd -->|"verify by digest  
before deploy"| adm["Admission controller  
(Book 6, Ch 5-6)"]
    adm -->|"ENFORCE: identity +  
provenance + attestations"| run["Pod runs"]
    adm -.->|"deny"| reject["Rejected"]
    style adm fill:#c0392b,color:#fff
    style run fill:#27ae60,color:#fff
```

- **At pull / in CI** — verify the provenance of *dependencies* before consuming them (e.g., `slsa-verifier` on a downloaded release, `cosign verify` on a base image). This is the most aspirational gate because much of the ecosystem still lacks provenance, but where it exists it stops a poisoned dependency at the door.
- **At the internal registry (ingest)** — when an artifact enters your controlled registry (Book 2, Chapter 8), verify it once at the boundary

between "built" and "storable." This is a good place to attach your own re-attestation.

- **In the CD pipeline (before deploy)** — the promotion gate. Verify by digest before the pipeline hands the artifact to the platform. Pair with a Verification Summary Attestation (VSA, Chapter 6) so downstream stages can trust a cheap signed summary instead of re-verifying deeply.
- **At admission in the runtime platform — the backstop.** This is the most important gate because it is the *last* one and the one closest to execution. Kubernetes admission (Book 6, Chapters 5-6; and deployment gates in Chapter 10) verifies image signatures, identity, and attestations *before a pod is allowed to run*. Every other gate can be bypassed by someone who deploys out-of-band — `kubect`l apply straight to the cluster, a rogue GitOps commit, a compromised CD system. Admission cannot be bypassed by anything short of compromising the control plane itself, because *nothing runs* without passing through it. That property — unavoidable, closest to execution — is what makes admission the enforcement backstop that justifies all the upstream signing.

Multiple gates are not redundant; they are defense in depth. Upstream gates give fast feedback and catch problems early and cheaply; admission guarantees that *nothing* runs unverified regardless of how it got there.

Common failure modes and pitfalls

These are the ways real verification setups fail. Every one has been observed in production supply-chain programs.

1. Verifying validity but not identity. The classic mistake, restated because it dominates: accepting anything validly signed by anyone. A pipeline that runs `cosign verify --key cosign.pub` against a shared org key, or keyless `cosign verify` without identity flags (impossible in cosign 2.0+, which is *why* the flags became mandatory), is verifying that *a* signature exists — not that *your* signer made it. The fix is layer 2: always pin `--certificate-identity(-regexp)` and `--certificate-oidc-issuer`, or the specific key.

2. Over-broad identity policy. The subtler cousin: you pin identity, but too loosely. `--certificate-identity-regexp "https://github.com/myorg/.*"` accepts *any repo in the org* and *any workflow* — including a

low-security sandbox repo an attacker can push to, or a non-release workflow. The identity footgun from Chapter 4 lives here. Scope the regex to the specific repository, the specific workflow file, and the specific ref. Anchor it (`^...$`). Every character you leave as `.*` is trust you're extending to someone.

3. Sign but don't verify. The organizational failure: teams stand up `cosign sign` and SLSA provenance generation, celebrate the coverage metric, and never build the consuming gate. The attestations pile up in the registry, verified by nothing. This is the Book 4, Chapter 3 warning made concrete — provenance that gates no decision protects nothing. The remedy is not more production; it is a *gate*, ideally at admission, that fails closed on missing or non-conforming attestations.

4. Not checking subject/digest match. Verifying an attestation whose signature and identity are valid but whose `subject.digest` describes a *different* artifact. An attacker who can present a legitimately-signed attestation for image A alongside malicious image B slips through if you never bind the check to B's digest. Tools like `cosign verify-attestation` bind subject to the image reference automatically *when you pass a digest* — which is another reason to always reference by digest.

5. TOCTOU — time-of-check to time-of-use. You verify `app:latest`, resolve it to digest X, and approve — then between check and deploy the tag is repointed to malicious digest Y, and the platform pulls Y. The defect is verifying a *mutable* reference. The fix is end-to-end digest pinning: verify by digest, record the digest, deploy *that* digest, and have admission re-verify the digest it's actually about to run. Kyverno's `mutateDigest:true` operationalizes this by pinning the verified digest into the admitted spec (ties to Book 6, Chapter 1 on image references).

6. Ignoring transparency-log verification. Passing `--insecure-ignore-tlog`, or accepting signatures from expired certificates without a Rekor timestamp to prove signing-time validity. This silently discards auditability *and* breaks verify-after-expiry — the keyless model depends on the log to make ephemeral certificates verifiable later (Chapters 3 and 5). Don't disable it outside of tightly-scoped air-gapped cases with a private log.

7. Availability coupling on the verifier chain. If admission verification calls out to Fulcio's root, Rekor, or an OIDC provider synchronously, an outage in any of those (Chapters 5 and 9) can stall or fail your deploys.

This is not a reason to skip verification; it is a reason to design the verifier, log, and policy engine as tier-1 infrastructure and to make an explicit fail-open / fail-closed decision — which is the next section.

Fail-open vs fail-closed

Eventually the verifier itself cannot decide. Rekor is unreachable, the policy engine's webhook times out, the Fulcio root can't be fetched, the OIDC issuer is down. The gate faces a binary choice with no good universal answer: **fail-open** (admit despite being unable to verify) or **fail-closed** (deny). This is the security-versus-availability tension in its rawest form, and pretending it doesn't exist is how you get either a self-inflicted outage or a gate that quietly stops gating.

```
flowchart TD
    A["Verification cannot  
complete (log down,  
webhook timeout, root  
unreachable)"] --> B["Workload  
sensitivity?"]
    B -->|High: prod, PCI,  
internet-facing| C["FAIL CLOSED  
deny admission"]
    B -->|Low: dev, sandbox,  
batch| D["FAIL OPEN  
admit + alert"]
    C --> E["Availability risk:  
verifier is now on the  
critical path – must be  
tier-1, HA, monitored"]
    D --> F["Security risk:  
unverified code can run  
during the outage window"]
    E --> G["Mitigate: cache trust  
roots, staged rollout,  
break-glass with audit"]
    F --> G
```

The reasoning framework:

- **Match the mode to the blast radius.** For high-sensitivity workloads — production, regulated environments, internet-facing services — **fail closed**. The cost of running one unverified artifact exceeds the cost of a deploy freeze during an outage. For low-

sensitivity paths — developer sandboxes, batch jobs behind other controls — fail-open-and-alert may be acceptable.

- **Fail-closed makes the verifier a hard dependency**, so you must earn it: run the policy engine and log as **tier-1 HA infrastructure**, cache trust roots and (where possible) inclusion proofs locally, use bundle-based **offline verification** (`cosign verify --offline`) so admission does not call Rekor synchronously on the hot path, and monitor the verifier's own availability as a first-class SLO.
- **Roll out staged**. Introduce a new fail-closed policy in **audit/warn** mode first (`validationFailureAction: Audit` in Kyverno; dry-run in policy-controller), watch what it *would* have denied, fix the false positives, and only then flip to `Enforce`. Flipping straight to fail-closed across a fleet is how you take down every deploy at once.
- **Provide break-glass, with audit**. A documented, logged, time-boxed override for genuine emergencies — but one that generates a loud, auditable trail so it isn't the default path.

The honest summary: fail-closed is correct for anything that matters, *and* it obligates you to operate the verification chain as critical infrastructure. You cannot have a meaningful backstop that is also allowed to be flaky.

Distributed-systems lens

At the scale this suite assumes — hundreds of services, dozens of teams, thousands of deploys a day — verification stops being a command an engineer runs and becomes a property of the platform. Several consequences follow.

Verification must be automated and enforced at gates, uniformly.

No human runs `cosign verify` before a deploy at this scale; the paved-road deploy path and the admission controllers (Book 6) do it on *every* artifact, every time, without exception. The moment verification is optional or manual, it is skipped under deadline pressure, and the gate that is skipped 1% of the time is the gate an attacker targets. Uniform automated enforcement is the only kind that holds.

Policy is centralized; enforcement is distributed. *What* identities are trusted, *which* builders are acceptable, and *which* attestations are required belong in one place — central policy, managed as code (Book 8, Chapter 4 on policy-as-code; Book 8, Chapter 4 also frames continuous compliance). That central policy is then *applied* at every admission

controller across every cluster. A single change ("we now require an SBOM attestation on all prod images") propagates to the whole fleet without touching individual services. The `ClusterImagePolicy` / `Kyverno ClusterPolicy` resources are the distribution mechanism.

Verify by digest, end to end. Immutable references are the distributed-systems answer to swap attacks. When the CI output, the registry entry, the CD promotion, and the admission decision all name the *same digest*, there is no seam for a TOCTOU substitution. Tags are for humans; the machine path is digests all the way down.

The verifier, policy engine, and transparency log are tier-1 infrastructure. If admission is the backstop that makes all upstream signing worthwhile — and it is — then the components that admission depends on inherit production-grade availability requirements. A flaky policy engine is not a minor annoyance; it is either a deploy outage (fail-closed) or a silent security hole (fail-open). Treat it like the load balancer: HA, monitored, on-call.

VSA lets you verify centrally and delegate the summary. Re-verifying deep provenance and a full attestation set at every gate for every artifact is expensive in latency and in load on Rekor and Fulcio. The Verification Summary Attestation (Chapter 6) is the scaling lever: a trusted central verifier does the deep check once, emits a signed VSA recording the result and the levels achieved, and downstream gates verify the *summary* — one cheap signature check — instead of redoing the work. This is the same delegated-trust pattern that shows up throughout distributed systems: verify once at a trusted authority, propagate a compact signed assertion, and let consumers check the assertion rather than re-deriving the fact.

The through-line: producing signatures and provenance is the easy, visible half. Verification — automated, identity-pinned, digest-anchored, enforced at admission, failing closed for anything that matters — is the half that actually stops attacks. Everything in Chapters 1–6 is potential energy. The gate in this chapter is where it converts to work.

Key takeaways

- **Produce AND verify.** Signatures, provenance, and SBOMs that nothing checks are security theater. Security is enforced at the *verifier*, not the producer. A system that signs diligently and verifies

nothing has *negative* value — false assurance over a real gap. This warning recurs across the suite (Book 4, Ch 3; Book 3, Ch 9) because the mistake is endemic.

- **Valid $\neq$ trusted.** A cryptographically valid signature by *anyone* is nearly worthless; Fulcio and Rekor will happily issue and log a signature for an attacker's throwaway identity. You must verify the **expected identity** — issuer *and* subject (workflow, repo, ref) for keyless; the specific key for keyed. **Pin the identity.** This is the most-botched step and the one that matters most.
- **Verification is a layered checklist:** signature validity $\rightarrow$ identity vs policy $\rightarrow$ subject/digest match $\rightarrow$ provenance content vs policy (source, builder, params) $\rightarrow$ Rekor inclusion $\rightarrow$ required attestation set $\rightarrow$ freshness. Deny on the first failure. Each layer closes a specific hole.
- **Use the real tools with the real flags.** `cosign verify --certificate-identity --certificate-oidc-issuer ; cosign verify-attestation --type --policy ; slsa-verifier verify-artifact/verify-image --source-uri --builder-id`. In cosign 2.0+ the keyless identity flags are *mandatory* — a deliberate fix for identity-blind verification.
- **Enforce at admission as the backstop.** Verify at CI, at registry ingest, and in CD for fast feedback — but Kubernetes admission (policy-controller `ClusterImagePolicy`, Kyverno `verifyImages`) is the unavoidable, last-line gate that guarantees nothing runs unverified regardless of how it was deployed. Use `mutateDigest` /digest pinning to defeat TOCTOU.
- **Verify by digest, end to end.** Mutable tags enable swap attacks; content-addressed digests close the seam. Verify the digest, record it, deploy it, re-verify it at admission.
- **Decide fail-open vs fail-closed deliberately.** Fail closed for anything that matters, and pay the price: run the verifier, policy engine, and log as tier-1 HA infrastructure; use offline bundle verification off the hot path; roll out in audit mode before enforcing; provide audited break-glass. Fail-open is only for low-sensitivity paths.
- **At fleet scale:** centralize policy (what's trusted/required), distribute enforcement (every admission controller applies it), and use **VSA** to

verify deeply once and propagate a cheap signed summary — managing latency and load without weakening the gate.

Further reading

- **Cosign documentation** — `cosign verify` and `cosign verify-attestation`, the certificate identity/issuer flags, predicate `--type` values, and CUE/Rego `--policy`. <https://docs.sigstore.dev/> and <https://github.com/sigstore/cosign>. The cosign 2.0 release notes document the shift to *mandatory* keyless identity flags — read them for the rationale.
- **slsa-verifier** — `verify-artifact`, `verify-image`, `verify-npm-package`, and the `--source-uri / --builder-id / --source-tag` semantics. <https://github.com/slsa-framework/slsa-verifier>. The authority on what SLSA provenance verification actually checks (developed in Book 4, Chapter 3).
- **sigstore/policy-controller** — the `ClusterImagePolicy` CRD: `authorities`, `keyless.identities` (issuer/subject regex), `ctlog`, and `attestations` with CUE/Rego policy blocks. <https://docs.sigstore.dev/policy-controller/overview/> and <https://github.com/sigstore/policy-controller>.
- **Kyverno image verification** — the `verifyImages` rule, `attestors / attestations`, `mutateDigest`, and `verifyDigest`. <https://kyverno.io/docs/policy-types/cluster-policy/verify-images/>. Applied at admission in Book 6, Chapters 5–6.
- **in-toto Attestation Framework** — the Statement/predicate model whose content these tools verify; the predicate-type registry. <https://github.com/in-toto/attestation> (Chapter 6).
- **SLSA v1.0 — Verifying artifacts / Verification Summary**
Attestation — the provenance and VSA specs, and the delegated-verification model. <https://slsa.dev/spec/v1.0/verifying-artifacts> and https://slsa.dev/verification_summary/v1.
- **Rekor** — inclusion proofs, signed timestamps, and offline/bundle verification enabling `verify-after-expiry`. <https://docs.sigstore.dev/logging/overview/> (mechanics in Chapter 5).
- Cross-references: Book 5, Chapter 1 (crypto, digests, DSSE), Chapters 3–5 (Sigstore, keyless identity, Rekor), Chapter 6 (in-toto attestations, VSA), Chapter 10 (attestation-based deployment gates);

Book 4, Chapter 3 (SLSA provenance); Book 2, Chapter 8 (internal registries); Book 6, Chapters 1, 5–6 (image references, image signing in Kubernetes, admission and policy engines); Book 8, Chapter 4 (policy as code and continuous compliance).

Chapter 9 — Key Management and PKI for the Enterprise

What this chapter covers. Chapter 4 spent a chapter dismantling the word "keyless," and its conclusion was sobering rather than triumphant: keyless signing removes the *long-lived, human-held* key, but it does not remove keys from the world. Someone runs Fulcio's certificate authority, and Fulcio has a signing key. Someone holds the TUF root keys that anchor Sigstore's own trust (Chapter 7). Someone provisions the KMS keys behind an internal CA, and someone signs release binaries with a high-assurance key that no OIDC token will ever stand in for. The enterprise reality is that even a fully keyless artifact pipeline sits on a foundation of real keys and real public-key infrastructure — for internal CAs, mTLS between services (Book 9), an internal Fulcio, a private Rekor, and the crown-jewel release and root keys that must survive audits and adversaries. This chapter is the practical, operational treatment of that foundation: how to store keys so they are hard to steal, how to build and run internal PKI, how to move the whole fleet onto short-lived certificates so revocation stops mattering, how to run the key lifecycle as a disciplined operation rather than a folder of `.pem` files, and how to stand up a self-hosted Sigstore that meets the same tier-1 availability bar as the rest of your platform.

Learning goals — after this chapter you should be able to:

- Place any key-storage option on the **extraction-cost / assurance spectrum** — software file → passphrase-encrypted → KMS → HSM — and justify the enterprise default: *sign via an API; never hold the key material*.
- Use **cloud KMS** (AWS KMS, GCP Cloud KMS, Azure Key Vault) and **HSMs** as signing backends, including the real `cosign` KMS URI schemes (`aws:kms://`, `gcpkms://`, `azurekms://`, `hashivault://`) and PKCS#11 for hardware.

- Design an **enterprise PKI hierarchy** with an offline HSM-backed root and online issuing intermediates — the same online/offline split TUF uses — and pick the right tool (Vault PKI, step-ca, cfssl, AWS Private CA, GCP CAS, EJBCA/Dogtag).
- Explain why **short-lived certificates + automation** (ACME, cert-manager, SPIRE) beat long-lived certificates plus revocation, and how they eliminate both expired-cert outages and the revocation problem Chapter 2 dissected.
- Run the **key lifecycle** — generation, distribution, rotation (without breaking verification), revocation, destruction, and audit — and run a **root-key ceremony** with m-of-n threshold, offline material, and witnesses (the TUF model, Chapter 7).
- Choose between and combine the two org-scale signing architectures: a **centralized KMS-backed signing service** and **keyless workload-identity signing** — and stand up **self-hosted Sigstore** (private Fulcio/Rekor/TUF/TSA) as tier-1 infrastructure.

Why key management is the hard part

Chapter 2 built an entire failure taxonomy for classic code signing, and its dominant entry was **key theft**: Stuxnet's operators signed with driver-signing keys stolen from Realtek and JMicron; the 2022 NVIDIA breach leaked code-signing certificates that were promptly reused to sign malware. In every one of those incidents the cryptography was flawless. What failed was *custody* — the operational problem of keeping a durable secret secret for the one-to-three-year window during which it is, cryptographically, the identity.

Chapter 3 and Chapter 4 showed keyless signing shrinking that window from years to seconds. But "shrink" is not "eliminate," and the residue is exactly the set of keys an enterprise cannot make ephemeral:

- **Fulcio's CA key.** Every keyless certificate is signed by Fulcio's intermediate, which chains to a root. Those are long-lived, high-value keys. If you self-host Fulcio, they are *your* keys.
- **The TUF root keys** that anchor whichever Sigstore trust root your clients use (Chapter 7). Threshold-held, offline, ceremony-rotated — the most valuable keys in the system.
- **Rekor's signing key**, which signs checkpoints and Signed Entry Timestamps (Chapter 5).

- **Release and high-assurance keys** where a regulator, a customer contract, or an OS root program requires a specific, auditable, hardware-bound signer that OIDC identity cannot satisfy.
- **The mTLS/SPIFFE PKI** that every service in the fleet uses to authenticate every other service (Book 9) — thousands of certificates, issued and rotated continuously.

Key management is where cryptography meets operations, and operations is where it breaks. The math never fails; the folder of unrotated `.pem` files, the CI secret copied into a Slack thread, the intermediate that expired at 2 a.m. and took the fleet down — those fail. The goal of everything in this chapter can be stated in one sentence: **keys that are hard to steal, easy to rotate, and whose every use is controlled and audited.** Those three properties pull in different directions — a key locked in an offline HSM is hard to steal and hard to rotate — and the craft of enterprise key management is resolving that tension deliberately for each *class* of key rather than treating all keys the same.

Key storage and protection: the spectrum

Every private key lives somewhere, and where it lives sets a single dominant property: the **cost to an attacker of extracting the key material**. Think of storage options as a spectrum from "trivially copyable file" to "non-exportable hardware," and read the whole spectrum as a campaign to raise that extraction cost until it exceeds the value of the key.


```

flowchart LR
    F["Software key  
(.pem on disk)  
extraction: copy a file"]
    E["Passphrase-encrypted  
key file  
extraction: file + guess/keylog"]
    K["Cloud KMS  
(sign via API)  
extraction: breach the KMS boundary"]
    H["HSM / CloudHSM  
(non-exportable)  
extraction: defeat FIPS hardware"]
    F -->|"raise extraction cost"| E -->|"cannot exfiltrate the key,  
only its use"| K -->|"key never exists  
outside hardware"| H
    classDef bad fill:#fdd,stroke:#900
    classDef ok fill:#dfd,stroke:#090
    class F,E bad
    class K,H ok

```

Software keys — the worst option

A private key in a file is a private key one `cat`, one misconfigured S3 bucket, one leaked CI log, or one `docker history` away from being someone else's. Book 4, Chapter 6 (secrets in CI) is a catalog of the ways this file escapes: it gets committed, printed to a build log, baked into an image layer, exfiltrated by a poisoned dependency in the build (event-stream, Codecov). The key is **exportable by construction** — possession is the private key — so the extraction cost is "obtain read access to one file," which in a large organization is not a high bar. Software keys are acceptable only for throwaway, low-value, ephemeral material (the ephemeral keypair of keyless signing lives in memory as a file-like object for seconds — that is fine precisely because it is worthless after use).

Passphrase-encrypted keys — marginal

Wrapping the key file in a passphrase (Cosign's default `import-key-pair / generate-key-pair` produces a password-encrypted private key) raises the cost from "copy a file" to "copy a file *and* obtain the passphrase." That is better, but the passphrase must be *somewhere* to be usable by automation — a CI secret, an env var, a config file — and now you are protecting the passphrase with the same broken tools you were protecting the key with. Encryption at rest defends against a stolen

backup or a lost laptop; it does nothing against an attacker who already has the process that decrypts and uses the key. Marginal, not solved.

Cloud KMS — the enterprise default for signing

The structural leap is a **key management service**: AWS KMS, GCP Cloud KMS, or Azure Key Vault. The defining property is that **the private key material never leaves the service**. You do not possess the key; you possess *permission to ask the service to sign on your behalf*. You send a digest to a `Sign` API, IAM decides whether your caller is allowed to invoke that key for that operation, the service signs inside its own boundary, and you get back a signature. The key is **not exportable** — there is no API that returns the private bytes.

Re-read Chapter 2's key-theft class against that. There is nothing to steal from the runner, because the runner never held the key. An attacker who fully compromises your CI can, at worst, *use* the key while their access lasts — and every such use is an IAM-authenticated, logged API call. That converts silent, permanent key theft into a bounded, revocable, *auditable* misuse window, which is exactly the trade keyless signing makes with identity, achieved here with a key you still nominally own. This is why KMS is the enterprise default for any signing key that must be a *key* (rather than a keyless identity).

Cosign speaks KMS natively through URI references — the key is named by a URI and Cosign calls the provider's signing API instead of reading a file:

```
# Generate an asymmetric signing key inside the KMS (never leaves the
boundary),
# then reference it by URI for signing and verification.

# AWS KMS – reference by alias, key id, or full ARN
cosign generate-key-pair --kms awskms:///alias/release-signing
cosign sign --key awskms:///alias/release-signing \
  registry.example.com/app@sha256:1a2b...
# Verify against the KMS-held public key (or an exported copy)
cosign verify --key awskms:///alias/release-signing \
  registry.example.com/app@sha256:1a2b...

# GCP Cloud KMS – the URI is the full CryptoKeyVersion resource path
cosign sign \
  --key gcpkms://projects/acme/locations/global/keyRings/supply-chain/
cryptoKeys/release/cryptoKeyVersions/1 \
  registry.example.com/app@sha256:1a2b...

# Azure Key Vault – vault host + key name (auth via AZURE_* env /
managed identity)
cosign sign --key azurekms://acme-kv.vault.azure.net/release-signing \
  registry.example.com/app@sha256:1a2b...

# HashiCorp Vault Transit – the transit key name
cosign sign --key hashivault://release-signing \
  registry.example.com/app@sha256:1a2b...
```

Three operational points that matter more than the syntax. **First**, the KMS key must be an *asymmetric signing* key with a compatible algorithm — ECDSA P-256 (ECC_NIST_P256 in AWS) or RSA-2048/3072; a symmetric KMS key cannot sign. **Second**, IAM is now your access control for signing: an AWS key policy plus IAM grants decide *which principals* may call `kms:Sign` on *which* key, and that is where "who can sign the release image" is actually enforced — not in a wiki. **Third**, every `Sign` call lands in CloudTrail / Cloud Audit Logs / Azure Monitor with caller identity, key, and timestamp, giving you a per-signature audit trail for free. That audit stream is a fleet-wide misuse detector (Book 8): a `kms:Sign` on the release key from an unexpected principal or region is an alarm.

HSM — hardware for the crown jewels

A **Hardware Security Module** is the same "sign via API, key never leaves" idea implemented in dedicated, tamper-resistant hardware validated to **FIPS 140-2** or **FIPS 140-3** (the successor standard; Level 3 adds *tamper-response* — the module zeroizes keys on physical intrusion).

Keys are generated *inside* the HSM by its hardware RNG and are **non-exportable** by policy; you drive it through the **PKCS#11** interface (the vendor-neutral cryptographic-token API) or a vendor/KMS-fronted API. Options span **on-prem appliances** (Thales Luna, Entrust nShield), **cloud HSMs** (AWS CloudHSM, Azure Dedicated HSM / Managed HSM, Google Cloud HSM — the last is a FIPS 140-2 Level 3 backing tier *within* Cloud KMS), and USB tokens (YubiHSM, or a YubiKey's PIV applet for a developer-scale hardware key).

Cosign can sign with a PKCS#11 token directly:

```
# PKCS#11 URI naming a token slot and key label; PIN via env or config
cosign sign --key "pkcs11:token=release-hsm;object=code-signing?module-
path=/usr/lib/softhsm/libsofthsm2.so" \
  registry.example.com/app@sha256:1a2b...
```

Reach for an HSM specifically when: you run a **high-assurance root or intermediate CA** (the signing key of an internal CA is worth an HSM because it vouches for everything below it); a **regulatory or contractual regime** mandates FIPS-validated key storage (FedRAMP, PCI DSS, many OS root programs require the CA's key in a FIPS 140-2 Level 3 module); or you are protecting a **root of trust** — a TUF root key, a release key that signs software shipped to customers — where the difference between "very hard to extract" (KMS) and "cannot be extracted without defeating certified tamper hardware" (HSM) is worth the operational cost. For most *application* signing keys, cloud KMS is the correct default and an HSM is over-provisioning; reserve HSMs for the small set of keys whose compromise is catastrophic.

TPM and secure enclaves for workload and edge keys

Two hardware roots of trust show up at the *workload* rather than the *CA* tier and deserve a mention. A **TPM** (Trusted Platform Module) is a chip present on most server and laptop hardware that can generate and hold non-exportable keys and — importantly — *attest* to the boot and software state of the machine. SPIRE (below, and Book 9) can use TPM-based node attestation to bind a workload identity to a specific attested machine. **Secure enclaves** (Intel SGX, AWS Nitro Enclaves, Arm CCA) provide an isolated execution environment where a key can be generated, used, and sealed such that even the host OS cannot read it. For edge and IoT fleets, a per-device TPM or secure element is what makes "one non-exportable

identity key per device" tractable at scale. The principle is identical to the CA-tier one, pushed to the leaf: *the key is born in hardware, is used in hardware, and never becomes a file.*

The storage decision, tabulated

Storage	Assurance	Extraction cost	Rotation ease	Fits
Software file (.pem)	Very low	Copy one file	Trivial	Ephemeral/ throwaway keys only
Passphrase- encrypted file	Low	File + passphrase (often colocated)	Trivial	Dev-local, low value
Cloud KMS (AWS/GCP/ Azure)	High	Breach KMS boundary + IAM	Easy (new key version + rebind)	Default for signing keys
HSM / CloudHSM (PKCS#11)	Very high	Defeat FIPS 140-2/3 tamper hardware	Harder (ceremony)	CA roots, release/root keys, regulated
TPM / secure enclave	High (per- node)	Defeat chip + attestation	Per-device reissue	Workload/ edge identity keys

The rule that spans the whole table: **prefer "key never leaves hardware/KMS, sign via an API" over "key in a file," always.** Possession of key material is a liability; permission to invoke a key is an asset you can revoke.

PKI for the enterprise: building and running internal trust

An enterprise needs its own public-key infrastructure for reasons that have nothing to do with the public web PKI: issuing certificates for **internal service mTLS** and **SPIFFE identities** (Book 9), **internal code signing**, the **TUF** roles you may run (Chapter 7), and — the reason this

book cares — an **internal Fulcio** that issues keyless-signing certificates against *your* OIDC identities rather than the public good instance's. All of these need a trusted issuer, and that issuer is an internal CA.

The offline-root / online-intermediate hierarchy

The load-bearing pattern of every serious PKI is a **two-tier (or deeper) hierarchy** that separates the key that *anchors* trust from the keys that *do* the daily issuing:

- A **root CA** whose key is the ultimate trust anchor. It signs almost nothing — only intermediate CA certificates — and it is kept **offline**, its key in an **HSM**, powered on rarely and only for a witnessed ceremony. Its certificate is what you distribute to clients as the trust root; because it is used so seldom, its key is extraordinarily hard to compromise, and its long validity (10–20 years) is acceptable *because* it is offline.
- One or more **intermediate (issuing) CAs**, whose keys are **online** in KMS/HSM and which do the actual high-volume work of issuing leaf certificates. If an intermediate is compromised, you revoke *it* at the root and stand up a new one, without touching the root or re-distributing the trust anchor to every client.
- **Leaf certificates** — the short-lived, high-churn certs issued to services, workloads, and signers.

This is not a coincidence of naming: it is **exactly the online/offline split TUF uses** (Chapter 7), where the offline **root** role delegates to online **targets/snapshot/timestamp** roles so that the keys touched by automation every few minutes are never the keys that anchor trust. Same threat model, same solution: keep the anchor cold, delegate the hot path.

```

flowchart TD
    subgraph offline["OFFLINE – HSM, ceremony only"]
        ROOT["Root CA"]
        key in HSM, powered on rarely
        validity ~10-20 yr
        signs ONLY intermediates[]
    end
    subgraph online["ONLINE – KMS/HSM, automated issuance"]
        INT1["Issuing CA: mTLS/SPIFFE"]
        short-lived leaf certs[]
        INT2["Issuing CA: internal Fulcio"]
        keyless signing certs[]
        INT3["Issuing CA: code signing"]
    end
    ROOT -->|"signs (ceremony)"| INT1
    ROOT -->|"signs (ceremony)"| INT2
    ROOT -->|"signs (ceremony)"| INT3
    INT1 -->|"issues, TTL ~hours"| L1["service mTLS certs"]
    INT2 -->|"issues, TTL ~10 min"| L2["workload signing certs"]
    INT3 -->|"issues"| L3["signed internal artifacts"]

```

Compromise of an intermediate is *recoverable* (revoke at the root, rotate); compromise of the root is *catastrophic* (re-establish trust everywhere) — which is precisely why the root does almost nothing and lives offline.

Tools for running internal PKI

You do not build a CA from OpenSSL scripts anymore; several mature tools implement the hierarchy and, crucially, the *automation* that makes short-lived certs viable:

- **HashiCorp Vault PKI secrets engine** — issues certificates on demand via API, with roles constraining allowed subjects/SANs/TTLs, and can act as both root and intermediate. Vault also now ships an **ACME server** so standard ACME clients (and cert-manager) can enroll against it. Its natural mode is **dynamic, short-lived certs**: a service requests a cert, uses it for an hour, and lets it expire.
- **step-ca (Smallstep)** — a small, opinionated online CA with **ACME**, **OIDC**, **JWK**, and **X5C** provisioners; issues X.509 *and* SSH certificates; defaults to short lifetimes. Excellent for an internal ACME endpoint and for developer/host certificate automation.

- **cfssl (CloudFlare)** — a toolkit/library for CA operations and issuance; lower-level, good for embedding certificate issuance into your own control plane.
- **Cloud private CA — AWS Private CA** (formerly ACM Private CA) and **Google Cloud Certificate Authority Service (CAS)** run the CA as a managed service with the root/intermediate key in the cloud provider's HSM (FIPS 140-2 Level 3), IAM-controlled issuance, and native integration with the rest of the cloud. You trade some control for not operating CA infrastructure yourself.
- **EJBCA** and **Dogtag** — full enterprise CA platforms (Common Criteria / WebTrust-grade) for organizations that need a heavyweight, audited, feature-complete CA — multiple profiles, CMP/EST enrollment, formal validation. Reach for these when you are effectively a PKI operator.

Short-lived certificates: the modern default

Chapter 1 and Chapter 2 established that **certificate revocation does not work well**: CRLs grow unbounded and are fetched late or not at all; OCSP adds a latency/privacy/availability dependency on every validation and is widely soft-failed (an attacker who blocks the OCSP responder turns "revoked" into "unknown → accept"). The web PKI has spent two decades failing to make revocation reliable. The modern internal answer is to **not need revocation**: issue certificates with a very short **time-to-live** and let them **expire** instead of revoking them.

If a service certificate is valid for one hour and is reissued continuously, then "revoke" becomes "stop renewing" — a compromised or decommissioned identity simply stops getting new certs and its current cert dies within the TTL. There is no CRL to distribute, no OCSP responder to keep up, no soft-fail hole. The security window equals the TTL. This is why **Vault**, **step-ca**, and **SPIRE** all default to short lifetimes, and it is the same reasoning behind **Fulcio's ~10-minute certificates** (Chapter 3–4) taken to the extreme: Fulcio issues a cert so short-lived that revocation is not even a concept — the cert is expired before a revocation entry could propagate, and verification instead relies on the Rekor timestamp proving the signature happened *during* the validity window (Chapter 5). SPIRE sits in the middle, issuing **X.509-SVIDs** with a default one-hour TTL and rotating them automatically for every workload in the mesh (Book 9).

TTL class	Example	Revocation strategy
Years	Web PKI leaf (legacy), offline root	CRL/OCSP — the broken model
Hours	SPIRE X.509-SVID, Vault/step-ca service cert	Stop renewing; expiry
~10 min	Fulcio keyless signing cert	None — expiry + logged timestamp

The trade you accept is a **hard dependency on automation and on the issuing CA's availability**: if certs live an hour, your CA must be up and your renewal loop must run, or things expire and the fleet stops talking. That is the right trade — an available, automated CA is a solvable operations problem; reliable revocation across a large fleet is not — but it moves the CA firmly into tier-1 infrastructure (see the distributed-systems lens).

Certificate lifecycle automation

Short-lived certs are only viable if issuance, renewal, and rotation are **fully automated**; manual certificate management at any scale produces the classic **expired-certificate outage** — the incident where a cert nobody was tracking lapsed and took down a load balancer, an API, or an mTLS mesh at the worst possible hour. Automate all of it:

- **ACME** (the protocol behind Let's Encrypt) is not just for the public web. Run it *internally*: Vault's ACME server, step-ca, and Smallstep all speak ACME, so any ACME client can enroll and auto-renew against your internal CA using the same battle-tested protocol.
- **cert-manager** is the Kubernetes-native answer. You declare an `Issuer / ClusterIssuer` (backed by Vault, step-ca, an ACME endpoint, AWS Private CA, or a self-signed root) and a `Certificate` resource, and cert-manager issues the cert into a `Secret`, watches its expiry, and **renews it automatically** before it lapses. The desired state is declarative; the renewal loop is the controller's job, not a human's.

```

# cert-manager: an internal issuing CA and an auto-renewed short-lived
cert
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: internal-ca
spec:
  vault:                                # back the issuer with Vault PKI
    server: https://vault.internal:8200
    path: pki_int/sign/service
    auth:
      kubernetes:
        role: cert-manager
        mountPath: /v1/auth/kubernetes
---
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: payments-mtls
spec:
  secretName: payments-mtls-tls
  issuerRef:
    name: internal-ca
    kind: ClusterIssuer
  duration: 1h                          # short-lived
  renewBefore: 20m                      # controller renews well before expiry
  dnsNames:
    - payments.svc.cluster.local

```

The pattern generalizes: **declare the desired certificate, let a controller keep it fresh**. The human never touches a `.pem`, and there is no expiry to forget.

The key lifecycle

Storage and PKI are two facets of a single discipline: managing a key across its whole life. Every key — signing, root, mTLS, TUF — moves through the same stages, and the operational quality of your program is how deliberately you handle each one.

```

flowchart LR
    G["Generate  
secure RNG, in HSM/KMS"] --> D["Distribute / issue  
never copy private keys"]
    D --> U["Use  
sign via API"]
    U --> R["Rotate  
scheduled + on-compromise"]
    R --> X["Revoke / expire  
prefer short-lived"]
    X --> Z["Destroy  
zeroize / delete key version"]
    U -. "every use" .-> A["Audit log  
who signed what, when"]
    R -. " " .-> A
    X -. " " .-> A

```

Generation. Keys must come from a cryptographically secure RNG, and the strongest form is **generation inside the HSM/KMS** so the private material never exists outside the hardware boundary — no "generate on a laptop and import" step, which briefly creates the exportable file you spent the whole chapter avoiding. Cloud KMS "create key" and HSM "generate key pair" both do this; `cosign generate-key-pair --kms ...` delegates generation to the provider.

Distribution — the anti-pattern to eliminate. The single most common key-management failure is **copying a private key to where it is needed**: onto every server, into every CI project, across regions. Each copy is another place to steal it from, and you can no longer say *where* your key is. The rule is **never copy a private key**. Either (a) **issue a distinct key per location** (per service, per node, per workload — SPIRE issues a unique SVID to each workload), so there is no shared secret to leak, or (b) **keep one central key and have everyone call a signing API** (KMS or a signing service — next section), so the key stays in one controlled place and callers hold only permission. Both eliminate the copied secret; which you choose depends on whether the identity is per-workload (issue-per-location) or a shared organizational identity (central signing).

Use. Every use should be an *authenticated, authorized, logged* operation — the KMS/HSM `Sign` API gives you all three. This is where audit lives.

Rotation. Rotate keys **on a schedule** (to bound the exposure of any single key and to keep the rotation mechanism exercised — a rotation

path you never run will not work in the emergency) and **on compromise** (immediately, when you suspect exposure). The hard part is rotating a *signing key* **without breaking verification**: verifiers pinned to the old public key will reject everything signed by the new one. Three mechanisms handle this, in increasing order of elegance:

- **Overlap / multiple trusted keys.** Configure verifiers to trust *both* the old and new public key during a transition window. Start signing with the new key while old signatures still verify; once everything in flight is re-signed and the window passes, drop the old key. This is the general-purpose answer for pinned-key signing.
- **Threshold / role rotation (TUF).** TUF's root role is designed to rotate: a new root version is signed by a **threshold of the old root keys** and the new ones, so clients holding the old root can validate the transition to the new root and update their trust safely (Chapter 7). The offline root's whole purpose is to make this rare, deliberate, and compromise-resistant. This is the gold standard for rotating a trust anchor.
- **Identity instead of key (keyless).** With keyless signing there is *no signing key to rotate* — the durable anchor is the OIDC identity and the Fulcio root, and the per-signature key is thrown away in seconds (Chapter 4). Rotation of the signing key is a non-problem by construction; you rotate the *identity's* credentials at the IdP instead.

Revocation and its limits. Revocation is the escape hatch for "this key is compromised and must stop being trusted *now*," but Chapter 2 showed it works poorly (CRL/OCSP fragility, soft-fail). Prefer designs that **don't depend on revocation**: short-lived certs (expiry replaces revocation), keyless (nothing durable to revoke), and multiple-trusted-keys (drop the bad key from the trust set). Where you must revoke — a compromised long-lived intermediate — do it at the layer that distributes trust (revoke the intermediate at the root; update the TUF metadata; drop the key from the verifier's allowlist), not by hoping every client fetches a CRL in time.

Destruction. When a key's life ends, it must be **irrecoverably destroyed** — HSM zeroization, KMS key-version deletion (note the mandatory waiting period AWS/GCP impose to prevent accidental loss), and destruction of any backups. A "destroyed" key that still exists in a snapshot is not destroyed.

Audit. Log **every** key use — signer identity, key, operation, timestamp — centrally. KMS and HSM give this natively; a signing service adds application-level context ("which artifact, which pipeline, which policy"). This log is both your compliance evidence and your **misuse detector** (Book 8): unexpected use of a high-value key is one of the highest-signal alerts you can build.

Root-key ceremonies

The highest-value keys — a root CA key, a TUF root key, a top-level release key — are generated and managed through a **key ceremony**: a formal, documented, *witnessed* procedure, performed **offline** (air-gapped hardware, HSM), producing an auditable record of exactly what happened. The defining properties:

- **m-of-n threshold.** The key (or the authority to use it) is split so that no single person can wield it — a quorum of `m` out of `n` designated key-holders must act together. Chapter 1 covered the mechanisms (Shamir secret sharing to split a key; true threshold signatures like FROST; or, most commonly, `m` distinct signing keys held by `m` people with a policy requiring a threshold of signatures). This removes the single insider and the single stolen laptop as points of failure.
- **Offline and air-gapped.** The ceremony runs on hardware never connected to a network, with the key in an HSM, so there is no online path to the material at any point.
- **Documented and witnessed.** A written script executed step by step, multiple witnesses, recorded evidence (often video), and signed attestations of what occurred — so the provenance of the trust anchor is itself auditable.

This is not theoretical: it is **exactly how Sigstore's own TUF root is managed** (Chapter 7). The Sigstore root-signing ceremony uses threshold-held keys on hardware, is performed by named key-holders, and — through the `tuf-on-ci` / `root-signing` workflow — is now conducted with the ceremony steps and signatures captured in a public, auditable repository. When you run your own internal root CA or private Sigstore, this is the model to copy: the crown-jewel key is born and rotated in a ceremony, and everything else delegates from it.

Signing infrastructure patterns for the org

Bring storage, PKI, and lifecycle together and two org-scale architectures for "how does an artifact get signed here" emerge. They are not competitors so much as answers to different questions, and most mature enterprises run both.

Centralized signing service vs. keyless

The **centralized signing service** pattern: teams do **not** hold signing keys. Instead they call an internal **signing API** — backed by KMS or an HSM — that signs on their behalf after checking policy. The service owns the keys, enforces "who can sign what" (this team may sign images in this namespace; only the release pipeline may invoke the release key), and emits a single, uniform audit stream for every signature in the company. It is the "central key, everyone calls the API" distribution model from the lifecycle section, elevated to a platform.

The **keyless / workload-identity** pattern (Chapter 3-4): there is no signing key to hold at all. The build's *workload identity* — its OIDC token — is presented to Fulcio, which issues a ~10-minute certificate, the build signs with an ephemeral key, and the signature plus certificate go to Rekor. There is no key custody problem because there is no durable key.

```
flowchart TD
    subgraph central["Centralized signing service (KMS-backed)"]
        T1["Team pipeline"] -->|"authN + request"| SS["Signing service  
(policy: who signs what)"]
        SS -->|"kms:Sign (IAM-gated)"| KMS["KMS / HSM  
keys held here"]
        KMS -->|"signature"| T1
        SS -->|"every sign"| AUD["Central audit log"]
    end
    subgraph keyless["Keyless workload-identity signing"]
        T2["Build (workload identity)"] -->|"OIDC token + pubkey"| FUL["Fulcio  
(issues ~10-min cert)"]
        FUL -->|"cert"| T2
        T2 -->|"sign w/ ephemeral key"| REK["Rekor  
(transparency log)"]
        REK -->|"public record"| AUD2["Fleet-wide signing audit"]
    end
```

Compare them directly:

Dimension	Centralized KMS signing service	Keyless (Sigstore)
Keys held	Yes — in KMS/HSM, you own them	No durable signing key
Access control	IAM + service policy	OIDC identity + verifier policy
Audit	Central app + KMS logs (can be private)	Rekor transparency log (public unless self-hosted)
Trust anchor	Your KMS key / internal CA	Fulcio root + TUF (Chapter 7)
Best for	Release/high-assurance/root keys, regulated signing, privacy	High-volume CI artifact signing
Failure if down	Cannot sign until service recovers	Cannot sign until Fulcio/Rekor recover

The practical enterprise posture is **both: keyless for the high-volume CI artifact stream** (no keys to manage across thousands of pipelines, identity-based verification), and a **KMS-backed centralized signing service for the high-assurance tier** — release keys, root keys, anything with a regulatory FIPS requirement, or anything you cannot expose in a public transparency log for privacy reasons (Chapter 5's metadata-leak concern). Use each where its properties fit; do not force one architecture to cover both jobs.

Self-hosted Sigstore

The bridge between the two — keyless signing but on infrastructure you own — is **self-hosted (private) Sigstore**. Enterprises stand this up when they cannot or will not depend on the public good instance:

- **Availability / SLAs.** The public `sigstore.dev` instance is a public good with best-effort availability; if signing artifacts is on your critical release path, you need a **tier-1** signing service with *your* SLA, HA, and DR — not a shared community deployment.
- **Privacy.** The public Rekor log is exactly that — public — and every entry leaks metadata: what you build, when, under which identities

(Chapter 5, and Chapter 5's privacy section). A private Rekor keeps that internal.

- **Internal-only trust and policy.** A private **Fulcio** issues certificates against *your* corporate OIDC (Okta/Entra) identities and *your* internal CA, so your verification policy binds to internal identities rather than public providers.

A self-hosted Sigstore is a set of cooperating services, each of which you now operate:

```
flowchart TD
    IDP["Corporate OIDC IdP  
(Okta / Entra / CI tokens)"] -->| "identity token" | FUL["Private Fulcio  
(CA)"]
    FUL -->| "logs issuance (SCT)" | CT["Private CT log"]
    FUL -->| "signature + cert" | REK["Private Rekor  
(KMS/HSM)"]
    FUL -->| "timestamp req" | TSA["Timestamp Authority  
(RFC 3161)"]
    FUL -->| "Merkle root / checkpoint" | KMS2["Rekor signing key  
(KMS/HSM)"]
    FUL -->| "distributes trusted keys of" | TUF["Internal TUF root  
(ceremony, threshold)"]
    TUF --> REK
    TUF --> CT
    TUF --> TSA
```

The components, and what operating each entails:

- **Fulcio** (the CA) — backed by your KMS/HSM signing key and chaining to your internal root (which is why the PKI section came first: private Fulcio is an issuing CA in your hierarchy). It validates tokens from *your* IdP.
- **Rekor** (the transparency log) — a Trillian-backed Merkle log with a MySQL/Spanner backend (Chapter 5), signing checkpoints with a KMS-held key. Now it is *your* append-only log to keep available and consistent.
- **A CT log** for Fulcio's certificate issuance (the SCT embedded in each cert; Chapter 3).

- **A Timestamp Authority (TSA)** — an RFC 3161 timestamping service, so signatures can be timestamped without solely depending on Rekor's log time.
- **An internal TUF root** distributing the trusted keys of *all* of the above to clients, rotated by ceremony (Chapter 7). This is the trust anchor of your private Sigstore, and it is the one key set you manage with the full ceremony discipline.

The **sigstore/scaffolding** project (Helm charts and Terraform for a Kubernetes deployment) is the canonical way to stand these up, and `tuf-on-ci` manages the TUF root. The essential point for a backend engineer is the operational one: **self-hosted Sigstore is not a checkbox, it is four or five stateful, security-critical services** — a CA, one or two transparency logs, a TSA, and a TUF repository — each of which must meet the availability, consistency, and DR bar of your most important infrastructure. If your builds cannot sign because private Fulcio is down, your release pipeline is down; if verifiers cannot reach the private Rekor, you face the same fail-open / fail-closed decision Chapter 8 dissected. Run it like the tier-1 service it is, or use the public instance and accept its properties — but do not run a half-maintained private Sigstore on your critical path.

Distributed-systems lens

At fleet scale — many services, many teams, many pipelines — key management stops being about protecting *a* key and becomes about a fleet-wide policy for *all* keys. The through-lines:

- **No team should hold signing keys.** With hundreds of pipelines, "each team manages its own signing key" guarantees leaked keys. The two scalable answers are **keyless/workload-identity** (Chapter 4 — no key to hold) and a **centralized KMS-backed signing service** (teams hold IAM permission, not key material). Both replace `n` copied secrets with one controlled surface.
- **One PKI anchors both identity and signing.** The same internal PKI that issues **mTLS/SPIFFE identities** to every service (Book 9) can back an internal Fulcio that anchors **artifact signing** — the Chapter 4 convergence, where "who this service is" and "who signed this artifact" draw on the same root of trust. Do not run two disconnected trust hierarchies.

- **Short-lived certs + automated rotation eliminate two whole failure classes.** Across thousands of services, cert-manager/Vault/SPIRE issuing hour-long certs and renewing them automatically removes both the **expired-certificate outage** (a controller renews before expiry) and the **revocation problem** (expiry replaces revocation). Long-lived certs plus CRLs do not scale; short-lived certs plus automation do.
- **Signing and verification infrastructure is tier-1, with the availability question front and center.** A self-hosted Sigstore, an internal CA, a KMS — if signing is down, *can you ship?*; if the verification log is unreachable, do you **fail open or fail closed** (Chapter 8)? These are HA/DR design decisions, not afterthoughts, and they determine whether your security control becomes an availability outage.
- **Offline root + online intermediates + HSM for the crown jewels.** The handful of keys whose compromise is catastrophic — the root CA, the TUF root, the top release key — get the maximum treatment (offline, HSM, m-of-n ceremony) precisely so the millions of daily operations can run on cheap, rotatable, delegated keys.
- **Every sign operation is audited centrally.** KMS logs, a signing service's application log, and Rekor's public ledger all converge on the same capability: a fleet-wide, queryable record of who signed what and when, which is the substrate for **misuse detection** (Book 8). At scale, the audit stream is as valuable as the access control.

Key takeaways

- **Keyless does not abolish keys; it relocates them.** Someone runs Fulcio's CA, the TUF root, and Rekor's key; enterprises still hold release, root, and mTLS keys. Key management is the residual hard problem, and it is where cryptography meets operations and usually breaks.
- **Storage is an extraction-cost spectrum.** Software file (copy one file) → passphrase-encrypted (marginal) → **KMS** (key never leaves; sign via IAM-gated API; every use logged — the enterprise default) → **HSM** (FIPS 140-2/3, non-exportable, PKCS#11 — for CA roots, release/root keys, regulated signing). Always prefer "sign via an API, key never leaves hardware" over "key in a file." Cosign speaks KMS

natively: `awskms://`, `gcpkms://`, `azurekms://`, `hashivault://`, and `pkcs11:`.

- **Enterprise PKI is an offline-root / online-intermediate hierarchy** — the same online/offline split as TUF. The root lives offline in an HSM and signs only intermediates; online issuing CAs (Vault PKI, step-ca, cfssl, AWS Private CA, GCP CAS, EJBCA/Dogtag) do the high-volume leaf issuance and are recoverable if compromised.
- **Short-lived certificates + automation replace revocation.** Issue hour-long (or ~10-minute) certs and let them expire; automate issuance/renewal with ACME and cert-manager. This kills both the revocation problem (Chapter 2) and the expired-cert outage — at the cost of a hard dependency on an available, automated CA.
- **Run the key lifecycle deliberately:** generate in-HSM/KMS, **never copy private keys** (issue per-location or centralize signing), rotate on schedule and on compromise (overlap/multiple trusted keys, or TUF-style threshold rotation, or keyless where there is nothing to rotate), prefer expiry over revocation, destroy irrecoverably, and **audit every use**. Crown-jewel keys are born and rotated in **offline, m-of-n, witnessed ceremonies** — the Sigstore TUF-root model.
- **Two org-scale signing architectures, and you likely want both:** a **centralized KMS-backed signing service** (you hold keys, IAM-gated, private audit — for release/root/regulated keys) and **keyless** (no keys, OIDC identity — for high-volume CI artifacts). **Self-hosted Sigstore** (private Fulcio/Rekor/CT/TSA/TUF via `sigstore/scaffolding`) gives keyless on infrastructure you own, for availability, privacy, and internal-only trust — but it is four or five tier-1 stateful services, not a checkbox.
- **Distributed-systems bottom line:** no team holds keys; one PKI anchors both service identity (mTLS/SPIFFE) and artifact signing; short-lived certs plus automated rotation eliminate expiry outages and revocation pain across thousands of services; and your signing/verification infrastructure is tier-1 — design its HA/DR and its fail-open/closed behavior before an outage designs it for you.

Further reading

- **AWS KMS, Google Cloud KMS, and Azure Key Vault** developer guides — asymmetric signing keys, the `Sign` API, key policies/IAM, and audit integration (CloudTrail / Cloud Audit Logs / Azure Monitor). <https://docs.aws.amazon.com/kms/>, <https://cloud.google.com/kms/docs>, <https://learn.microsoft.com/azure/key-vault/>.
- **Cosign KMS support** — the `awskms://`, `gcpkms://`, `azurekms://`, `hashivault://` URI schemes and PKCS#11 signing. https://docs.sigstore.dev/cosign/key_management/ and <https://github.com/sigstore/cosign/blob/main/KMS.md>.
- **FIPS 140-2 / FIPS 140-3** (NIST) and **PKCS#11** (OASIS Cryptographic Token Interface) — the validation standard and the interface for HSMs. <https://csrc.nist.gov/publications/detail/fips/140/3/final> and the OASIS PKCS#11 spec.
- **HashiCorp Vault PKI secrets engine** (including its ACME server) and **Smallstep step-ca** — API CAs built for short-lived certificate automation. <https://developer.hashicorp.com/vault/docs/secrets/pki> and <https://smallstep.com/docs/step-ca/>.
- **cert-manager** — Kubernetes certificate lifecycle automation (`Issuer` / `ClusterIssuer`, `Certificate`, automatic renewal). <https://cert-manager.io/docs/>.
- **AWS Private CA and Google Cloud Certificate Authority Service** — managed private CAs with HSM-backed keys. <https://docs.aws.amazon.com/privateca/> and <https://cloud.google.com/certificate-authority-service/docs>.
- **ACME (RFC 8555)** — the enrollment/renewal protocol, usable internally. <https://www.rfc-editor.org/rfc/rfc8555>.
- **SPIFFE / SPIRE** — short-lived X.509-SVID/JWT-SVID issuance and rotation for workload identity (developed in Book 9). <https://spiffe.io/docs/>.
- **Sigstore self-hosting** — `sigstore/scaffolding` (Helm/Terraform for private Fulcio/Rekor/CT/ TSA) and the Fulcio/Rekor operational docs. <https://github.com/sigstore/scaffolding> and <https://docs.sigstore.dev/>.
- **The Update Framework (TUF) and Sigstore root-signing / tuf-on-ci** — threshold root keys, secure rotation, and the ceremony model (Chapter 7). <https://theupdateframework.io/>, <https://>

github.com/sigstore/root-signing, and <https://github.com/theupdateframework/tuf-on-ci>.

- Cross-references: Book 5, Chapter 1 (crypto foundations, Shamir/threshold/FROST, DSSE), Chapter 2 (classic code signing — key theft, broken revocation), Chapters 3–4 (Sigstore, Fulcio, keyless signing and workload identity), Chapter 5 (transparency logs, Rekor internals, privacy/metadata leakage), Chapter 7 (TUF — offline root, threshold rotation, ceremonies), Chapter 8 (provenance verification, fail-open/closed), Chapter 10 (deployment gates); Book 4, Chapter 6 (secrets in CI); Book 8 (monitoring key use as fleet-wide misuse detection); Book 9 (mTLS, SPIFFE/SPIRE, service identity PKI).

Chapter 10 — Designing Attestation-Based Deployment Gates

What this chapter covers. This is the capstone of Book 5 and, in a real sense, of the whole supply-chain arc that runs from Book 3 through Book 5. Everything the earlier chapters built — signatures (Chapters 2–4), the transparency log that makes ephemeral keys auditable (Chapter 5), in-toto attestations carrying provenance, SBOMs, and scan results (Chapter 6), TUF-protected roots of trust (Chapter 7), the verification checklist (Chapter 8), and the PKI/key custody underneath it all (Chapter 9) — is *production*: it emits signed claims. None of it changes an attacker's calculus by itself. The value is realized at exactly one place: the **deployment gate**, the control that reads all that metadata, compares it to a **policy**, and refuses to run anything that fails. This chapter is about designing that gate — where it lives, what it checks, how you roll it out across a fleet without breaking every deploy on day one, and how you make "only verified artifacts run in production" a structural property of your platform rather than an aspiration on a slide.

Learning goals — after this chapter you should be able to:

- Explain why the deployment gate is *the* enforcement point of supply chain security, and why **admission control in the runtime platform is the non-bypassable backstop** behind CI and CD gates.
- Assemble a concrete, multi-property **deployment policy** from the mechanisms of Books 3–5: signature + expected identity, digest

pinning, provenance vs source/builder/SLSA level, required attestations, transparency-log inclusion, and vulnerability/VEX posture.

- Implement gates with real tooling: **sigstore policy-controller** (`ClusterImagePolicy`), **Kyverno** (`verifyImages` + attestations), **OPA/Gatekeeper** + **Ratify**, **Connaisseur**, and managed services like **Google Binary Authorization**.
- Design the **policy lifecycle**: warn/audit → enforce rollout, break-glass and time-boxed exceptions, tiered requirements for legacy and third-party images, and a defensible **fail-open vs fail-closed** stance.
- Instrument the gate with metrics that *prove* it works, and reason about it as a fleet-wide, tier-0, HA control in the pod-scheduling path.
- Draw the end-to-end Books 3–5 architecture: produce (build + sign + attest) → store/log → gate verifies → admit/deny → runs.

The gate is where supply chain security is enforced

State the thesis plainly, because the industry's incentives constantly push against it. Producing signatures and attestations is visible, demo-friendly, and satisfying: you run `cosign sign`, a green check appears, a compliance box is ticked. Verifying them is invisible when it works and a production outage when it is too strict. So the producing half is chronically over-invested and the consuming half chronically neglected — and the result is an ecosystem full of diligently signed artifacts that nothing on the consuming side ever inspects. As Chapter 8 put it: a pipeline that signs everything and verifies nothing has *more* attack surface than one that does neither, because it manufactures assurance while providing no gate.

The deployment gate is the trust boundary between **built** and **running in production**. On the near side is a heterogeneous, partly untrusted world: dozens of build pipelines, hundreds of repos, external base images, vendor artifacts, a registry that anyone with push credentials can write to, and the standing possibility that any one of those was compromised (Book 1's case studies — SolarWinds' build-time implant, the Codecov script tamper, the xz-utils backdoor — are all failures that a downstream gate checking provenance and identity could have blunted). On the far side is your runtime: the thing that actually executes code against production data and credentials. The gate is the last place you

can say *no* before an artifact becomes a running process with network access and a service-account token.

This is the **produce-AND-verify** principle from Chapter 8 and Book 4, Chapter 3, restated as an architectural mandate: for every signed claim your build platform emits, there must be a corresponding check at a gate that *refuses to proceed* when the claim is missing, invalid, or non-compliant. A claim that never gates anything is a log line with extra cryptography. The gate is the mechanism that turns metadata into a decision, and the decision into a control.

Defense in depth, and admission as the backstop

You do not want a single gate. You want gates at several points, each defending a different trust boundary, arranged so that the *last* one is the one an attacker cannot route around.

There are three natural gate locations in a modern deployment path:

1. **Registry ingest** — verify on push (and/or on pull). Catches bad artifacts early, close to the build. But it is bypassable: an attacker who can push directly to the registry, or who compromises the ingest checker's configuration, can defeat it, and not every path to production goes through your ingest hook.
2. **CD pipeline** — a verification step in Argo CD, Flux, Spinnaker, or your bespoke deploy tooling, run *before* rollout. This is the natural place for rich, slow checks (fetch and evaluate full provenance, cross-reference a vulnerability service). But CI/CD is itself a high-value attack surface (Book 4, Chapters 4 and 7 on pipeline poisoning): a compromised pipeline can simply *skip* its own verification step. A gate that the thing being gated can turn off is not a control.
3. **Admission control in the runtime platform** — a Kubernetes `ValidatingAdmissionWebhook` (or a policy engine wired into one) that verifies signatures and attestations at the moment a Pod is created, *before the kubelet is ever told to pull and run the image*. This runs in the API server's request path, outside the CI/CD trust domain, and it sees every workload regardless of how it got there — `kubectl apply` by a human, a GitOps reconcile, an operator, a Job spawned by another controller. It is the **backstop**.

The design principle: **treat admission as the non-bypassable backstop and everything upstream as shift-left convenience**. The registry and CD gates exist to fail fast and give developers feedback early; they reduce load on admission and catch problems before they reach a cluster. But you assume they *can* be bypassed, and you make the platform itself refuse to run an unverified image. Even if CI is fully compromised and pushes a malicious image with a valid-looking but wrong-identity signature, admission — running in the cluster, evaluating against central policy, checking the *expected* signer identity — still says no.

```
flowchart LR
    build["Build platform  
(Book 4)"] --> reg["Registry"]
    reg -->|"push"| g1["Ingest gate  
(bypassable)"]
    g1 --> cd["CD: Argo/Flux/Spinnaker"]
    cd --> g2["CD gate  
(bypassable if  
pipeline compromised)"]
    g2 --> api["Kube API server"]
    human["kubectl apply"] --> api
    operator["Operators / Jobs"] --> api
    api --> g3["Admission gate  
NON-BYPASSABLE backstop"]
    g3 -->|"deny"| stop["Rejected: no Pod created"]
    g3 -->|"admit"| kubelet["kubelet pulls + runs"]
    style g3 fill:#b30000,color:#ffffff
    style g1 fill:#e0a030
    style g2 fill:#e0a030
```

Notice the shape: the two upstream gates have side entrances (direct `kubectl`, operators, Jobs) that route *around* them straight to the API server. Only admission sits astride every path. That is the whole argument for centering your enforcement there. Books 6 (Chapters 5–6 on admission control and policy engines, Chapter 7 on GitOps verification) develops the runtime-platform mechanics in depth; here we treat admission as the enforcement primitive and focus on the *policy* it enforces.

One caveat you must design around: admission validates the *spec* of a Pod, and by default that spec may reference images by tag. A tag is a mutable pointer. If the gate verifies `app:v1.2.3` at admission time but the kubelet later resolves that tag to a different digest, you have a time-of-

check/time-of-use (TOCTOU) gap (Chapter 8). The fix is **verify-by-digest end to end**: the gate resolves the tag to a digest, verifies signatures and attestations *against that digest*, and — critically — mutates the Pod spec to pin the image to `app@sha256:...` so the kubelet pulls exactly the bytes that were verified. Both policy-controller and Kyverno do this digest-pinning mutation; it is not optional hardening, it is what makes the check sound.

What the gate checks: assembling the policy

A deployment gate is only as good as the policy it evaluates. This is where Books 3–5 converge: each mechanism contributes one clause. A serious prod policy is a *conjunction* of checks, and every one of them defends against a specific class of failure.

Check	Mechanism (source)	Defends against
Valid signature	Cosign / DSSE (Ch 2–4)	Unsigned or corrupted artifacts
Expected signer identity	Keyless cert identity / OIDC issuer + subject (Ch 4)	Attacker signing with <i>their own</i> valid key; the single most-botched check
Digest match / pinning	Content-addressed refs (Ch 8)	TOCTOU, tag mutation, wrong-artifact substitution
Provenance vs policy	SLSA provenance (Book 4 Ch 3)	Built from wrong source repo/branch, wrong builder, insufficient SLSA level
Required attestations present	in-toto predicates (Ch 6)	Missing SBOM, missing/failed tests, missing scan
Transparency-log inclusion	Rekor inclusion proof (Ch 5)	Off-log (unauditable) signatures; backdated/forged signing events
Vulnerability + VEX posture	Scan attestation + VEX (Book 2, Book 3 Ch 6)	Known-exploitable, reachable vulns shipping to prod

The two most important and most frequently skipped are **expected identity** and **verify-by-digest**. "Is this validly signed?" is nearly useless — anyone can produce a valid signature over anything with a key they

control. The question that matters is "is this signed *by the identity our policy expects* (our build platform's keyless identity, from our OIDC issuer, for a workflow in an approved repo) and is the signed subject *this exact digest?*" Every gate in this chapter is built around answering that question, not the useless one.

A worked prod policy

Here is a concrete, opinionated policy for a tier-0 production service. It is deliberately strict; the tiering discussion later relaxes it for other classes of workload.

To deploy to prod , an image must: 1. Be **signed keyless** by our build platform's identity — OIDC issuer `https://token.actions.githubusercontent.com` , certificate subject matching the release workflow `.../.github/workflows/release.yml@refs/heads/main` in an **approved build repo**. 2. Carry a **SLSA v1.0 provenance** attestation (`predicateType https://slsa.dev/provenance/v1`) whose `buildDefinition` shows the source is an approved repository, built on `main` (or a release tag), by our reusable/trusted builder — i.e., **SLSA Build L3** (Book 4, Chapter 3): non-forgable provenance from an isolated, hosted builder. 3. Carry an **SBOM attestation** (CycloneDX or SPDX; Book 3). 4. Carry a **passing vulnerability-scan attestation** with **no reachable KEV-listed vulnerabilities unaccounted for by VEX** (Book 2 Chapter 7 on reachability; Book 3 Chapter 6 on VEX). 5. Have its signature **logged in Rekor** with a verifiable inclusion proof (Chapter 5). 6. Be admitted **by digest**, with the Pod spec pinned to the verified `sha256` .

Every clause maps to a book. Clause 1 is Chapters 3–4. Clause 2 is Book 4, Chapter 3. Clause 3 is Book 3. Clause 4 is Books 2 and 3. Clause 5 is Chapter 5. Clause 6 is Chapter 8. The gate is the place where all of it is finally *load-bearing*.

Implementing the gate

Kubernetes admission: sigstore policy-controller

The sigstore project's **policy-controller** is a Kubernetes admission controller whose policy object is the `ClusterImagePolicy` (CIP). It matches images by glob and requires that each matched image satisfy a

set of *authorities* (who must have signed) and, optionally, attestation requirements.

```
apiVersion: policy.sigstore.dev/v1beta1
kind: ClusterImagePolicy
metadata:
  name: prod-keyless-and-attestations
spec:
  images:
    - glob: "registry.example.com/prod/**"
  authorities:
    - name: build-platform-keyless
      keyless:
        url: https://fulcio.sigstore.dev
        identities:
          - issuer: https://token.actions.githubusercontent.com
            subjectRegExp: "^https://github.com/example-org/[^/]+/\\
\\.github/workflows/release\\.yaml@refs/heads/main$"
      ctlog:
        url: https://rekor.sigstore.dev
      attestations:
        - name: require-slsa-provenance
          predicateType: https://slsa.dev/provenance/v1
          policy:
            type: cue
            data: |
              predicateType: "https://slsa.dev/provenance/v1"
              predicate: {
                buildDefinition: {
                  buildType: "https://actions.github.io/buildtypes/
workflow/v1"
                  externalParameters: {
                    workflow: {
                      repository: =~"^https://github.com/example-org/"
                    }
                  }
                }
              }
        - name: require-sbom
          predicateType: https://cyclonedx.org/bom
  mode: enforce
```

Read what this enforces. `keyless.identities` demands a Fulcio-issued certificate whose OIDC **issuer** and **subject** match — this is the expected-identity check, expressed as a regex over the release workflow's identity across any approved repo in the org. `ctlog.url` requires a Rekor entry (transparency, Chapter 5). The `attestations` block requires *both* a SLSA provenance predicate and a CycloneDX SBOM predicate to be present

and signed by that same authority, and the CUE policy on the provenance asserts the build came from an `example-org` repository via the GitHub Actions build type. `mode: enforce` makes a non-compliant image a hard admission failure; `mode: warn` (the audit posture) logs the violation and admits — the lifecycle lever we return to below.

`policy-controller` enforces only in namespaces you opt in by label (`policy.sigstore.dev/include: "true"`), and it digest-pins on admission. Its default behavior when *no* CIP matches an image is itself a policy decision (allow vs deny unmatched images) that you must set deliberately — a fleet where unmatched images are silently allowed has a gaping hole.

Kubernetes admission: Kyverno

Kyverno is a general-purpose policy engine with first-class image-verification support via `verifyImages`. It can check signatures *and* attestations, and evaluate conditions over attestation *contents* — useful for asserting things like "provenance says it was built from this repo."

```

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: prod-verify-images
spec:
  validationFailureAction: Enforce      # Audit during rollout, then
  Enforce
  webhookTimeoutSeconds: 20
  failurePolicy: Fail                    # fail-closed: deny if the
webhook errors/times out
  background: false
  rules:
    - name: verify-keyless-signature-and-provenance
      match:
        any:
          - resources:
              kinds: ["Pod"]
              namespaces: ["prod", "prod-*"]
      verifyImages:
        - imageReferences:
            - "registry.example.com/prod/*"
          mutateDigest: true              # pin the verified digest into
the Pod spec
          verifyDigest: true
          required: true
          attestors:
            - count: 1
              entries:
                - keyless:
                    subject: "https://github.com/example-org/*.github/
workflows/release.yml@refs/heads/main"
                    issuer: "https://
token.actions.githubusercontent.com"
                    rekor:
                      url: https://rekor.sigstore.dev
                attestations:
                  - type: https://slsa.dev/provenance/v1
                    attestors:
                      - entries:
                          - keyless:
                              subject: "https://github.com/example-org/
*/.github/workflows/release.yml@refs/heads/main"
                              issuer: "https://
token.actions.githubusercontent.com"
                              conditions:
                                - all:
                                    - key: "{{ buildDefinition.buildType }}"
                                      operator: Equals
                                      value: "https://actions.github.io/buildtypes/
workflow/v1"

```

```
- key: "{{ regex_match('^https://github.com/
example-org/',
buildDefinition.externalParameters.workflow.repository) }}"
    operator: Equals
    value: true
```

Two things earn their place here. `mutateDigest: true` is the TOCTOU fix — Kyverno rewrites the image reference to `...@sha256:<verified>` so the kubelet cannot resolve a tag to different bytes. And `failurePolicy: Fail` is the *fail-closed* stance: if the Kyverno webhook is unreachable or times out, the API server *rejects* the Pod rather than admitting it unchecked. `validationFailureAction` is the audit/enforce toggle (Audit logs a `PolicyReport` and admits; `Enforce` denies) — precisely the lifecycle control the next section formalizes.

OPA/Gatekeeper + Ratify, and Connaisseur

Two other production-grade patterns:

- **OPA/Gatekeeper + Ratify.** Gatekeeper enforces Rego constraints at admission but has no native notion of "verify a cosign signature." **Ratify** fills that gap: it is a verification engine (signature, SBOM, vulnerability-report verifiers via plugins) wired to Gatekeeper as an *external data provider*. A Gatekeeper `ConstraintTemplate` calls out to Ratify, which fetches and verifies the artifact's signatures/attestations from the registry and returns a pass/fail that the Rego constraint consumes. This suits shops already standardized on OPA/Rego for general policy who want image verification in the same engine.
- **Connaisseur** is a focused admission controller specifically for verifying container image signatures (cosign/Sigstore and Notary v1), designed to be small and auditable. It resolves tags to digests, verifies against configured trust roots/identities, and mutates the spec to the verified digest. It does less than Kyverno/policy-controller (it is signature-centric, less about rich attestation-content policy) but is a clean choice when signature enforcement is the whole job.

Choosing among them: if you want the richest attestation-content policy in a mainstream engine, Kyverno; if you are all-in on Sigstore and want its native object model, policy-controller; if you already run OPA/Gatekeeper for everything, Ratify; if you want a minimal signature-only gate, Connaisseur. All four are real, actively maintained admission-time

gates that do the same core job: verify expected-identity signatures and (except Connaisseur's narrower scope) required attestations, by digest, before the Pod runs.

CD-pipeline and GitOps gates

Admission is the backstop, but you also want to fail fast in the deploy pipeline so developers get feedback before a bad artifact reaches a cluster. In a CD tool this is a pre-rollout verification step. With Cosign directly:

```
cosign verify \  
  --certificate-identity-regexp "^https://github.com/example-org/.+/  
\.github/workflows/release\.yaml@refs/heads/main$" \  
  --certificate-oidc-issuer https://  
token.actions.githubusercontent.com \  
  registry.example.com/prod/checkout@sha256:${DIGEST}  
  
cosign verify-attestation --type slsaprovenance \  
  --certificate-identity-regexp "^https://github.com/example-org/.  
+@refs/heads/main$" \  
  --certificate-oidc-issuer https://  
token.actions.githubusercontent.com \  
  --policy prod-provenance.cue \  
  registry.example.com/prod/checkout@sha256:${DIGEST}
```

Note the same discipline as admission: verify by digest, assert the certificate identity, run a policy over the provenance predicate. In **GitOps** flows, Flux can verify the OCI artifacts and Helm charts it reconciles against a cosign identity (its source-controller supports Sigstore/keyless verification on `OCIRepository` / `HelmChart` sources), and Argo CD can gate sync on external verification (a pre-sync hook or an admission gate on the resources it applies). The GitOps mechanics are Book 6, Chapter 7; the point for us is that these are *shift-left* gates whose job is early feedback — they do not replace admission, because a compromised GitOps controller could skip them, and admission cannot be skipped.

Managed gates: Google Binary Authorization

Cloud providers offer managed deploy gates. **Google Binary Authorization** is an attestation-based admission control for GKE (and Cloud Run): you define a *policy* that requires images to have **attestations** from named **attestors** before they can be deployed, and Binary Authorization enforces it in the GKE admission path.

```
# Binary Authorization policy (gcloud container binauthz policy import)
globalPolicyEvaluationMode: ENABLE
admissionWhitelistPatterns:
  - namePattern: "gcr.io/google-containers/*"
  - namePattern: "gke.gcr.io/*"
defaultAdmissionRule:
  evaluationMode: REQUIRE_ATTESTATION
  enforcementMode: ENFORCED_BLOCK_AND_AUDIT_LOG
  requireAttestationsBy:
    - "projects/example-prod/attestors/built-by-approved-pipeline"
    - "projects/example-prod/attestors/vuln-scan-passed"
clusterAdmissionRules:
  "us-central1-a.prod-cluster":
    evaluationMode: REQUIRE_ATTESTATION
    enforcementMode: ENFORCED_BLOCK_AND_AUDIT_LOG
    requireAttestationsBy:
      - "projects/example-prod/attestors/built-by-approved-pipeline"
      - "projects/example-prod/attestors/vuln-scan-passed"
```

`enforcementMode: ENFORCED_BLOCK_AND_AUDIT_LOG` blocks non-compliant deploys and logs them; the alternative `DRYRUN_AUDIT_LOG_ONLY` is the audit posture (log, don't block) — again the same warn-then-enforce lever, this time as a first-class field. `admissionWhitelistPatterns` is how you exempt platform/system images that will never carry your attestations. AWS's analogous building blocks are **AWS Signer** (managed signing for container images and Lambda code) combined with admission enforcement; describe these to stakeholders accurately as *managed implementations of the same gate pattern* — an attestation/signature requirement enforced in the platform's deploy path — not as something categorically different from a self-run Kyverno/policy-controller gate.

The policy lifecycle: shipping a gate without an outage

The single fastest way to discredit supply-chain security in your org is to flip a fleet-wide gate to enforce on day one and break every deploy. A gate is a distributed rollout like any other risky change, and it needs the same discipline: observe, then enforce, incrementally.

Warn/audit before enforce

Every serious gate has a dry-run posture: evaluate the policy, **log what would be denied**, but admit. policy-controller's `mode: warn`, Kyverno's

`validationFailureAction: Audit`, `Binary Authorization's DRYRUN_AUDIT_LOG_ONLY` — all exist precisely for this. Run in audit across the fleet first and you get a free, complete inventory of exactly which images, namespaces, and teams would fail, *before* you break anyone. Then you drive the would-deny count to zero (fix the pipelines, onboard the stragglers, write the exceptions) and only then flip to enforce — and even then, flip it per-namespace or per-tier, not everywhere at once.

```
stateDiagram-v2
    [*] --> Audit: deploy policy, log-only
    Audit --> Audit: measure would-deny;
    fix pipelines / onboard teams
    Audit --> EnforceCanary: would-deny ~ 0 for a tier
    EnforceCanary --> Audit: canary breaks -> roll back
    EnforceCanary --> EnforceFleet: canary clean N days
    EnforceFleet --> EnforceFleet: steady state;
    monitor deny events
    EnforceFleet --> [*]
```

The state you must resist is jumping straight to `EnforceFleet`. The would-deny signal from audit is the cheapest risk assessment you will ever get; spend it.

Break-glass and time-boxed exceptions

A gate with no escape hatch will be ripped out the first time it blocks an emergency fix during an incident. So design the escape hatch deliberately, and make it *self-closing*.

- **Break-glass for emergencies.** A pre-defined, audited path to deploy past the gate during a declared incident — a specially labeled namespace, a signed break-glass approval, or an emergency-only policy variant. It must fire a loud, high-severity alert (a break-glass deploy is a security event by definition), be usable only by a small on-call group, and be reviewed after every use. The goal is not to prevent break-glass — you *want* engineers to be able to save prod — but to make it rare, visible, and accounted for.
- **Time-boxed, risk-accepted exceptions.** For known gaps ("vendor image X can't have our provenance yet"), a documented exception with an **owner and an expiry date**. The critical design choice is that exceptions *expire*: encode them as objects the gate reads (e.g., a policy exception CRD, or an allowlist entry with an `expiresAt` field) and report on expiry, so a temporary hole does not become

permanent. An exception without an expiry is just a permanent hole with better paperwork. The metric to watch is **outstanding exceptions and their age** — a rising line is your policy quietly rotting.

Fail-open vs fail-closed

The gate depends on services that can fail: the admission webhook itself, Fulcio/Rekor (or your private instances), the registry it fetches attestations from, the policy service. When the gate *cannot decide*, does it admit or deny? This is the availability-vs-security decision, and Chapter 8 framed it: the honest default for a security control is **fail-closed** — if you cannot prove the artifact meets policy, do not run it.

```
flowchart TD
    req["Pod create request"] --> ev{"Gate can  
evaluate policy?"}
    ev -->|"yes, compliant"| admit["ADMIT (pinned digest)"]
    ev -->|"yes, non-compliant"| deny["DENY"]
    ev -->|"no: verifier/log/  
registry down"| fp{"Fail policy?"}
    fp -->|"prod tier  
failurePolicy: Fail"| deny
    fp -->|"dev tier  
failurePolicy: Ignore"| admitOpen["ADMIT unverified  
(alert loudly)"]
    style deny fill:#b30000,color:#ffffff
    style admitOpen fill:#e0a030
```

But fail-closed has a brutal corollary: **if the gate is down, no deploy anywhere succeeds**. A crashing admission webhook with `failurePolicy: Fail` can wedge your entire platform — you cannot even deploy the fix to the gate. So fail-closed is only *safe* if the gate infrastructure is highly available: multiple replicas across zones, tight `webhookTimeoutSeconds`, a scoped `namespaceSelector` so the webhook never intercepts its own control-plane namespaces, and monitoring/paging on gate health. The defensible pattern is **tiered**: `failurePolicy: Fail` (fail-closed) for prod-critical namespaces where security dominates, `failurePolicy: Ignore` (fail-open, but *always* alerting) for low-risk dev namespaces where a gate outage should not block iteration. Never fail open silently — a fail-open admit under a gate outage is exactly the window an attacker waits for, so it must page.

This is why the gate is a **tier-0 service**: it sits in the pod-scheduling path, and its availability requirements are a direct consequence of running it fail-closed. You cannot demand fail-closed security without funding gate HA; the two decisions are one decision.

Tiered and graduated policy

The strict prod policy above is unachievable for most of your fleet on day one, and *permanently* unachievable for some of it. A third-party vendor image will never carry a SLSA L3 provenance attestation from *your* build platform — you did not build it. Requiring it uniformly means either blocking half your workloads or granting so many exceptions that the policy is meaningless. The answer is **tiered policy: required checks as a function of environment × criticality × artifact source**.

```
flowchart TD
    start["Image at gate"] --> src{"Source?"}
    src -->|"first-party  
(we built it)"| env{"Environment /  
criticality?"}
    env -->|"third-party  
(vendor/upstream)"| tp["Signature (if any)  
+ vuln-scan attestation  
+ from approved registry  
(cannot require our provenance)"]
    tp -->|"prod tier-0"| strict["Keyless identity + SLSA L3  
provenance from approved repo  
+ SBOM + scan(no reachable KEV)  
+ Rekor + digest pin"]
    strict -->|"prod tier-1/internal"| mid["Keyless identity + provenance  
+ SBOM + scan (warn on vulns)"]
    mid -->|"dev / staging"| relaxed["Signature present  
+ digest pin  
(audit provenance)"]
    style strict fill:#1b5e20,color:#ffffff
    style tp fill:#e0a030
```

The tiers, concretely:

- **First-party, prod tier-0:** the full worked policy — identity, SLSA L3 provenance from an approved source, SBOM, scan with no reachable KEV vulns, Rekor, digest pinning. No exceptions without break-glass.
- **First-party, internal/lower-risk:** identity + provenance + SBOM, but scan results *warn* rather than block, and the SLSA-level bar may be L2. Lets internal tooling ship while you mature it.

- **Dev/staging:** require a signature and digest pinning (so you are always running known bytes), but only *audit* provenance. Dev velocity dominates; you still get the inventory.
- **Third-party images:** you cannot demand your own provenance, so require what is achievable — a signature *if the vendor publishes one* (many now do via Sigstore), a vulnerability-scan attestation *you* produce at ingest, and that the image comes from an approved registry/namespace. This tier is where honest engineering matters: do not pretend a vendor image meets L3; state its weaker guarantees explicitly and compensate elsewhere (network policy, runtime restrictions).

Tiered policy is not a compromise of the thesis; it is how the thesis survives contact with a real fleet. A uniform policy that everyone routes around with exceptions enforces nothing. A tiered policy that is *actually enforced at each tier* enforces a lot. The engineering judgment is in setting the tiers honestly and ratcheting them upward over time — this quarter dev is signature-only; next quarter it requires provenance-in-audit; the quarter after, provenance-in-enforce. The gate's audit metrics tell you when a tier is ready to ratchet.

Metrics: proving the gate works

A gate you cannot measure is a gate you cannot trust or defend in an audit. Instrument it as a first-class control (this ties to Book 8, Chapter 8 on measuring a supply-chain program):

- **Coverage** — % of running workloads, across all clusters and namespaces, that pass through an *enforcing* gate. The uncomfortable number: what fraction of prod is actually gated vs merely audited or exempt? This is the headline metric.
- **Policy pass rate** — % of admission decisions that pass on the first try, trended per tier and per team. A team stuck at 60% is a team whose pipeline needs help.
- **Deny events** — every enforce-mode denial, as a *security signal*. A deny at admission means something tried to run an image that failed policy; that is either a misconfigured deploy or an attack, and both deserve a look. Feed deny events into detection/IR (Book 8): a spike in denials, or a denial for an image whose signer identity is wrong, is an incident indicator.

- **Would-deny (audit) volume** — while rolling out, the count of images that *would* be denied, driven toward zero before enforce. In steady state this should stay near zero; a rise means new non-compliant artifacts are appearing.
- **Outstanding exceptions and their age** — how many risk-accepted holes are open and how old. Rising or aging = policy decay.
- **Break-glass uses** — count and review status. Each is a security event.

The ultimate test — the one to put in the exec summary — is a question, not a metric: **can an unsigned, unverified, or unauthorized-source image reach prod?** If the honest answer requires "well, if someone did X and Y...", you have a bypass to close. In a well-designed system the answer is *structurally no*: every path to a running Pod passes through admission, admission is enforcing and fail-closed for prod, and the only escapes are loud, audited, and self-expiring break-glass. That structural "no" is the deliverable of this entire book.

Capstone: bringing Books 3-5 together

Step back and look at the whole machine. The three books you have worked through describe two halves of one loop, closed by the gate.

```

flowchart LR
    subgraph produce["PRODUCE – paved-road build platform (Book 4)"]
        src["Approved source repo  
(protected main)"] --> builder["Hosted isolated builder  
SLSA L3"]
        builder --> img["Image @ sha256"]
        builder --> prov["SLSA provenance  
(Book 4 Ch3)"]
        builder --> sbom["SBOM (Book 3)"]
        builder --> scan["Scan/VEX attestation  
(Book 2, Book 3 Ch6)"]
        img --> sign["Keyless sign +  
attest (Book 5 Ch3-4,6)"]
        prov --> sign
        sbom --> sign
        scan --> sign
    end
    sign --> store["Registry (OCI) +  
Rekor transparency log  
(Book 5 Ch5)"]
    store --> gate

    subgraph verify["VERIFY – deployment gate (this chapter)"]
        gate["Admission gate  
vs central policy  
(Book 8 Ch4)"]
    end
    gate -->| "deny" | rej["Rejected"]
    gate -->| "admit by digest" | run["Runs in prod"]
    run -->| "deny events" | ir["Detection / IR (Book 8)"]
    style gate fill:#1b5e20,color:#ffffff

```

Trace one artifact through it. A commit lands on a protected `main` in an approved repo (Book 7 on source security). The **paved-road build platform** (Book 4, Chapter 10) builds it in an isolated, hosted builder — SLSA Build L3 — and emits, alongside the image, a **SLSA provenance** attestation (Book 4, Chapter 3), an **SBOM** (Book 3), and a **scan/VEX** attestation (Books 2 and 3). The platform **signs** the image and wraps each attestation in a DSSE envelope using **keyless** signing (Book 5, Chapters 3–4 and 6), and every signature is recorded in **Rekor** (Chapter 5). The image and its attestations sit in the registry, content-addressed by digest. Then someone tries to run it — via GitOps, `kubectl`, or an operator — and the request hits the **deployment gate** (this chapter), which evaluates the artifact against **central policy** (Book 8, Chapter 4): expected keyless identity, provenance from an approved source at the required SLSA level, the required attestation set present, Rekor

inclusion, no reachable KEV vulns — and admits *by digest* or denies. Deny events flow to detection and IR (Book 8).

That is the realized supply-chain security architecture for a distributed backend. The build platform and the gate are the two halves — **produce and verify** — and neither is worth much without the other. A build platform that emits perfect attestations nothing checks is theater. A gate with a strict policy and no build platform producing the attestations it demands blocks everything. The two are designed together, share one vocabulary (in-toto predicates, Sigstore identities, digests), and close the loop: the platform produces exactly the claims the gate is configured to verify, and the gate refuses anything the platform did not vouch for.

Distributed-systems lens

The deployment gate is a **fleet-wide control**, and everything hard about it is a distributed-systems problem.

- **One policy, uniformly enforced, no bypass.** The entire value proposition is that *no service bypasses it*. In a world of hundreds of services and dozens of teams, you cannot rely on each team to verify correctly; you centralize verification at admission so the guarantee holds uniformly whether a team opted in or not. A per-service check that a team can forget is not a fleet control; a platform admission gate that intercepts every Pod is.
- **Central policy, distributed enforcement, per-tier variation.** Policy is authored centrally (Book 8, Chapter 4) — one source of truth for "what prod requires" — but *enforced* at every cluster's admission point. The policy is not uniform in its *requirements* (tiers by environment × criticality × source) but it is uniform in its *authority*: teams cannot weaken it locally, only central governance ratchets it.
- **The gate is in the critical path, so it is tier-0.** Admission runs synchronously in the API server's request path; a slow gate slows every deploy and every scale-up, and a down fail-closed gate blocks all of them. This forces real availability engineering: multi-replica, multi-zone, tight timeouts, scoped selectors that exclude control-plane namespaces, and health-based paging. You do not get fail-closed security for free — you buy it with HA.

- **Verify-by-digest end to end.** Content-addressing is what makes verification sound across the distributed hops from build to registry to kubelet. The gate resolves and pins the digest so the thing verified is the thing that runs; tags — mutable pointers — are a TOCTOU trap at every hop.
- **The gate closes the loop with the build platform.** The paved road (Book 4, Chapter 10) produces the attestations; the gate consumes them. Designed together, the two halves make "only verified artifacts run in production" a structural invariant rather than a hope.
- **Deny events are a detection feed.** Because the gate sees every deploy, its denials are a high-signal, low-noise security telemetry source (Book 8): a deploy blocked for a wrong signer identity is an attempted intrusion or a serious misconfiguration, and either way IR wants to know.

Key takeaways

- **The gate is the enforcement point.** All of Books 3–5's signing, provenance, and attestation machinery is realized only where a gate *refuses to run* artifacts that fail policy. Produce AND verify; a claim that gates nothing is theater.
- **Admission is the non-bypassable backstop.** Gate at registry ingest and in CD for fast feedback, but center enforcement at runtime admission — it is the only point every path to a running Pod must cross, and it is outside the CI/CD trust domain that upstream gates depend on.
- **Policy is a conjunction, and identity + digest are the load-bearing clauses.** Verifying "validly signed" is nearly useless; verify the *expected* signer identity and pin the *exact* digest. Assemble the rest — provenance vs source/builder/SLSA level, required attestations, Rekor inclusion, vuln/VEX posture — into one concrete policy.
- **Use real tools, real syntax.** `policy-controller` `ClusterImagePolicy`, Kyverno `verifyImages` + attestations, OPA/Gatekeeper + Ratify, Connaisseur, Google Binary Authorization — all enforce expected-identity signatures and required attestations by digest at admission.

- **Roll out audit → enforce, per tier.** Dry-run to inventory would-deny, drive it to zero, then enforce incrementally. Build in self-expiring exceptions and loud, audited break-glass. Fail-closed for prod — but only if the gate is HA, because fail-closed plus a down gate wedges the whole platform.
- **Tier the policy** by environment × criticality × source. Strict for first-party prod tier-0; achievable checks (signature + your-own-scan) for third-party images you did not build. Tiering is how the thesis survives a real fleet; ratchet tiers upward using audit metrics.
- **Measure it and pass the ultimate test.** Coverage, pass rate, deny events (as a security signal), outstanding exceptions, break-glass uses. The exec-summary question: *can an unsigned/unverified/unauthorized-source image reach prod?* A well-designed gate makes the answer structurally no.
- **Distributed-systems reality:** one central policy, enforced uniformly at every cluster's admission, in the tier-0 pod-scheduling path, verifying by digest the attestations the paved-road build platform produced — the two halves of the loop, closed.

Further reading

- **sigstore/policy-controller** — `ClusterImagePolicy` reference, keyless authorities, attestation policies (CUE/Rego), and enforce/warn modes. <https://docs.sigstore.dev/policy-controller/overview/>.
- **Kyverno** — image verification (`verifyImages`), attestation checks and conditions, and `validationFailureAction` . <https://kyverno.io/docs/writing-policies/verify-images/> and <https://kyverno.io/policies/> (the "verify-image" samples).
- **OPA/Gatekeeper** and **Ratify** — Gatekeeper constraints plus Ratify as an external-data verification engine for signatures, SBOMs, and vulnerability reports. <https://open-policy-agent.github.io/gatekeeper/> and <https://ratify.dev/docs/>.
- **Connaisseur** — admission controller for container image signature verification (Sigstore/Notary). <https://github.com/sse-secure-systems/connaisseur>.
- **Google Binary Authorization** — attestation-based deploy gating on GKE/Cloud Run: policies, attestors, dry-run vs enforced modes. <https://cloud.google.com/binary-authorization/docs>.

- **AWS Signer** — managed signing for container images and Lambda code, and its verification model. <https://docs.aws.amazon.com/signer/>.
- **Cosign** — `cosign verify / cosign verify-attestation , --certificate-identity(-regexp) , --certificate-oidc-issuer , --type , --policy` . <https://docs.sigstore.dev/cosign/verifying/verify/>.
- **SLSA v1.0** — provenance predicate, build levels (L1-L3), and verification guidance. <https://slsa.dev/spec/v1.0/>.
- **in-toto attestation framework** and predicate types (provenance, SBOM, test, vuln). <https://github.com/in-toto/attestation>.
- **Kubernetes admission control** — validating/mutating admission webhooks and `failurePolicy` . <https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/> and <https://kubernetes.io/docs/reference/access-authn-authz/extensible-admission-controllers/>.
- **Flux** (source-controller Sigstore/keyless verification) and **Argo CD** deployment gating — GitOps verification of OCI artifacts and manifests (developed in Book 6, Chapter 7). <https://fluxcd.io/flux/components/source/> and <https://argo-cd.readthedocs.io/>.
- Cross-references: Book 5, Chapter 4 (keyless signing, expected identity), Chapter 5 (Rekor transparency), Chapter 6 (in-toto attestations), Chapter 8 (verification checklist, fail-open/ closed), Chapter 9 (key management/HA for the gate's trust roots); Book 4, Chapter 3 (SLSA provenance), Chapter 10 (the secure paved-road build platform); Book 3 (SBOMs), Chapter 6 (VEX); Book 2, Chapter 7 (reachability/prioritization); Book 6, Chapters 5–7 (admission control, policy engines, GitOps verification); Book 8, Chapter 4 (central policy) and Chapter 8 (metrics, detection, IR).